

HOGESCHOOL PXL
PXL-DIGITAL

Java Advanced
Introductie tot Spring Boot

Author
Nele CUSTERS

2 december 2024

Inhoudsopgave

1 Kennismaking met Spring Boot	1
1.1 Enterprise toepassingen met Java	1
1.2 Bootstrapping van een eenvoudige Spring Boot applicatie	4
1.2.1 Gebruik van Spring Initializr	4
1.3 Het demo-project uitvoeren	6
1.4 De Maven POM file	8
1.5 Inversion of Control (IoC) en dependency injection (DI)	9
1.5.1 Wat is Inversion of Control (IoC)?	9
1.5.2 Spring Beans	12
1.5.3 Application context	13
1.5.4 IoC in werking	14
2 REST	19
2.1 HTTP-verzoekmethoden	19
2.2 Spring Boot Starter Web	20
2.3 De RestController	22
2.4 MusicPlaylist	24
2.4.1 Een liedje toevoegen aan een playlist	24
2.4.2 De playlist opvragen	29
2.4.3 Liedjes van één genre	32
2.4.4 Gegevens van een liedje aanpassen	34
2.4.5 Een liedje verwijderen	37
3 Collections: Map	43
3.1 Generieke klasse HashMap	43
3.2 De interface Map	43
3.3 Gebruik van HashMap	44
3.3.1 Voorbeeld 1	44
3.3.2 Voorbeeld 2	44
4 Foutafhandeling	47
4.1 Compile-time vs runtime errors	47
4.2 First catch	49
4.3 Java exception hiërarchie	50
4.3.1 Errors	51
4.3.2 Runtime exceptions	53
4.3.3 Checked exceptions	55
4.4 Multi-catch blok en finally	57

4.5	Eigen unchecked exceptions	59
5	Spring Validation	62
6	Unit testen met JUnit	68
6.1	JUnit	68
6.2	De klasse House	69
6.3	Klasse met de unit testen	71
6.4	Unit test voor de constructor	71
6.5	Unit test voor een setter	72
6.6	@BeforeEach	74
6.7	Unit test voor een berekende waarde	74
6.8	Testen van exceptions die gegoooid worden	75
6.9	Overzicht van alle testen voor de klasse House	75
7	Lambda expressies en Streams	78
7.1	Functionele interfaces	78
7.1.1	De interface StringConverter	78
7.1.2	Function<T, R>	80
7.1.3	Consumer<T>	81
7.1.4	Predicate<T>	82
7.2	External en internal iterators	83
7.3	Intermediate en terminal operations	85
7.3.1	Terminal operation .toList()	85
7.3.2	Intermediate operation .filter()	86
7.3.3	Terminal operation .forEach()	88
7.3.4	Intermediate operation .map()	89
7.3.5	Intermediate operation .sorted()	90
7.3.6	Intermediate operation .distinct()	90
7.3.7	Intermediate operation .limit()	91
7.3.8	Intermediate operation peek()	91
7.3.9	Terminal operation .count()	92
7.4	Intstream, LongStream en DoubleStream	92
7.4.1	sum()	92
7.4.2	range() en rangeClosed()	92
7.4.3	min(), max() en average()	93
7.5	Method reference	94
7.5.1	Static methods	94
7.5.2	Instance method (unbounded)	94
7.5.3	Instance method (bounded)	94
7.5.4	Constructor	95
8	3-lagen architectuur: introductie van Spring Data JPA	97
8.1	Een voorbeeld toepassing	98
8.2	Gegevens opslaan en ophalen	100
8.2.1	Entiteitsklasse Superhero	100
8.2.2	Repository	102
8.2.3	Service-layer	103
8.2.4	RestController	107

8.3	URL context path	110
8.4	API documentation	110
8.5	Frontend	113
9	Spring Data JPA	114
9.1	Wat is ORM?	114
9.2	Wat is JPA?	115
9.3	Datasource	116
9.4	De Entity klasse	119
9.5	Repository	121
9.6	Entity lifecycle	122
9.7	Queries	123
9.8	Relationships	124
9.8.1	Many-to-many bidirectional relationship	124
9.9	Transactions	128
10	Database relaties met Spring Data JPA	130
10.1	Relationships	130
10.1.1	One-to-One associatie	130
10.1.2	Many-to-one relatie	133
10.1.3	Many-to-many relaties	139
10.2	N + 1 query problem (optioneel)	141
10.3	Lazy loading en OSIV	146

Hoofdstuk 1

Kennismaking met Spring Boot

Learning goals

De junior-collega

1. kan beschrijven wat het Spring framework is
2. kan beschrijven wat Spring Boot is
3. kan de kenmerken van een enterprise toepassing benoemen
4. kan uitleggen wat inversion of control (IoC) is
5. kan uitleggen wat dependency injection is
6. kan beschrijven wat een Spring Bean is
7. kan beschrijven wat de Application Context is
8. kan een nieuwe Spring Boot applicatie aanmaken en opstarten
9. kan Spring Beans aanmaken en gebruiken

1.1 Enterprise toepassingen met Java

Spring Boot is een framework om enterprise toepassingen te bouwen met Java. Enterprise toepassingen zijn toepassingen die bedrijven bouwen of laten bouwen voor hun klanten en medewerkers. Enterprise toepassingen worden ontwikkeld om bedrijfsprocessen te ondersteunen.

Enterprise toepassingen hebben specifieke kenmerken omwille van de complexe aard en specifieke eisen van grootschalige bedrijfssystemen. Enkele belangrijke kenmerken van enterprise toepassingen vanuit het oogpunt van een ontwikkelaar zijn:

- **Schaalbaarheid (scalability):** Enterprise applicaties moeten een groot aantal gebruikers en gegevens aankunnen. Ontwikkelaars moeten de architectuur van de toepassing zodanig ontwerpen dat schalen mogelijk is om een groeiend aantal gebruikers en datavolumes aan te kunnen.
- **Betrouwbaarheid en hoge beschikbaarheid:** Van enterprise applicaties wordt verwacht dat ze 24/7 beschikbaar zijn en snelle responstijden hebben. Ontwikkelaars moeten oplossingen voorzien om problemen en downtime te voorkomen. Ze moeten de betrouwbare werking van de applicatie kunnen garanderen.
- **Beveiliging:** Enterprise applicaties verwerken gevoelige bedrijfsgegevens, waaronder financiële informatie, klantgegevens,... Ontwikkelaars moeten robuuste beveili-

gingsmaatregelen implementeren, zoals authenticatie, autorisatie, versleuteling van gegevens, audittrails, om de gegevens te beschermen tegen ongeautoriseerde toegang.

- **Integratie:** Enterprise applicaties moeten vaak worden geïntegreerd met verschillende bestaande systemen zoals databanken en externe services. Voorbeelden van externe services zijn bijvoorbeeld betaalsystemen en boekhoudpakketten. Ontwikkelaars moeten zorgen voor een veilige en betrouwbare gegevensuitwisseling tussen verschillende enterprise applicaties.
- **Aanpasbaarheid en flexibiliteit:** Enterprise applicaties worden gebruikt door gebruikers uit diverse afdelingen en teams binnen een organisatie, elk met hun eigen unieke workflows en eisen. Ontwikkelaars moeten applicaties bouwen die kunnen worden aangepast en geconfigureerd om aan deze specifieke behoeften te voldoen. Enterprise applicaties ondergaan vaak veranderingen en updates op basis van veranderende zakelijke behoeften. Daarnaast veranderen de workflows en eisen van de gebruikers ook. Ontwikkelaars moeten verandering effectief beheren door versiebeheer, wijzigingstracering en terugrolmechanismen te implementeren. Afhankelijk van de sector moeten enterprise applicaties voldoen aan specifieke wettelijke normen die doorheen de tijd kunnen veranderen.
- **Lange levenscyclus:** Enterprise applicaties hebben meestal een langere levenscyclus dan andere soorten software. Ontwikkelaars moeten zorgen voor voortdurend onderhoud, updates en verbeteringen.
- **Samenwerking en documentatie:** Aangezien meerdere ontwikkelaars en teams werken aan verschillende delen van een enterprise applicatie, zijn duidelijke codeer-richtlijnen, versiebeheer en samenwerkingstools essentieel.
- **Testen en kwaliteitsborging:** Grondig testen is essentieel voor enterprise applicaties om bugs, prestatieproblemen en kwetsbaarheden in de beveiliging te identificeren en aan te pakken. Ontwikkelaars moeten unit test, integratietesten en gebruikersacceptatietesten implementeren. Al deze testen worden geautomatiseerd en bij iedere aanpassing aan de code uitgevoerd.
- **Gebruikerservaring (UX):** Hoewel functionaliteit cruciaal is, is een positieve gebruikerservaring ook belangrijk. Ontwikkelaars moeten streven naar intuïtieve interfaces en workflows zodat de gebruikers productief zijn.

Over het algemeen vereist de ontwikkeling van enterprise applicaties een uitgebreid begrip van bedrijfsprocessen, technische expertise en het vermogen om functionaliteit, preformantie, beveiliging en gebruikerservaring in evenwicht te brengen om te voldoen aan de unieke behoeften van grote organisaties.

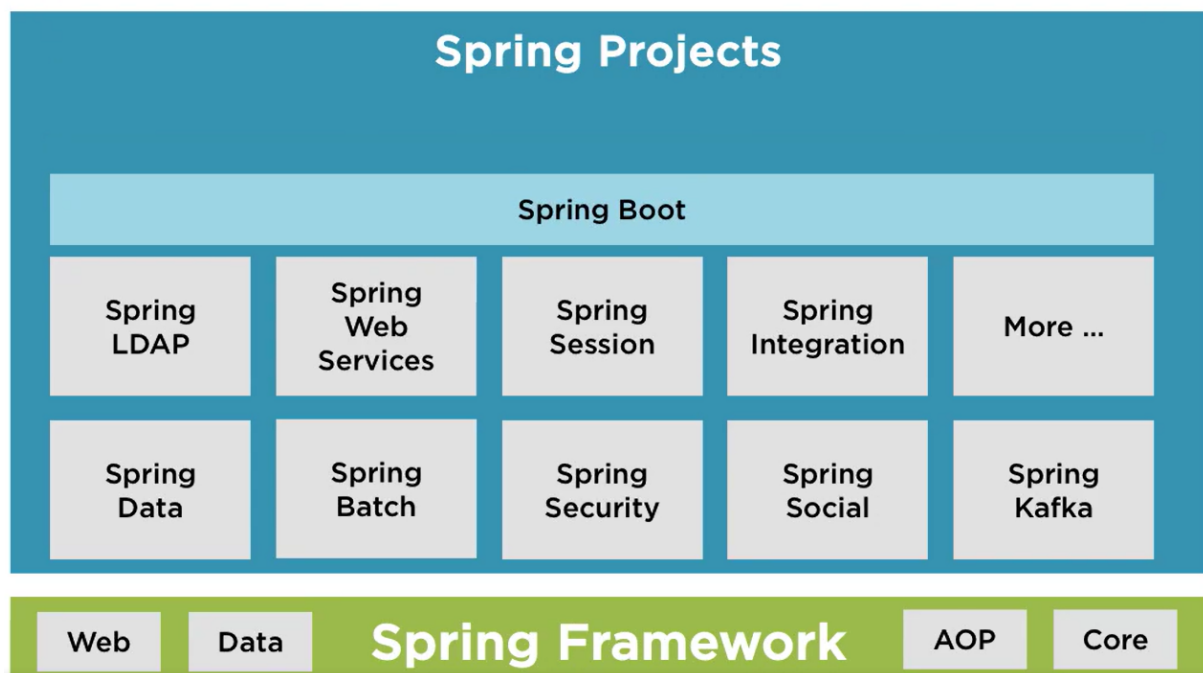
In 1999 besliste Sun Microsystems om de programmeertaal Java uit te bereiden om het ontwikkelen van enterprise applicaties te vergemakkelijken. Met de lancering van J2EE (Java 2 Enterprise Edition) en de applicatie servers om de J2EE-toepassingen op te deployen ontstond een schaalbaar en betrouwbaar platform. J2EE, dat werd hernoemd naar Java EE (Java Platform, Enterprise Edition), verwijst naar een verzameling specificaties voor het ontwikkelen van enterprise applicaties. Het bestaat uit een reeks standaarden en API's die ontwikkelaars gebruiken om de enterprise-software te implementeren.

Wanneer we spreken over "J2EE-specificaties" of "Java EE-specificaties", dan verwijst dit naar de reeks specificaties die samen Java EE vormen. Deze specificaties zijn de beschrijvingen van hoe bepaalde componenten en functionaliteiten in een enterprise Java-applicatie moeten worden geïmplementeerd. Sun Microsystems en later Oracle, dat in de 2010 Sun Microsystems overnam, zorgen steeds voor een bruikbare implementatie van de specificaties.

Laten we het versturen van mails als voorbeeld nemen. In bijna iedere enterprise applicatie moet het mogelijk zijn om op een eenvoudige manier mails te versturen. Daarom werden de JavaMail API Design Specifications uitgeschreven. Dit kan je zien als een uitgebreide analyse van de vereisten. De reference implementation is voorzien door Oracle zelf. Naast de reference implementation zijn er nog andere spelers op de markt die hun implementatie, vaak met bijkomende functionaliteiten, aanbieden. Voor de JavaMail API kan je bijvoorbeeld kiezen uit Apache Geronimo JavaMail, WildFly (JBoss) JavaMail, Google App Engine JavaMail, ...

Het Spring framework ontstond in 2003 als reactie op de complexiteit van de JEE-specificaties. Sommigen zien Spring als een concurrent van Java EE en zijn moderne opvolger Jakarta EE. Maar als je Spring leert kennen zal je merken dat Spring zorgvuldig individuele specificaties uit Jakarta EE integreert. Waar de ontwikkeling van Enterprise JavaBeans (EJB's) in een JEE toepassing eerder omslachtig was, bood Spring een eenvoudigere benadering met Spring Beans. Toch blijft het Spring framework complex omwille van de configuratie. Deze configuratie gebeurde initieel door (veel) XML bestanden.

Spring Boot is een open-source Java framework dat gebouwd is boven op het Spring Framework. Het biedt alles om enterprise applicaties te ontwikkelen in Java. Met Spring Boot kan je op een eenvoudigere en snellere manier een toepassing te creëren, configureren en uitvoeren.



Spring Boot biedt verschillende voordelen voor ontwikkelaars.

- Gemakkelijker en sneller creëren en implementeren van enterprise applicaties.

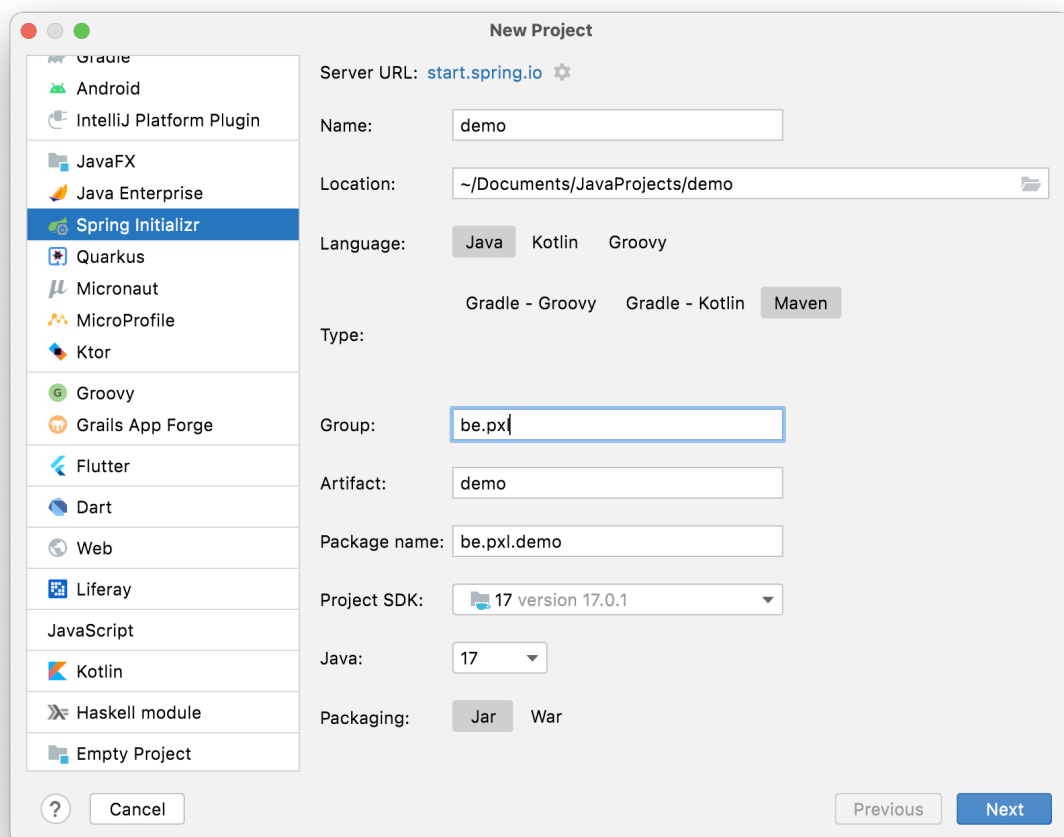
- Minder of bijna geen configuratie.
- Eenvoudiger te leren framework.
- Verhoogt de productiviteit.

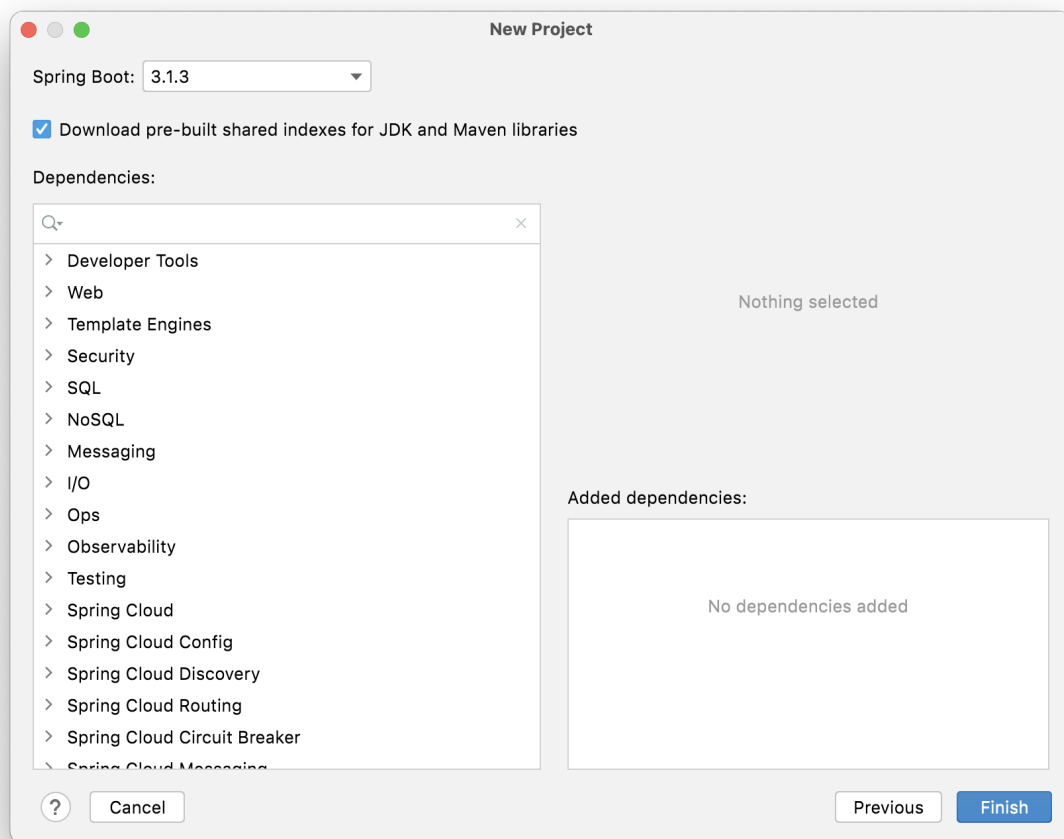
1.2 Bootstrapping van een eenvoudige Spring Boot applicatie

1.2.1 Gebruik van Spring Initializr

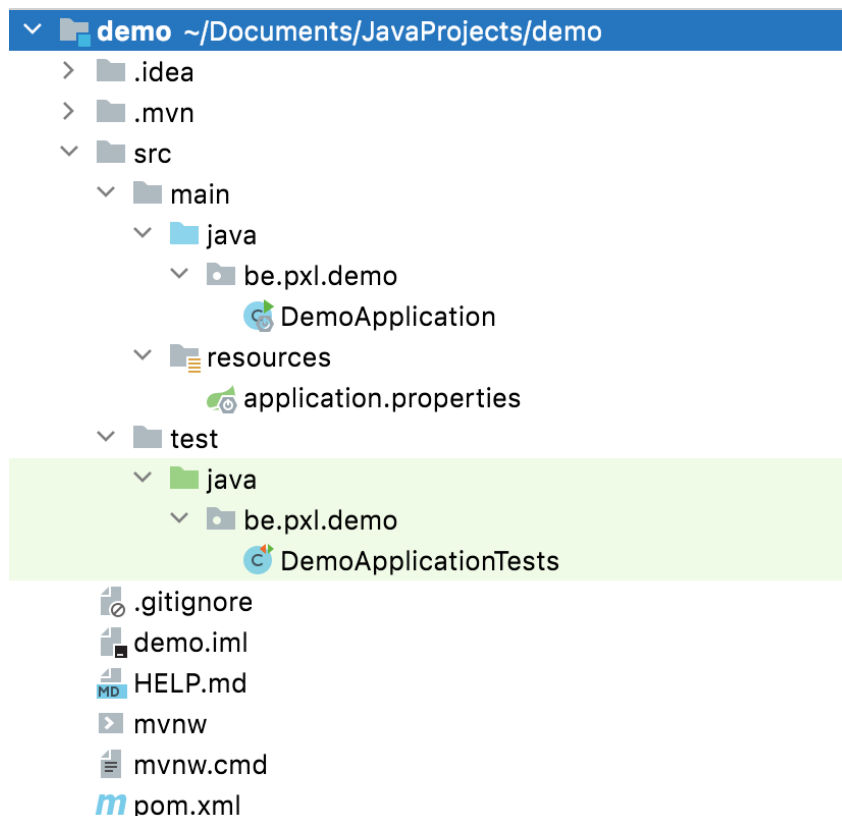
Spring Initializr is een webtoepassing waarmee je een Spring Boot-project kunt genereren. De URL voor deze webtoepassing is <https://start.spring.io/>. Je kunt de benodigde configuratie selecteren zoals de buildtool, programmeertaal, versie van het Spring Boot-framework en eventuele afhankelijkheden voor je project. IntelliJ IDEA Ultimate biedt de Spring Initializr-projectwizard die integreert met de Spring Initializr API om je project rechtstreeks in de IDE te genereren en te importeren.

We prefer Java 21 for this course because Java 21 is the newest LTS(Long-Term Support)-Version and Spring Boot version 3.3.3. The screenshots are not always updated to these most recent versions.





Wanneer je Maven als buildtool kiest, krijgt je nieuw project de typische folder-structuur van Maven.



De eigenlijke broncode komt in de src folder. De testen, die we later leren schrijven, komen in de test folder terecht.

Oefening 1.1. Maak het demo-project aan zoals in de screenshots. Je kan de wizard van IntelliJ IDEA Ultimate of de webtoepassing <https://start.spring.io/> gebruiken.

1.3 Het demo-project uitvoeren

Het startpunt van een Spring Boot toepassing is de klasse met de main-methode. Deze klasse is geannoteerd met **@SpringBootApplication**. Je vindt deze klasse terug in de folder `/src/main/java` in het package `be.pxl.demo`. Spring Boot biedt heel veel functionaliteit aan in de vorm van annotaties om het werk van de ontwikkelaars te vereenvoudigen.

```
package be.pxl.demo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class DemoApplication {

    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }
}
```


1.4 De Maven POM file

POM staat voor Project Object Model. Omdat we voor Maven hebben gekozen als buildtool van onze Spring Boot-toepassing staan hier de project-coördinaten in. Alle dependencies die door ons project worden gebruikt staan hier opgelijst en worden door Maven gedownload. Een dependency is een externe module (library) die een Java-project nodig heeft om correct te kunnen functioneren.

```
?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  ↪ xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  ↪ xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
  ↪ https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.1.3</version>
    <relativePath/> <!-- lookup parent from repository -->
  </parent>
  <groupId>be.pxl</groupId>
  <artifactId>demo</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <name>demo</name>
  <description>demo</description>
  <properties>
    <java.version>17</java.version>
  </properties>
  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter</artifactId>
    </dependency>

    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-test</artifactId>
      <scope>test</scope>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <plugin>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-maven-plugin</artifactId>
      </plugin>
    </plugins>
```

```
</build>

</project>
```

De spring-boot-starter-parent is het basisproject dat alle standaardconfiguratie biedt voor op Spring gebaseerde toepassingen. Hier kies je de versie van Spring Boot.

Voor grote projecten is het beheren van afhankelijkheden (dependencies) niet altijd eenvoudig. Spring Boot lost dit probleem op door bepaalde afhankelijkheden samen te bundelen. Deze groepen van afhankelijkheden worden starters genoemd. Alle Spring Boot starters volgen dezelfde richtlijnen voor hun naamgeving. Ze beginnen allemaal met spring-boot-starter-*, waarbij * aangeeft welke functionaliteiten de starter aanbiedt.

spring-boot-starter-test (met scope test) is de starter voor het testen van Spring Boot-toepassingen met o.a. JUnit Jupiter, Hamcrest en Mockito.

1.5 Inversion of Control (IoC) en dependency injection (DI)

1.5.1 Wat is Inversion of Control (IoC)?

In een traditioneel object-georiënteerd Java-programma maken klassen direct hun afhankelijkheden aan, wat leidt tot een nauwe koppeling tussen de klasse en de associaties. Hier is een eenvoudig voorbeeld zonder IoC:

```
public class Engine {
    public void start() {
        System.out.println("Engine started");
    }
}

public class Car {
    private Engine engine;

    public Car() {
        // Car creates its own Engine instance
        engine = new Engine();
    }

    public void drive() {
        engine.start();
        System.out.println("Car is driving");
    }
}

public class Main {
    public static void main(String[] args) {
        Car car = new Car();
    }
}
```

```
        car.drive();
    }
}
```

In bovenstaand voorbeeld is er een nauwe koppeling tussen de Car- en de Engine-klasse. IoC verwijst naar het omkeren van traditionele programmacontrolestructuren om meer flexibiliteit en eenvoudiger testbaarheid van applicaties te bereiken. Inversion of Control (IoC) is één van de basisprincipes van het Spring framework. Bij het software engineering principe Inversion of Control wordt het creëren en beheren van objecten de verantwoordelijkheid van een apart onderdeel binnen het programma. Dit onderdeel noemen we een container. Binnen Spring Boot is deze container een object van de klasse Application Context. We spreken daarom kortweg over de application context om deze container binnen het programma te benoemen.

We zullen eerste en vooral de Engine flexibeler maken door het te definiëren als een interface. Nu kunnen we verschillende implementaties van Engine hebben.

```
public interface Engine {
    void start();
}

public class PetrolEngine implements Engine {
    public void start() {
        System.out.println("Petrol engine started");
    }
}

public class ElectricEngine implements Engine {
    public void start() {
        System.out.println("Electric engine started");
    }
}
```

De Car-klasse is nu afhankelijk van de Engine-interface en verwacht dat deze geïnjecteerd wordt, in plaats van zelf de motor aan te maken.

```
public class Car {
    private Engine engine;

    // Constructor injection
    public Car(Engine engine) {
        this.engine = engine;
    }

    public void drive() {
        engine.start();
        System.out.println("Car is driving");
    }
}
```

Spring maakt gebruik van XML configuratie en/of annotaties om te configureren welke objecten aangemaakt en beheerd moeten worden.

```
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.ConfigurableApplicationContext;
import org.springframework.context.annotation.Bean;

@SpringBootApplication
public class DemoIoCApplication {

    public static void main(String[] args) {

        ConfigurableApplicationContext applicationContext =
            ↪ SpringApplication.run(DemoIoCApplication.class, args);
        Car myElectricCar = (Car)
            ↪ applicationContext.getBean("myElectricCar");
        myElectricCar.drive();
        Car myOldCar = (Car) applicationContext.getBean("myOldCar");
        myOldCar.drive();
    }

    @Bean
    public Engine electricEngine() {
        return new ElectricEngine();
    }

    @Bean
    public Engine petrolEngine() {
        return new PetrolEngine();
    }

    @Bean
    public Car myElectricCar() {
        return new Car(electricEngine());
    }

    @Bean
    public Car myOldCar() {
        return new Car(petrolEngine());
    }
}
```

Dit principe leidt tot een losse koppeling tussen Car- en Engine-klasse en maakt het systeem flexibeler en eenvoudiger te testen.

Naast de annotatie @Bean kunnen we ook annotatie @Component gebruiken om beans (objecten) aan de Spring-container toe te voegen. Let wel op, er zijn belangrijke verschillen in hoe en wanneer je ze gebruikt.

@Component is een annotatie die gebruikt wordt om een klasse automatisch als een Spring-bean te markeren. Spring maakt dan zelf een object van deze klasse en beheert het in de Spring-container.

Als we de klasse Car annoteren met @Component, dan hoeven we zelf geen Car-object aan te maken.

```
@Component
public class Car {
    private Engine engine;

    // Constructor injection
    public Car(Engine engine) {
        this.engine = engine;
    }

    public void drive() {
        engine.start();
        System.out.println("Car is driving");
    }
}
```

```
@SpringBootApplication
public class DemoIoCApplication {

    public static void main(String[] args) {
        ConfigurableApplicationContext applicationContext =
            ↪ SpringApplication.run(DemoIoCApplication.class, args);
        Car myCar = applicationContext.getBean(Car.class);
        myCar.drive();
    }

    @Bean
    public Engine myEngine() {
        return new ElectricEngine();
    }
}
```

1.5.2 Spring Beans

Spring Beans zijn componenten die volledig worden beheerd door het Spring Boot framework. Wanneer een klasse gebruik wil maken van de functionaliteiten van een dergelijke Spring Bean, zal Spring Boot ervoor zorgen dat de instantie van de Spring Bean beschikbaar is voor de betreffende klasse.

De Spring Boot `CommandLineRunner` is een interface die het mogelijk maakt om een stukje code uit te voeren zodra de Spring Boot applicatie geïntialiseerd is.

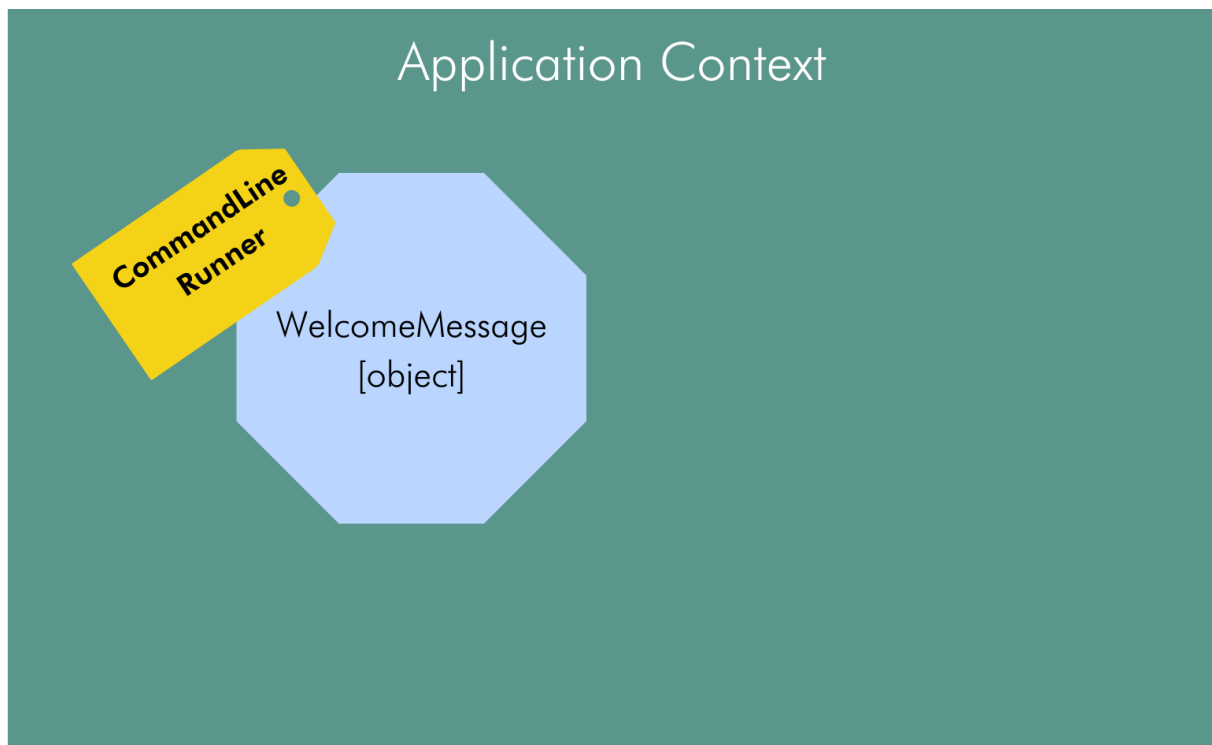
```
@Component
public class WelcomeMessage implements CommandLineRunner {

    @Override
    public void run(String... args) throws Exception {
        System.out.println("Welcome to Java Advanced");
    }
}
```

Zodra je de `CommandLineRunner` interface implementeert in een klasse kan je de `run`-methode overschrijven. In de `run`-methode plaats je de code die uitgevoerd moet worden als de applicatie opstart. Zodra de Spring Boot applicatie opstart worden een aantal initialisatie-fasen doorlopen. Eerst wordt de **application context** aangemaakt en alle Spring beans worden geladen. Als de applicatie volledig is geïntialiseerd wordt de `run`-methode van de `CommandLineRunner` automatisch door het Spring boot framework aangeroepen.

1.5.3 Application context

De application context is een slimme doos waarin alle beans, informatie en instellingen voor de Spring Boot-toepassing worden bewaard. Je kunt de application context beschouwen als het brein van de Spring Boot-applicatie dat alles coördineert en beschikbaar maakt voor de verschillende onderdelen van het programma.



Figuur 1.1: De Application Context

Als de Spring Boot-applicatie opstart doorlopen we verschillende fasen in de initialisatie. Nadat de Application Context is aangemaakt, worden alle Spring beans geladen en daarna wordt de run-methode van de klassen die de CommandLineRunner interface implementeren uitgevoerd.

De annotatie `@SpringBootApplication` zet eigenlijk 3 features van Spring Boot in werking.

- Activeert het auto-configuratie mechanisme van Spring Boot (`@EnableAutoConfiguration`)
- Activeert het scannen naar componenten met annotatie `@Component` (en ook `@RestController`, `@Service` en `@Repository`) binnen het package van de Spring Boot applicatie (`@ComponentScan`)
- Laat toe dat binnen de klasse zelf extra beans worden gedefinieerd die door andere klassen gebruikt kunnen worden. (`@Configuration`)

1.5.4 IoC in werking

We starten eerst met de klasse `Pet` (huisdier). We willen ons huisdier doorheen de ganse applicatie ter beschikking hebben.

```
package be.px1.demo;

public class Pet {
    private String name;
    private String breed;

    public Pet(String name, String breed) {
        this.name = name;
        this.breed = breed;
    }

    public String getBreed() {
        return breed;
    }

    public String getName() {
        return name;
    }

    @Override
    public String toString() {
        return "Pet{" +
            "name='" + name + '\'' +
            ", breed='" + breed + '\'' +
            '}';
    }
}
```

Nu kunnen we dus in de main-klasse een Spring bean aanmaken voor ons favoriete huisdier.

```
package be.px1.demo;

import be.px1.demo.domain.Pet;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.ConfigurableApplicationContext;
import org.springframework.context.annotation.Bean;

@SpringBootApplication // can be replaced by @EnableAutoConfiguration
    ↪ @ComponentScan and @Configuration
public class DemoProjectApplication {

    public static void main(String[] args) {
        ConfigurableApplicationContext applicationContext =
            ↪ SpringApplication.run(DemoProjectApplication.class, args);
        System.out.println(applicationContext.getApplicationName());
        applicationContext.close();
    }

    @Bean
    public Pet myPet() {
        return new Pet("Scott", "Scotch Collie");
    }
}
```

Het Pet-object wordt bijgehouden in de application context. Elke klasse die nu een Pet-object nodig heeft, kan dit object laten injecteren door de application context. We spreken dan over **dependency injection**. We laten de application context als het ware objecten injecteren in andere objecten. Dat is dus de manier waarop Spring Boot zorgt dat inversion of control mogelijk is.

```
package be.px1.demo.beans;

import be.px1.demo.domain.Pet;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.CommandLineRunner;
import org.springframework.stereotype.Component;

@Component
public class WelcomeMessage implements CommandLineRunner {
    private final Pet myPet;

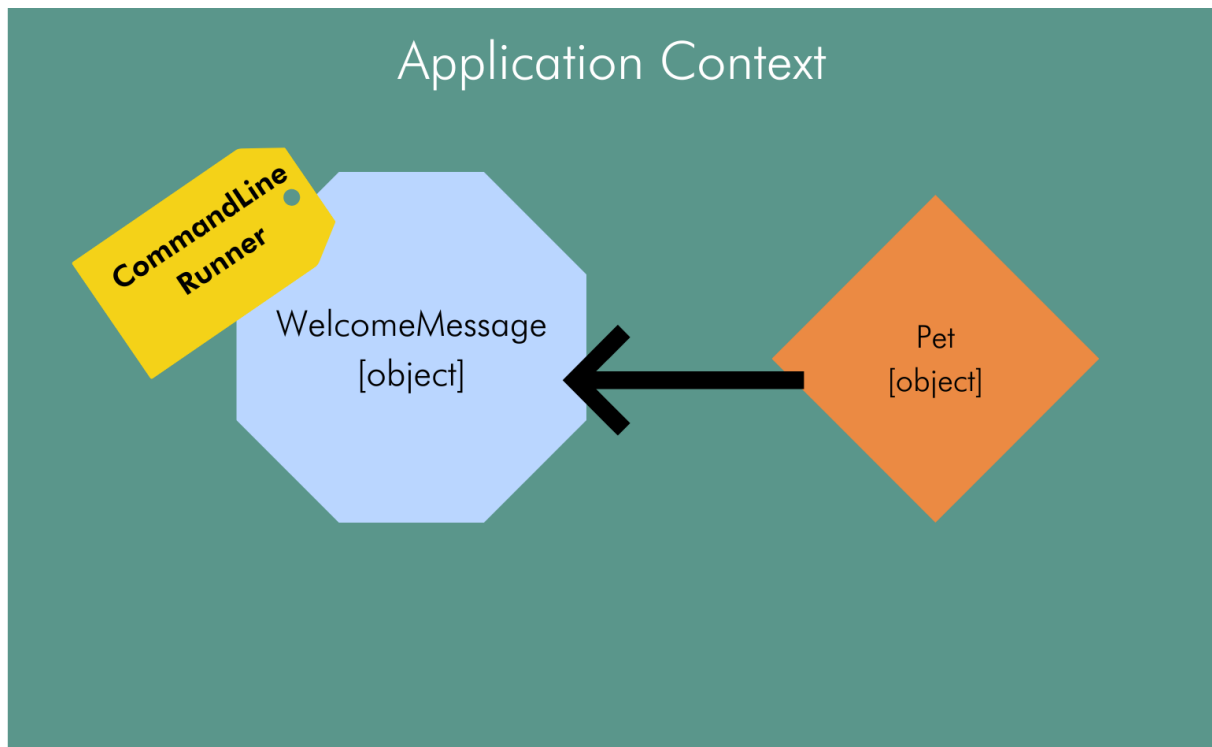
    @Autowired // annotation not necessary
    public WelcomeMessage(Pet myPet) {
        this.myPet = myPet;
    }
}
```

```

    }

    @Override
    public void run(String... args) throws Exception {
        System.out.println("Welcome to Java Advanced");
        System.out.println(myPet);
    }
}

```



Figuur 1.2: De Application Context

Wanneer je het programma uitvoert zal nu het volgende in de console verschijnen:

```

Welcome to Java Advanced
Pet{name='Scott', breed='Scotch Collie'}

```

Om een idee te krijgen van de auto-configuratie die in de Spring Boot applicatie achter de schermen wordt uitgevoerd kan je het loglevel van de applicatie eens aanpassen. Het loglevel van Spring Boot aanpassen doe je door een lijn toe te voegen in application.properties bestand dat je vindt in de folder **/src/main/resources**.

```
logging.level.org.springframework=debug
```

logging.level.org.springframework is één van vele application properties. Een overzicht van beschikbare application properties vind je terug op de documentatie website van Spring Boot <https://docs.spring.io/spring-boot/docs/current/reference/html/application-properties.html>.

In de logging vind je nu tussen de vele lijnen onderstaande regels:

... Creating shared instance of singleton bean 'welcomeMessage'
... Creating shared instance of singleton bean 'myPet'
... Autowiring by type from bean name 'welcomeMessage' via constructor to bean named 'myPet'

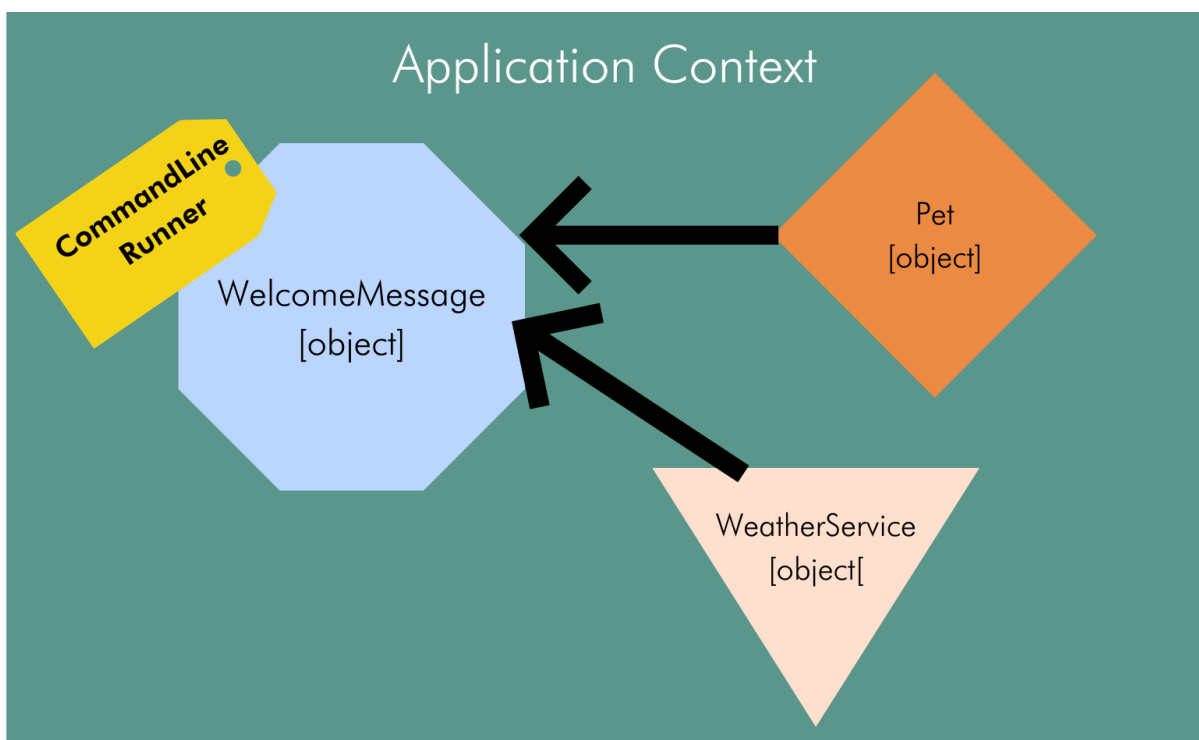
We geven nog een tweede voorbeeld van dependency injection.

```
package be.px1.demo.service;

import org.springframework.stereotype.Component;

@Component
public class WeatherService {
    public void printWeather() {
        System.out.println("The weather is sunny with a 20% chance of rain");
    }
}
```

We laten nu de instantie van de WeatherService injecteren in onze CommandLineRunnerWelcomeMessage.



Figuur 1.3: De Application Context

```
package be.px1.demo.beans;

import be.px1.demo.domain.Pet;
import be.px1.demo.service.WeatherService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.CommandLineRunner;
```

```
import org.springframework.stereotype.Component;

@Component
public class WelcomeMessage implements CommandLineRunner {

    private final Pet myPet;
    private final WeatherService weatherService;

    @Autowired // annotation not necessary
    public WelcomeMessage(Pet myPet, WeatherService weatherService) {
        this.myPet = myPet;
        this.weatherService = weatherService;
    }

    @Override
    public void run(String... args) throws Exception {
        System.out.println("Welcome to Java Advanced");
        System.out.println(myPet);
        weatherService.printWeather();
    }
}
```

Hoofdstuk 2

REST

Learning goals

De junior-collega

1. kan beschrijven wat een RESTful web applicatie is
2. kan een POST, GET, PUT en DELETE-verzoek afhandelen in Spring Boot
3. kan beschrijven wat spring-boot-starter-web is
- 4.

2.1 HTTP-verzoekmethoden

Spring Boot is een goede keuze als je een REST API (of voluit een RESTful web API) wilt ontwikkelen in Java.

REST (Representational State Transfer) is een gestandaardiseerde manier om communicatie tussen verschillende softwaretoepassingen over het internet mogelijk te maken. In een RESTful web applicatie wordt de functionaliteit van de applicatie beschikbaar gesteld als resources, die kunnen worden geïdentificeerd door URI's (Uniform Resource Identifiers). Gebruikers en andere applicaties kunnen met deze resources communiceren via standaard HTTP-verzoekmethoden (HTTP-request, HTTP-method of HTTP-verb). In essentie is HTTP het transportprotocol dat wordt gebruikt om gegevens over te dragen, terwijl REST de verzameling van ontwerpprincipes is die bepalen hoe die gegevens moeten worden georganiseerd en benaderd.

- **GET:** Het GET-verzoek wordt gebruikt om gegevens op te halen van een specifieke resource.

Voorbeeld URI: GET /api/products/123

Dit verzoek haalt informatie op over het product met ID 123.

- **POST:** Het POST-verzoek wordt gebruikt om nieuwe gegevens naar een resource te verzenden. Het wordt vaak gebruikt voor het maken van nieuwe resources of het toevoegen van gegevens aan een bestaande resource.

Voorbeeld URI: POST /api/products

Dit verzoek voegt een nieuw product toe aan de lijst van producten.

- **PUT:** Het PUT-verzoek wordt gebruikt om gegevens bij te werken voor een specifieke resource of om een nieuwe resource te maken als deze niet bestaat. Het is idempotent, wat betekent dat meerdere PUT-verzoeken hetzelfde resultaat opleveren.

Voorbeeld URI: PUT /api/products/123

Dit verzoek bijwerken de informatie van het product met ID 123.

- **DELETE:** Het DELETE-verzoek wordt gebruikt om een resource te verwijderen of te deactiveren.

Voorbeeld URI: DELETE /api/products/123

Dit verzoek verwijdert het product met ID 123 uit de lijst van producten.

2.2 Spring Boot Starter Web

Spring Boot Starter Web is de verzameling van alle bibliotheken (third party libraries) die we nodig hebben om RESTful web applicaties te bouwen. De verzameling bestaat ondermeer uit Spring MVC, REST, jackson en Tomcat. Apache Tomcat is een populaire open source web server voor Java toepassingen. Als je de dependency spring-boot-start-web toevoegt, start de Tomcat web server op zodra je de Spring boot applicatie opstart. We spreken van de embedded web server in Spring boot. Je kan er ook voor kiezen om één van de alternatieve web servers te gebruiken zoals jetty of undertow. Jackson is een populaire library om Java-objecten naar JSON te converteren en vice versa.

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

Je voegt de dependency toe in het bestand pom.xml.

Start nu de Spring Boot applicatie opnieuw op.

De poortnummer kan aangepast worden in het bestand `application.properties`. Je gebruikt de eigenschap `server.port` om de gewenste poortnummer te kiezen.

```
server.port=8081
```

2.3 De RestController

Spring boot heeft een annotatie voorzien voor de Spring bean die verantwoordelijk is voor het afhandelen van HTTP requests nl. `@RestController`. Spring boot heeft ook een annotatie `@Controller`, maar de `@RestController` zorgt ervoor dat het respons op het HTTP-request automatisch wordt omgezet (geserialiseerd) naar JSON of XML en wordt teruggestuurd naar de client.

```
package be.px1.demo.api;

import jakarta.annotation.PostConstruct;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.ArrayList;
import java.util.List;
import java.util.Random;

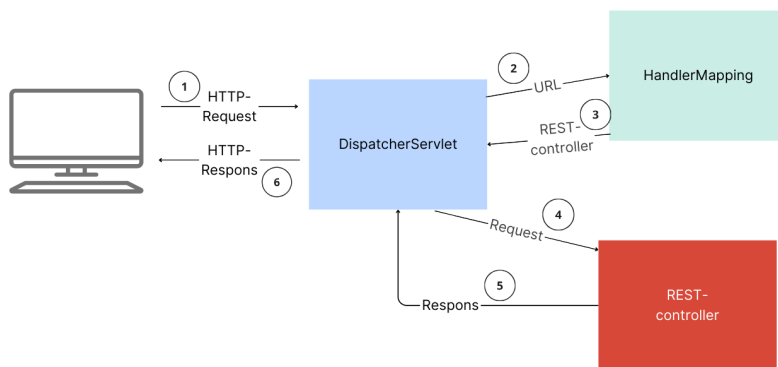
@RestController
@RequestMapping("/greetings")
public class GreetingController {

    private final List<String> messages = new ArrayList<>();
    private static final Random RANDOM = new Random();

    @PostConstruct
    public void init() {
        messages.add("Peek-a-boo!");
        messages.add("Howdy-doody!");
        messages.add("My name's Ralph, and I'm a bad guy.");
        messages.add("I come in peace!");
        messages.add("Put that cookie down!");
    }

    @GetMapping("/hello")
    public String doGreeting() {
        return messages.get(RANDOM.nextInt(messages.size()));
    }
}
```

De `@RestController` markeert de klasse `GreetingController` als een REST-controller. De annotatie `@RequestMapping("/greetings")` specificeert het basispad voor alle requests die door deze controller worden afgehandeld. `@GetMapping("/hello")` geeft aan dat de `goGreeting`-methode wordt uitgevoerd wanneer een HTTP GET-verzoek wordt gemaakt naar het pad `/greetings/hello`. Het resultaat van deze methode wordt automatisch omgezet in tekst en teruggestuurd als de respons.



Figuur 2.1: Een REST-verzoek afhandelen

De component van Spring Boot die verantwoordelijk is dat een HTTP-request door de juiste REST-controller wordt afgehandeld is de `DispatcherServlet`. De `DispatcherServlet` is onderdeel van Spring MVC. De `DispatcherServlet` bepaalt welke controller een HTTP-request moet afhandelen, hij geeft het request door aan de juiste controller en verwerkt het respons van de controller om een HTTP-respons terug te sturen naar de client.

Om te achterhalen welke REST-controller verantwoordelijk is om een HTTP-request af te handelen raadpleegt de `DispatcherServlet` de `HandlerMapping`. De `HandlerMapping` is als het ware een kaart die URL's koppelt aan specifieke controllerklassen en methoden. Op basis van de URL in het binnenkomende request bepaalt de `DispatcherServlet` welke controllerklasse en methode verantwoordelijk zijn voor het afhandelen van het request.

Oefening 2.1. Maak het package `be.pxl.demo.api`. Voeg de klasse **GreetingController** toe het package. Start de Spring Boot applicatie en open de URL <http://localhost:8080/greetings/hello> in een browser. Voeg in de REST-controller een methode toe met de URI GET `/greetings/daytime` die de huidige dag en het tijdstip teruggeeft in het formaat 'Maandag 18 september 2023'.

2.4 MusicPlaylist

We gaan een nieuwe Spring Boot toepassing aanmaken waarmee we een muziek playlist beheren.

Oefening 2.2. Maak een nieuwe Spring boot toepassing MusicPlaylist. We gebruiken Spring MVC om een RESTful web applicatie te maken.

2.4.1 Een liedje toevoegen aan een playlist

POST	/playlist/songs <i>Add a new song to the playlist.</i>
Body	application/json
<pre>{ "title": "Hello", "artist": "Adele", "duration_seconds": 293, "genre": "POP" }</pre>	
Response	application/json
200 ok	

Om te beginnen hebben we de klasse Song nodig.

```
public class Song {
    private String title;
    private String artist;
    @JsonProperty("duration_seconds")
    private int durationSeconds;
    private Genre genre;

    // Default constructor
    public Song() {
    }

    // Parameterized constructor
    public Song(String title, String artist, int durationSeconds, Genre
        ↪ genre) {
        this.title = title;
        this.artist = artist;
        this.durationSeconds = durationSeconds;
    }
}
```

```

        this.genre = genre;
    }

    // Getter and setter methods
    public String getTitle() {
        return title;
    }

    public void setTitle(String title) {
        this.title = title;
    }

    public String getArtist() {
        return artist;
    }

    public void setArtist(String artist) {
        this.artist = artist;
    }

    public int getDurationSeconds() {
        return durationSeconds;
    }

    public void setDurationSeconds(int durationSeconds) {
        this.durationSeconds = durationSeconds;
    }

    public Genre getGenre() {
        return genre;
    }

    public void setGenre(Genre genre) {
        this.genre = genre;
    }

    @Override
    public String toString() {
        return "Song{" +
            "title='" + title + '\'' +
            ", artist='" + artist + '\'' +
            ", durationSeconds=" + durationSeconds +
            ", genre='" + genre + '\'' +
            '}';
    }
}

```

Nu gaan we de REST-controller implementeren. We gaan hierin een methode voorzien

die een POST op de URI `/playlist/songs` kan afhandelen. Initieel gaan we enkel de titel in de loggegevens tonen.

```
@RestController
@RequestMapping("/playlist/songs")
public class MusicPlaylistController {

    private static final Logger LOGGER =
        ↪ LoggerFactory.getLogger(MusicPlaylistController.class);

    @PostMapping
    public void addSong(@RequestBody Song song) {
        if (LOGGER.isInfoEnabled()) {
            LOGGER.info("Adding song [" + song.getTitle() + "]");
        }
    }
}
```

De liedjes die aan de playlist worden toegevoegd willen we bijhouden. Later zullen we ze wegschrijven in een databank, maar voorlopig gaan we ze bijhouden in een lijst. Om dit mogelijk te maken gaan we een nieuwe Spring Bean toevoegen: de `MusicPlaylistService`.

```
package be.px1.demo;

import be.px1.demo.domain.Song;
import org.springframework.stereotype.Service;

import java.util.ArrayList;
import java.util.List;

@Service
public class MusicPlaylistService {
    private final List<Song> myPlaylist = new ArrayList<>();

    public void addSong(Song song) {
        myPlaylist.add(song);
    }
}
```

De `MusicPlaylistService` wordt geannoteerd met `@Service`. In onze Spring Boot applicaties gaan we steeds de business-logica implementeren in de service-laag. De `@Service` annotatie wordt gebruikt voor de componenten (Spring Beans) in de service-laag. Wanneer onze Spring Boot applicatie opstart wordt er exact één instantie van de `MusicPlaylistService` aangemaakt in de `ApplicationContext` en deze instantie wordt tijdens de volledige levensduur van de toepassing gebruikt. Dit noemen we de scope van de service en de default scope noemen we **singleton**. Dit betekent dat we één enkele, gedeelde playlist hebben voor alle gebruikers.

Nu gaan we de `MusicPlaylistService` beschikbaar maken in de `MusicPlaylistController`. We maken gebruik van **constructor injection**. Zodra de instantie van de `MusicPlaylistController` door Spring Boot wordt aangemaakt, zal er eerst gezorgd worden dat de instantie `MusicPlaylistService` aangemaakt wordt. Deze instantie wordt dan achter de schermen meegegeven aan de constructor van de `MusicPlaylistController`. Zo kan ons `MusicPlaylistController`-object het `MusicPlaylistService`-object gebruiken. Omdat er maar één constructor is, is de annotatie `@Autowired` eigenlijk overbodig.

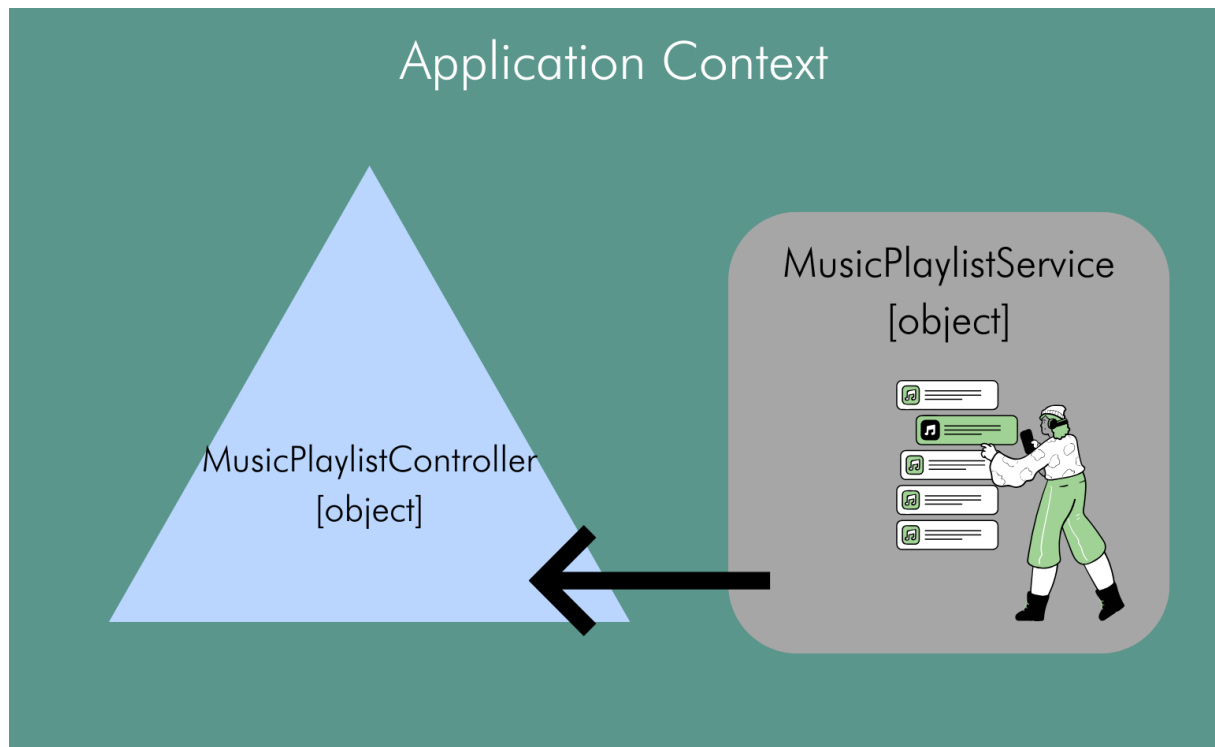
```
@RestController
@RequestMapping("/playlist/songs")
public class MusicPlaylistController {

    private static final Logger LOGGER =
        ↪ LoggerFactory.getLogger(MusicPlaylistController.class);
    private final MusicPlaylistService musicPlaylistService;

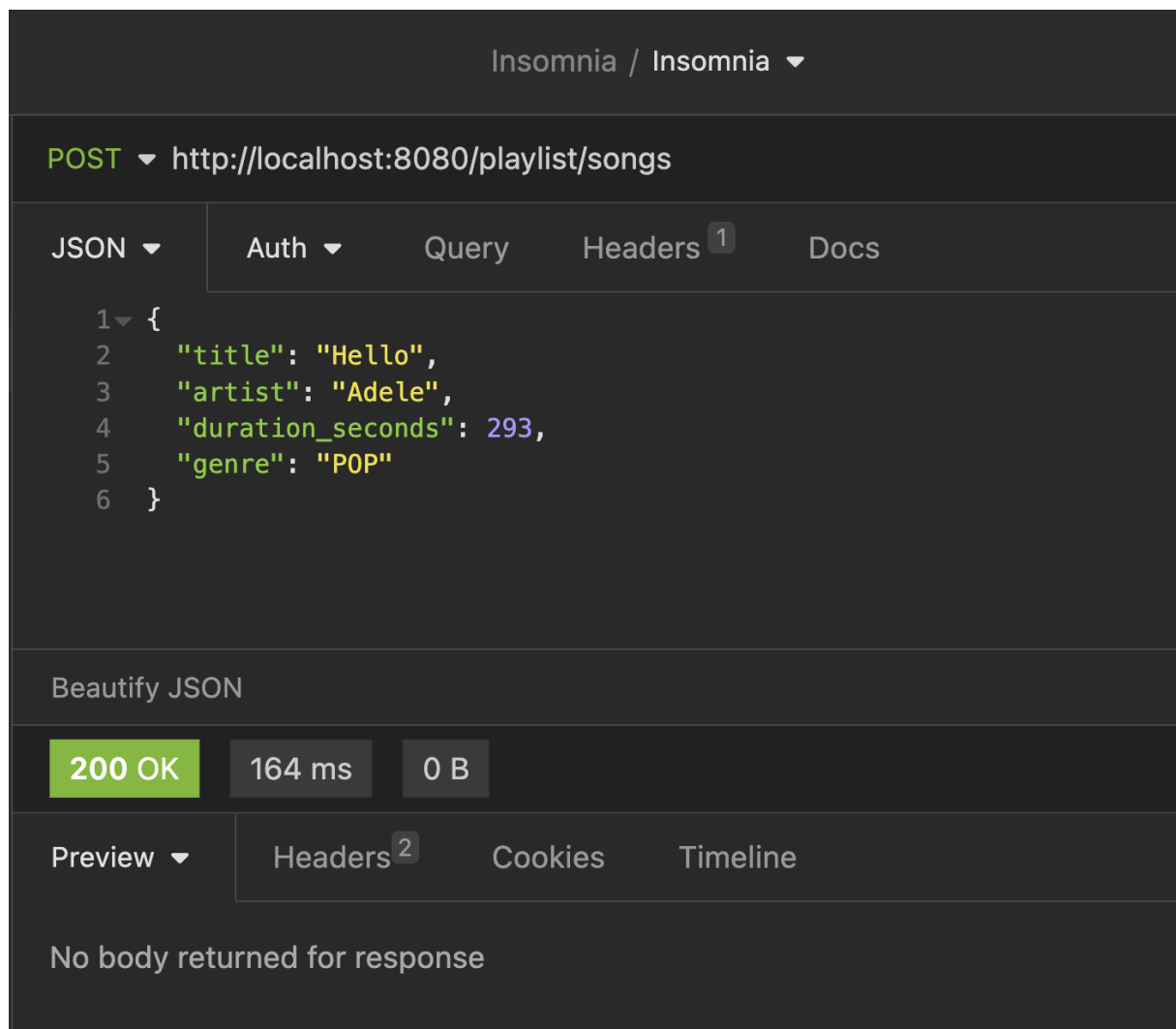
    @Autowired
    public MusicPlaylistController(MusicPlaylistService
        ↪ musicPlaylistService) {
        this.musicPlaylistService = musicPlaylistService;
    }

    @PostMapping
    public void addSong(@RequestBody Song song) {
        if (LOGGER.isInfoEnabled()) {
            LOGGER.info("Adding song [" + song.getTitle() + "]");
        }
        musicPlaylistService.addSong(song);
    }
}
```

Test nu het POST-verzoek. Je kan Postman, Insomnia of een andere tool gebruiken om een POST-verzoek naar de toepassing te sturen. De toegevoegde liedjes gaan uiteraard verloren wanneer je de toepassing herstart.



Figuur 2.2: Spring Beans in de Application Context



Figuur 2.3: POST-verzoek met Insomnia

2.4.2 De playlist opvragen

GET	/playlist/songs <i>Retrieve all songs from the playlist</i>
Parameter	
<i>no parameter</i>	
Body	application/json
Response	
200	ok
<pre>[{ "title": "Hello", "artist": "Adele", "genre": "POP", "duration_seconds": 293 }, { "title": "Shape of You", "artist": "Ed Sheeran", "genre": "POP", "duration_seconds": 233 }, { "title": "Umbrella", "artist": "Rihanna", "genre": "RNB", "duration_seconds": 264 }]</pre>	

```
package be.px1.demo;

import be.px1.demo.domain.Song;
import org.springframework.stereotype.Service;

import java.util.ArrayList;
import java.util.List;

@Service
public class MusicPlaylistService {
    private final List<Song> myPlaylist = new ArrayList<>();
}
```

```

    public void addSong(Song song) {
        myPlaylist.add(song);
    }

    public List<Song> getSongs() {
        return myPlaylist;
    }
}

```

```

package be.px1.demo.controller;

import be.px1.demo.MusicPlaylistService;
import be.px1.demo.domain.Song;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.List;

@RestController
@RequestMapping("/playlist/songs")
public class MusicPlaylistController {

    private static final Logger LOGGER =
        ↳ LoggerFactory.getLogger(MusicPlaylistController.class);
    private final MusicPlaylistService musicPlaylistService;

    @Autowired
    public MusicPlaylistController(MusicPlaylistService
        ↳ musicPlaylistService) {
        this.musicPlaylistService = musicPlaylistService;
    }

    @PostMapping
    public void addSong(@RequestBody Song song) {
        if (LOGGER.isInfoEnabled()) {
            LOGGER.info("Adding song [" + song.getTitle() + "]");
        }
        musicPlaylistService.addSong(song);
    }

    @GetMapping
    public List<Song> getSongs() {

```

```

        return musicPlaylistService.getSongs();
    }
}

```

2.4.3 Liedjes van één genre

GET	/playlist/songs/{genre}
<i>Retrieve all songs from the playlist with the given genre</i>	
Parameter	
genre	the requested genre
Body	application/json
Response	
200	ok
<pre> [{ "title": "Hello", "artist": "Adele", "genre": "POP", "duration_seconds": 293 }, { "title": "Shape of You", "artist": "Ed Sheeran", "genre": "POP", "duration_seconds": 233 }] </pre>	

```

package be.px1.demo.controller;

import be.px1.demo.MusicPlaylistService;
import be.px1.demo.domain.Genre;
import be.px1.demo.domain.Song;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;

```

```

import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.List;

@RestController
@RequestMapping("/playlist/songs")
public class MusicPlaylistController {

    private static final Logger LOGGER =
        ↪ LoggerFactory.getLogger(MusicPlaylistController.class);
    private final MusicPlaylistService musicPlaylistService;

    @Autowired
    public MusicPlaylistController(MusicPlaylistService
        ↪ musicPlaylistService) {
        this.musicPlaylistService = musicPlaylistService;
    }

    @PostMapping
    public void addSong(@RequestBody Song song) {
        if (LOGGER.isInfoEnabled()) {
            LOGGER.info("Adding song [" + song.getTitle() + "]");
        }
        musicPlaylistService.addSong(song);
    }

    @GetMapping
    public List<Song> getSongs() {
        return musicPlaylistService.getSongs();
    }

    @GetMapping("/{genre}")
    public List<Song> getSongs(@PathVariable Genre genre) {
        return musicPlaylistService.getSongsByGenre(genre);
    }
}

```

```

package be.px1.demo;

import be.px1.demo.domain.Genre;
import be.px1.demo.domain.Song;
import org.springframework.stereotype.Service;

import java.util.ArrayList;
import java.util.List;

@Service

```

```

public class MusicPlaylistService {
    private final List<Song> myPlaylist = new ArrayList<>();

    public void addSong(Song song) {
        myPlaylist.add(song);
    }

    public List<Song> getSongs() {
        return myPlaylist;
    }

    public List<Song> getSongsByGenre(Genre genre) {
        List<Song> response = new ArrayList<>();
        for (Song song : myPlaylist) {
            if (song.getGenre() == genre) {
                response.add(song);
            }
        }
        return response;
    }
}

```

2.4.4 Gegevens van een liedje aanpassen

Als je een nieuwe song in de playlist toevoegt, dan wordt deze steeds achteraan in de lijst toegevoegd. We kunnen nu de gegevens van het liedje op een gegeven positie in de lijst gaan overschrijven of aanpassen. De index die we meegeven is een waarde van 0 tot 1 minder dan de lengte van de lijst. Later zullen we zien hoe we een duidelijke foutboodschap kunnen geven aan de client als een foutieve index-waarde wordt gegeven.

Om de gegevens van een liedje aan te passen gebruiken we een PUT-verzoek. Je moet steeds alle gegevens van het liedje meegeven in de requestbody.

PUT	/musicplaylist/songs/{index} <i>Update the song at the given index.</i>
Parameter	
index	Index of the song to be updated.
Body	application/json
<pre>{ "title": "Hello", "artist": "Adele", "genre": "POP", "duration_seconds": 293 }</pre>	
Response	application/json
200	ok

```
package be.px1.demo.controller;

import be.px1.demo.MusicPlaylistService;
import be.px1.demo.domain.Genre;
import be.px1.demo.domain.Song;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.List;

@RestController
@RequestMapping("/playlist/songs")
public class MusicPlaylistController {

    private static final Logger LOGGER =
```

```

        ↪ LoggerFactory.getLogger(MusicPlaylistController.class);
private final MusicPlaylistService musicPlaylistService;

@Autowired
public MusicPlaylistController(MusicPlaylistService
    ↪ musicPlaylistService) {
    this.musicPlaylistService = musicPlaylistService;
}

@PostMapping
public void addSong(@RequestBody Song song) {
    if (LOGGER.isInfoEnabled()) {
        LOGGER.info("Adding song [" + song.getTitle() + "]");
    }
    musicPlaylistService.addSong(song);
}

@GetMapping
public List<Song> getSongs() {
    return musicPlaylistService.getSongs();
}

@GetMapping("/{genre}")
public List<Song> getSongs(@PathVariable Genre genre) {
    return musicPlaylistService.getSongsByGenre(genre);
}

@GetMapping("/{index}")
public void updateSong(@PathVariable int index, @RequestBody Song song)
    ↪ {
    musicPlaylistService.updateSong(index, song);
}
}

```

```

package be.px1.demo;

import be.px1.demo.domain.Genre;
import be.px1.demo.domain.Song;
import org.springframework.stereotype.Service;

import java.util.ArrayList;
import java.util.List;

@Service
public class MusicPlaylistService {
    private final List<Song> myPlaylist = new ArrayList<>();

    public void addSong(Song song) {

```



```

        myPlaylist.add(song);
    }

    public List<Song> getSongs() {
        return myPlaylist;
    }

    public List<Song> getSongsByGenre(Genre genre) {
        List<Song> response = new ArrayList<>();
        for (Song song : myPlaylist) {
            if (song.getGenre() == genre) {
                response.add(song);
            }
        }
        return response;
    }

    public void updateSong(int index, Song song) {
        myPlaylist.set(index, song);
    }

    public void deleteSong(int index) {
        myPlaylist.remove(index);
    }
}

```

We vervangen het liedje op de opgegeven index door een nieuw Song-object met de aangepaste gegevens.

2.4.5 Een liedje verwijderen

Om een liedje op een opgegeven index uit de playlist te verwijderen gaan we een DELETE-verzoek implementeren.

DELETE	<code>/musicplaylist/songs/{index}</code> <i>Delete the song at the given index.</i>
Parameter	
index	position of the song to be deleted
Response	application/json
200	ok

```
package be.px1.demo.controller;
```

```

import be.px1.demo.MusicPlaylistService;
import be.px1.demo.domain.Genre;
import be.px1.demo.domain.Song;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.List;

@RestController
@RequestMapping("/playlist/songs")
public class MusicPlaylistController {

    private static final Logger LOGGER =
        ↪ LoggerFactory.getLogger(MusicPlaylistController.class);
    private final MusicPlaylistService musicPlaylistService;

    @Autowired
    public MusicPlaylistController(MusicPlaylistService
        ↪ musicPlaylistService) {
        this.musicPlaylistService = musicPlaylistService;
    }

    @PostMapping
    public void addSong(@RequestBody Song song) {
        if (LOGGER.isInfoEnabled()) {
            LOGGER.info("Adding song [" + song.getTitle() + "]");
        }
        musicPlaylistService.addSong(song);
    }

    @GetMapping
    public List<Song> getSongs() {
        return musicPlaylistService.getSongs();
    }

    @GetMapping("/{genre}")
    public List<Song> getSongs(@PathVariable Genre genre) {
        return musicPlaylistService.getSongsByGenre(genre);
    }
}

```

```

@PutMapping("/{index}")
public void updateSong(@PathVariable int index, @RequestBody Song song)
    ↪ {
    musicPlaylistService.updateSong(index, song);
}

@DeleteMapping("/{index}")
public void updateSong(@PathVariable int index) {
    musicPlaylistService.deleteSong(index);
}
}

```

```

package be.px1.demo;

import be.px1.demo.domain.Genre;
import be.px1.demo.domain.Song;
import org.springframework.stereotype.Service;

import java.util.ArrayList;
import java.util.List;

@Service
public class MusicPlaylistService {
    private final List<Song> myPlaylist = new ArrayList<>();

    public void addSong(Song song) {
        myPlaylist.add(song);
    }

    public List<Song> getSongs() {
        return myPlaylist;
    }

    public List<Song> getSongsByGenre(Genre genre) {
        List<Song> response = new ArrayList<>();
        for (Song song : myPlaylist) {
            if (song.getGenre() == genre) {
                response.add(song);
            }
        }
        return response;
    }

    public void updateSong(int index, Song song) {
        myPlaylist.set(index, song);
    }
}

```

```

    public void deleteSong(int index) {
        myPlaylist.remove(index);
    }
}

```

Oefening 2.3. Huizenjacht We maken een Spring Boot toepassing om het aanbod op de huizenmarkt te beheren. Ontwikkel een RESTful web toepassing met onderstaande endpoints. Je voorziet een component HouseService met één enkele, gedeelde lijst van woningen voor alle gebruikers.

Een huis wordt voorgesteld als een resource met volgende eigenschappen:

- code (string): Unieke identificatie van het huis.
- name (string): Naam of beschrijving van het huis.
- status (enum): Status FOR_SALE of SOLD.
- city (string): Locatie van het huis.
- price (double): Prijs van het huis.

Overzicht REST Endpoints

POST	/houses <i>Create a new house.</i>
Parameter	
<i>no parameter</i>	
Body	application/json
<pre> { "code": "HAS_001", "name": "Beautiful house in the city", "city": "Hasselt", "price": 250000 } </pre>	
Response	application/json
200 ok	

PUT	/houses/{code} <i>Update the data (status, price,...) of the house with the given code.</i>
Parameter	
code	unique identification of a house
Body	application/json
<pre>{ "status": "SOLD", "name": "Beautiful house in the city", "city": "Genk", "price": 320000 }</pre>	
Response	application/json
200	ok

GET	/houses <i>Retrieve all houses.</i>
Parameter	
no parameter	
Response	application/json
200	ok
<pre>[{ "code": "GNK_001", "name": "Beautiful house in the city", "status": "SOLD", "city": "Genk", "price": 250000 }, { "code": "HAS_003", "name": "Cozy bungalow", "status": "FOR_SALE", "city": "Hasselt", "price": 180000 }]</pre>	

DELETE /houses/{code}

Delete the house with the given code.

Parameter

code unique identification of a house

Response

application/json

200 ok

Hoofdstuk 3

Collections: Map

In Java is de *Map* interface een onderdeel van het Java Collections Framework. Je gebruikt deze gegevensstructuur als je key-value paren (sleutel-waarde paren) wilt opslaan en beheren. De unieke sleutel wordt opgeslaan en gekoppeld aan een specifieke waarde. De belangrijkste implementaties van de Map interface in Java zijn HashMap, LinkedHashMap, TreeMap en Hashtable.

3.1 Generieke klasse HashMap

De interface Map en de klasse HashMap zijn generiek. Dit betekent dat ze zijn ontworpen om met verschillende datatypes te werken, zonder het datatype op voorhand vast te leggen. In het geval van HashMap, maakt het gebruik van generieke datatypes het mogelijk om de key- en value-datatypes flexibel te kiezen op het moment dat je een instantie van de HashMap maakt. Hierdoor kun je de HashMap gebruiken met verschillende soorten gegevens zonder dat je aparte klassen hoeft te schrijven voor elke combinatie van datatypes.

HashMap<K, V>

K staat voor het datatype van de sleutels die je in de HashMap wilt opslaan. V staat voor het datatype van de waarden die je in de HashMap wilt opslaan.

3.2 De interface Map

Hier is een overzicht van enkele veelgebruikte methoden in de Map interface:

- **put(K key, V value)** Voegt een sleutel-waarde paar toe aan de map.
- **get(Object key)** Geeft de waarde terug die is gekoppeld aan de opgegeven sleutel, of null als de sleutel niet in de map voorkomt.
- **containsKey(Object key)** Controleert of de map een specifieke sleutel bevat.
- **containsValue(Object value)** Controleert of de map een specifieke waarde bevat.
- **remove(Object key)** Verwijdert de opgegeven sleutel met zijn gekoppelde waarde.
- **keySet()** Geeft een set (Set) van alle sleutels in de map terug.

- **values()** Geeft een verzameling (Collection) van alle waarden in de map terug.

3.3 Gebruik van HashMap

3.3.1 Voorbeeld 1

```
import java.util.HashMap;
import java.util.Map;

public class HashMapVoorbeeld {
    public static void main(String[] args) {
        // Een HashMap maken met String sleutels en Integer waarden
        Map<String, Integer> scores = new HashMap<>();

        // Sleutel-waarde paren toevoegen
        scores.put("Alice", 25);
        scores.put("Bob", 30);
        scores.put("Charlie", 28);
        scores.put("David", 35);

        // Waarde ophalen op basis van een sleutel
        int scoreVanAlice = scores.get("Alice");
        System.out.println("Score van Alice: " + scoreVanAlice); // Geeft
        // → 25 weer

        // Controleren of een sleutel aanwezig is
        boolean bevatSleutel = scores.containsKey("Eve");
        System.out.println("Bevat sleutel 'Eve': " + bevatSleutel); //
        // → Geeft false weer

        // Sleutel-waarde paar verwijderen
        scores.remove("Bob");

        // Alle sleutels afdrukken
        System.out.println("Sleutels in de map: " + scores.keySet());
        // geeft [Alice, Charlie, David]

        // Alle waarden afdrukken
        System.out.println("Waarden in de map: " + scores.values());
        // geeft [25, 28, 35]
    }
}
```

3.3.2 Voorbeeld 2

In dit voorbeeld maken we voor de sleutel-waarden gebruik van Strings. Voor de waarden gebruiken we een zelfgedefinieerde klasse, in dit geval de klasse Player. Op deze manier

kunnen we dus heel gemakkelijk het juiste Player-object opzoeken en eventueel de score aanpassen als we de naam van de speler kennen.

```
import java.util.HashMap;
import java.util.Map;

public class Player {
    private String naam;
    private int score;

    public Player(String naam, int score) {
        this.naam = naam;
        this.score = score;
    }

    public String getNaam() {
        return naam;
    }

    public int getScore() {
        return score;
    }

    public void setScore(int score) {
        this.score = score;
    }
}
```

```
public class SpelerHashMap {
    public static void main(String[] args) {
        // Een HashMap maken met String-sleutels en Player-waarden
        Map<String, Player> spelerMap = new HashMap<>();

        // Spelers toevoegen
        spelerMap.put("Alice", new Player("Alice", 100));
        spelerMap.put("Bob", new Player("Bob", 85));
        spelerMap.put("Charlie", new Player("Charlie", 92));

        // Spelersinformatie ophalen op basis van naam
        String spelerNaam = "Bob";
        Player speler = spelerMap.get(spelerNaam);
        System.out.println("Speler " + spelerNaam + " heeft een score van "
            + speler.getScore());
    }
}
```

Oefening 3.1. Huizenjacht v2 Pas de oefening Huizenjacht uit het vorige hoofdstuk aan. In de service-laag gebruik je een HashMap zodat je een huis heel eenvoudig kunt opzoeken als je de code kent.

Hoofdstuk 4

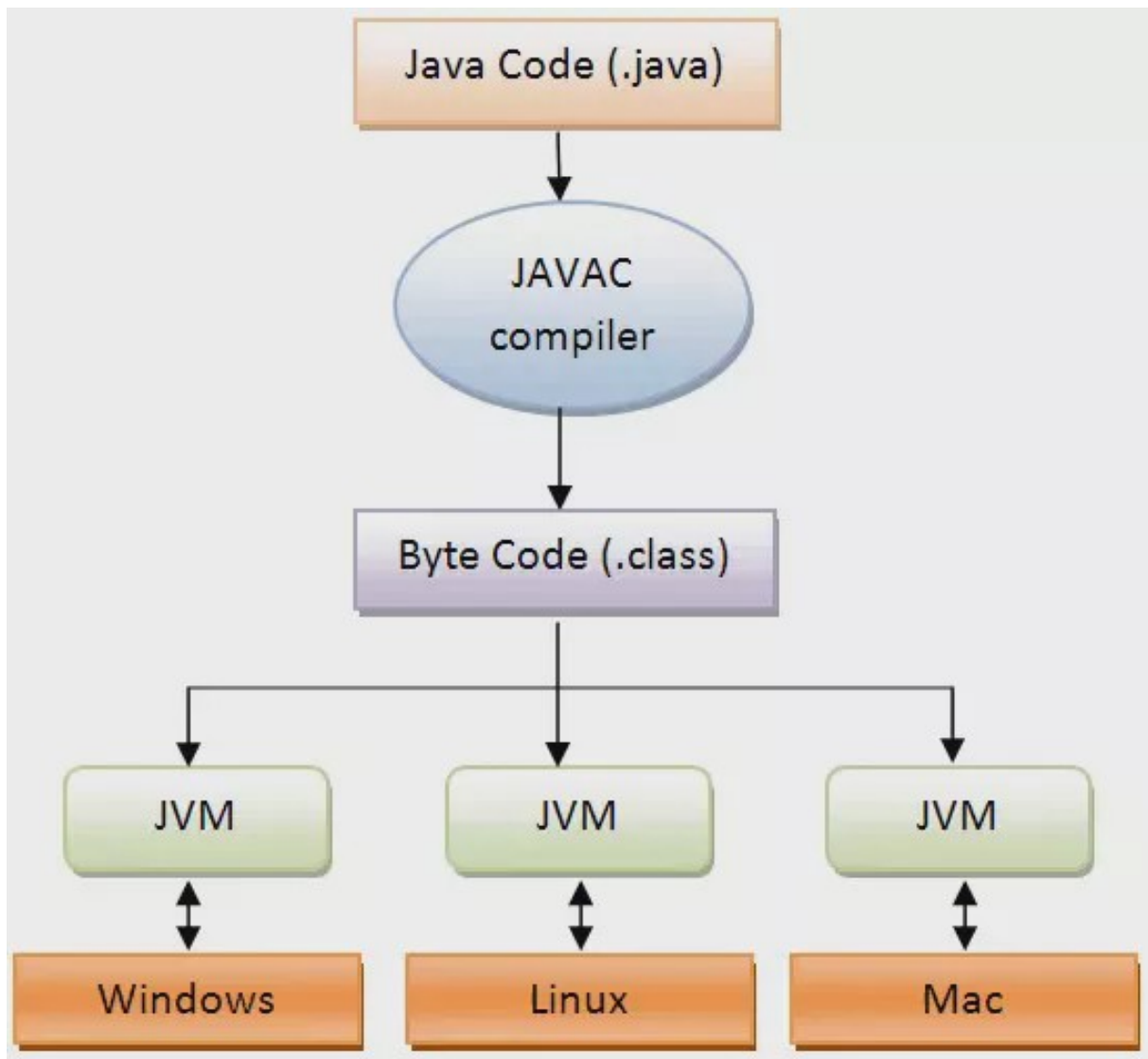
Foutafhandeling

Tijdens het uitvoeren van een programma kan en zal er vanalles foutgaan. De gebruiker van het programma geeft de datum in in een foutief formaat, een printer is offline, er is onvoldoende schrijfruimte vrij om een bestand weg te schrijven, het programma kreeg onvoldoende geheugen toegewezen,... Fouten die zich voordoen tijdens het uitvoeren van een programma, at runtime dus, verdelen we onder in errors en exceptions. Errors zijn de fouten die meestal veroorzaakt worden door het onderliggende besturingssysteem. Tijdens het verloop van het programma kunnen deze errors niet meer opgelost worden en het programma zal daarom beëindigd worden. Dit is niet het geval voor exceptions. Je kan je code op een defensieve manier schrijven en rekening houden met de mogelijke exceptions die kunnen optreden. “Exception handling” is het proces om deze exceptions op een correcte en liefst gebruiksvriendelijke manier af te handelen. Indien een fout niet correct wordt afgehandeld, zal het programma alsnog voortijdig afgebroken worden, maar met de juiste foutafhandeling kan de uitvoer van het programma gewoon verdergezet worden.

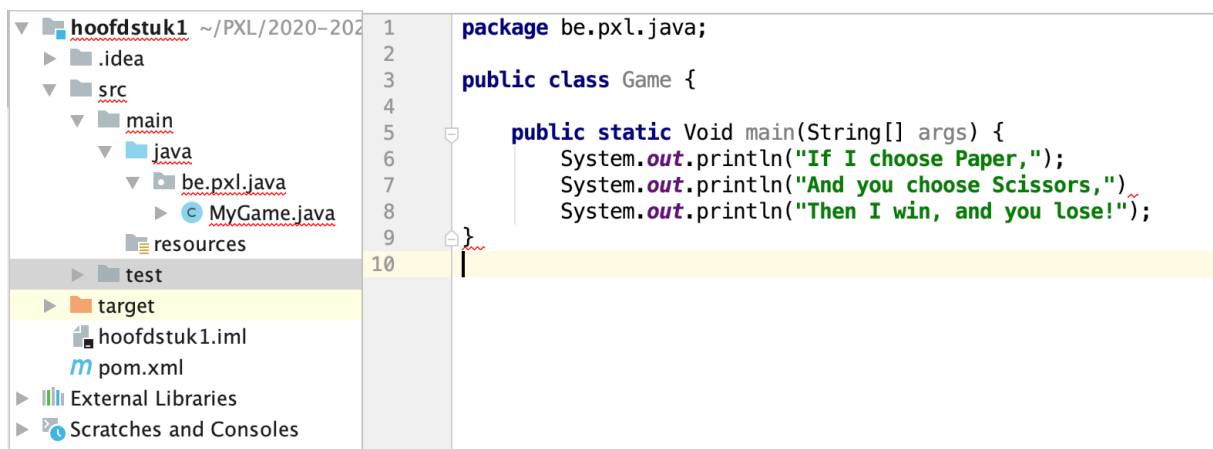
4.1 Compile-time vs runtime errors

Een programmeur schrijft Java code in zijn editor of favoriete IDE. Vervolgens wordt de Java code gecompileerd tot bytecode. Deze bytecode wordt door de JVM (Java Virtual Machine) geïnterpreteerd tot machinecode instructies die worden uitgevoerd door het computersysteem.

Wanneer dus een Java programma wordt opgestart, kunnen er 2 categorieën van problemen voorkomen. Het kan zijn dat het Java programma niet gecompileerd kan worden. In dat geval spreken we van een compile-time error. Indien het programma succesvol gecompileerd wordt, kan er zich tijdens het uitvoeren van de code een probleem voordoen, in dat geval spreken we van runtime errors of exceptions.



Figuur 4.1: Compiler, interpreter en Java Virtual Machine.



Figuur 4.2: Compile-time errors

Oefening 4.1. Welke compile-time errors kan je ontdekken in het volgende code-fragment in figuur 4.2?

Omdat Java een object-geïntendeerde programmeertaal is, wordt er ook een object gebruikt om aan te geven dat er iets fout ging bij uitvoeren van het Java programma. Zodra er zich een probleem voordoet tijdens het uitvoeren van een programma-instructie, wordt er een exception-object aangemaakt en “opgeworpen”. Hierdoor stopt de normale uitvoer van het programma. Er wordt nog geprobeerd om het exception-object op een keurige manier af te handelen (indien die code aanwezig is), maar als dat niet lukt zal het programma beëindigd worden. Het exception-object bevat nuttige informatie voor de ontwikkelaar zoals de methode en lijn-nummer waar de exception werd aangemaakt en het type van de exception.

4.2 First catch

```
public class DivisionByZero {  
  
    public static void main(String[] args) {  
        int a = (1 + 1) % 2;  
        int b = 5;  
        int c = b / a;  
        System.out.println("Het resultaat is " + c);  
    }  
}
```

Als je het bovenstaande programma uitvoert zal het volgende in de console verschijnen:

```
Exception in thread "main" java.lang.ArithmeticException: / by zero  
at be.pxl.java.DivisionByZero.main(DivisionByZero.java:6)<5 internal calls>
```

De variabele *a* bevat inderdaad de waarde 0 en hierdoor hebben we dus te maken met een deling door 0. Zodra de deling wordt uitgevoerd loopt het dus fout en gooit de java runtime een `ArithmeticException` op. Omdat de `ArithmeticException` nergens wordt afgehandeld eindigt het programma. Je zal enkel nog een stacktrace zien verschijnen in de console. Een stacktrace is de naam en foutboodschap van de exception gevolgd door de weg die de exception heeft afgelegd (doorheen de methoden van je klassen) vanaf het moment dat ze werd opgegooid. We zien dus dat de exception is veroorzaakt op regel 6 in de klasse `DivisionByZero`.

```
public class DivisionByZero {  
  
    public static void main(String[] args) {  
        int a = (1 + 1) % 2;  
        int b = 5;  
        try {  
            int c = b / a;  
        }  
    }  
}
```

```

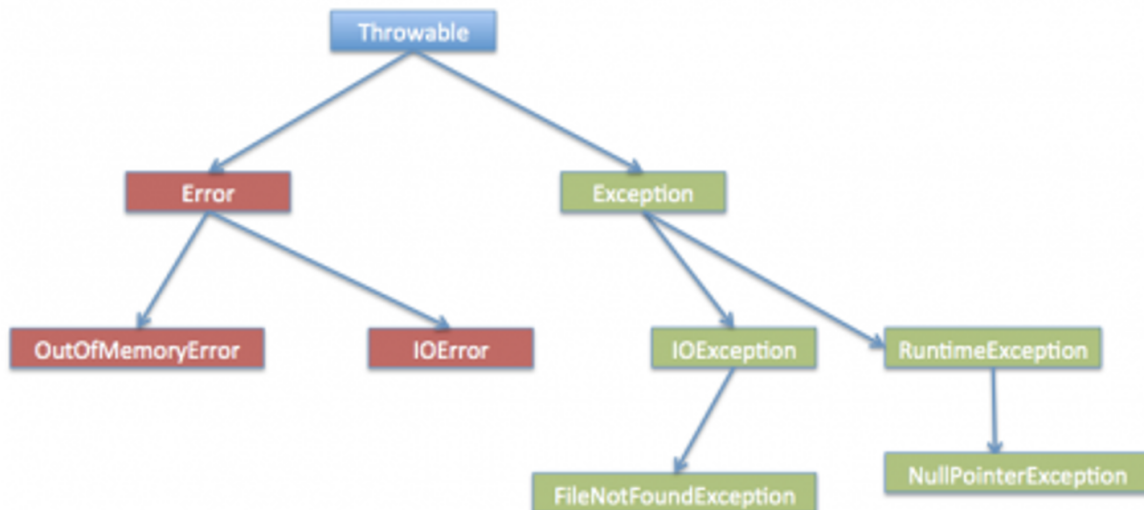
        System.out.println("Het resultaat is " + c);
    } catch (ArithmeticException e) {
        System.out.println("You should not divide a number by zero.");
    }
    System.out.println("First catch completed!");
}
}

```

You should not divide a number by zero.
First catch completed!

We hebben de instructie met de deling nu in een try-blok geplaatst. Omdat de variabele `c` in het try-blok wordt aangemaakt, kan deze variabele ook enkel binnen het try-blok gebruikt worden. Indien een exception optreedt binnen een try-blok zal de programma-uitvoer de resterende code binnen het try-blok overslaan en verdergaan bij het eerste catch-blok dat direct volgt achter het try-blok. Je bent als programmeur verplicht om een try-blok steeds te laten volgen door één of meerdere catch-blokken. In het catch-blok wordt dan de code uitgevoerd om het probleem op te lossen of tenminste een duidelijke boodschap voor de gebruiker van het programma te voorzien. Als alle instructies uit het catch-blok zijn uitgevoerd zal het programma zijn normale uitvoer verderzetten.

4.3 Java exception hiërarchie



Figuur 4.3: Exception hiërarchie

Zoals reeds vermeld wordt er een exception-object aangemaakt zodra zich een probleem voordoet in de code. Er is in Java een hiërarchie gebouwd van exception-classes om verschillende soorten fouten in een programma te categoriseren. Throwable is de superklasse van alle exceptions en errors in Java. Er zijn dus 2 afgeleide klassen van Throwable: Error and Exception. Exceptions zijn nog verder onderverdeeld in **checked exceptions** en **runtime exceptions**.

4.3.1 Errors

Errors zijn problemen die zich voordoen tijdens het uitvoeren van het programma en die meestal niet gerelateerd zijn aan het programma zelf. Daarom is het onmogelijk om erop te anticiperen en te herstellen van deze fouten. Dit kan gaan van hardware falen, over JVM crashes en out of memory errors. We hebben een aparte hiërarchie van errors en we zullen nooit code toevoegen in ons programma om deze fouten af te handelen. We tonen hier enkel voorbeelden van errors.

StackOverflowError

Je hebt ongetwijfeld al eens per ongeluk een programma geschreven met een oneindige lus.

```
public class DemoStackOverflow {  
  
    private static void printNumber(int x) {  
        System.out.println(x);  
        printNumber(x + 2);  
    }  
  
    public static void main(String[] args) {  
        printNumber(15);  
    }  
}
```

15

17

19

...

36597

36599

Exception in thread "main" java.lang.StackOverflowError

...

at be.px1.ja.DemoStackOverflow.printNumber(DemoStackOverflow.java:5)

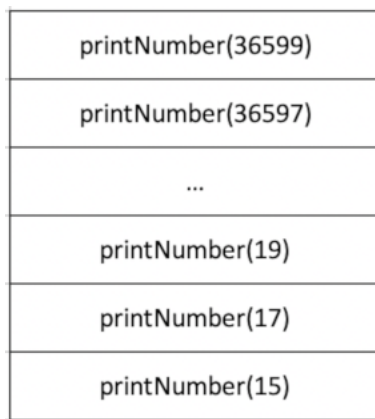
at be.px1.ja.DemoStackOverflow.printNumber(DemoStackOverflow.java:5)

at be.px1.ja.DemoStackOverflow.printNumber(DemoStackOverflow.java:5)

at be.px1.ja.DemoStackOverflow.printNumber(DemoStackOverflow.java:5)

...

De call stack is de manier waarop tijdens de uitvoer van een programma o.a. wordt bijgehouden welke functies worden aangeroepen. Wanneer je programma-uitvoer in een oneindige lus terechtkomt, stapelen de gegevens in de call stack zich razendsnel op en loopt de call stack vol. Het programma zal uiteindelijk eindigen met een StackOverflowError.



Figuur 4.4: Java call stack

OutOfMemoryError

```
package be.px1.java;

public class DemoOutOfMemory {

    private void generateOutOfMemory() {
        Long maxMemory = Runtime.getRuntime().maxMemory();
        System.out.println(maxMemory);

        int[] matrix = new int[(int) (maxMemory + 1)];
        for (int i = 0; i < matrix.length; ++i) {
            matrix[i] = i + 1;
        }
        System.out.println("Matrix filled" + matrix[(int)(Math.random() *
            ↪ 100)]);
    }

    public static void main(String[] args) {
        DemoOutOfMemory doom = new DemoOutOfMemory();
        doom.generateOutOfMemory();
    }
}
```

Om de OutOfMemoryError te illustreren maken we een array aan die meer geheugen-plaatsen inneemt dan de ruimte die het java programma ter beschikking heeft.

Als je dit programma uitvoert, pas je best het beschikbare geheugen voor het programma aan. Dit doe je door een waarde voor de VM optie -Xmx mee te geven.

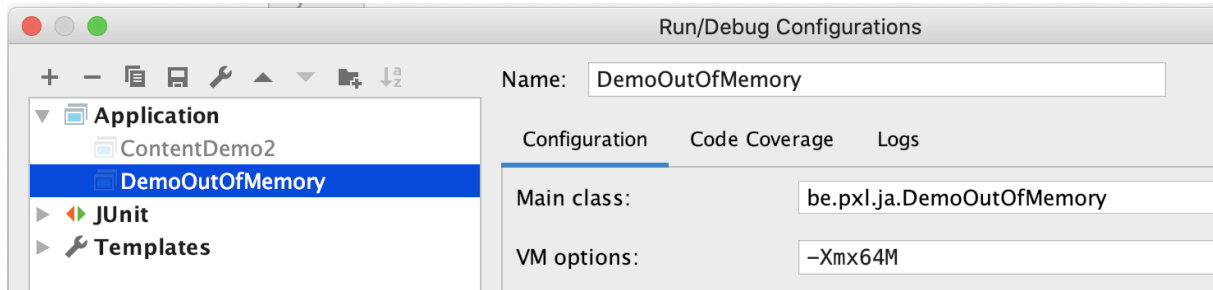
- -Xmssize: Geeft de initiële waarde voor de heap size.
- -Xmxsize: Geeft de maximale waarde voor heap size.

De heap size van een Java programma is de hoeveelheid geheugen dat een Java-programma

mag gebruiken om objecten op te slaan.

67108864

Exception in thread "main" java.lang.OutOfMemoryError: Java heap space
at be.pxl.ja.DemoOutOfMemory.generateOutOfMemory(DemoOutOfMemory.java:8)
at be.pxl.ja.DemoOutOfMemory.main(DemoOutOfMemory.java:16)<5 internal calls>



Figuur 4.5: Maximum heap size aanpassen

4.3.2 Runtime exceptions

Runtime exceptions zijn exceptions die vaak veroorzaakt worden door logische fouten in het programma. Typische voorbeelden van runtime exceptions zijn `ArrayIndexOutOfBoundsException`, `NullPointerException` en `IllegalArgumentException`. Vaak moet je ze niet afhandelen, maar moet je ervoor zorgen dat je de bug in je code oplost. Ook hier zijn onze unit testen heel belangrijk. Door goede unit testen te schrijven ga je runtime exceptions in je code opmerken en kunnen oplossen.

```
package be.pxl.ja;

public class Demo {

    public static void main(String[] args) {
        String tekst = "abc";
        System.out.println(tekst.repeat(-5));
    }
}
```

Exception in thread "main" java.lang.IllegalArgumentException: count is negative: -5
at java.base/java.lang.String.repeat(String.java:3586)
at be.pxl.ja.Demo.main(Demo.java:5)

Oefening 4.2.

```
@Service
public class MusicPlaylistService {

    private List<Song> playlist;

    public void addSong(Song song) {
        playlist.add(song);
    }
}
```

```
}  
}
```

Welke exception doet zich voor zodra je de methode `addSong()` aanroept? Waarom treedt die exception op?

Wanneer je programma gebruikmaakt van gegevens die door de gebruiker worden ingevoerd, moet je er altijd rekening mee houden dat de gebruiker foutieve gegevens kan ingeven. Deze foutieve gegevens kunnen aanleiding geven tot runtime exceptions. Ook hier moet je steeds anticiperen op de mogelijke input die de gebruiker kan invoeren.

```
package be.px1.ja;  
  
public class ElementInArray {  
  
    public static void main(String[] args) {  
        String[] elements = { "H", "He", "Li", "Be", "B", "C", "N", "O",  
                               ↪ "F", "Ne" };  
  
        Scanner scanner = new Scanner(System.in);  
  
        // OPLOSSING 1  
        System.out.println("Kies een nummer: ");  
        int chosen = scanner.nextInt();  
        if (chosen < elements.length) {  
            System.out.println(elements[chosen]);  
        } else {  
            System.out.println("U koos een verkeerd nummer.");  
        }  
  
        // OPLOSSING 2  
        System.out.println("Kies een nummer: ");  
        chosen = scanner.nextInt();  
        try {  
            System.out.println(elements[chosen]);  
        } catch (ArrayIndexOutOfBoundsException e) {  
            System.out.println("U koos een verkeerd nummer.");  
        }  
    }  
}
```

In bovenstaand codevoorbeeld gaat onze voorkeur uit naar oplossing 1 waarbij de ingevoerde waarde wordt gecontroleerd vooraleer het element uit de array wordt benaderd. Deze code is makkelijker leesbaar en onderhoudbaar.

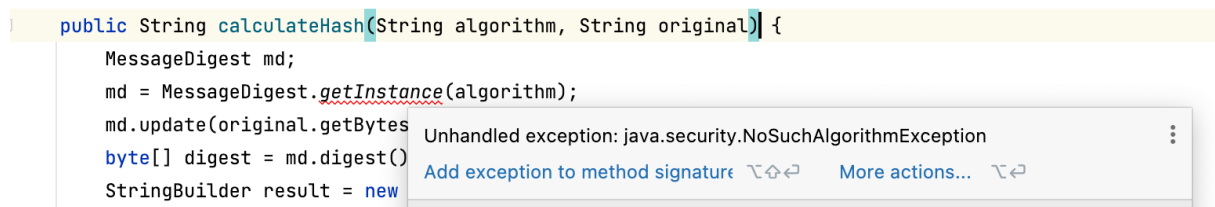
4.3.3 Checked exceptions

Wanneer je in java een methode aanroept, kan het voorkomen dat je direct een compileerfout voorgeschoteld krijgt. Het kan namelijk zijn dat java reeds anticipeert op mogelijke problemen en je dwingt om rekening te houden met het scenario dat er iets mis kan gaan. De compileerfout raakt pas opgelost wanneer je het afhandelen van de exception netjes programmeert.

Kijk eens naar onderstaand voorbeeld uit de streaming service. We gebruiken hier de klasse MessageDigest uit JDK. Message digests zijn functies waarmee we voor input-data van willekeurige lengte een hash-waarde met een vaste lengte kunnen berekenen. Als je de hash-waarde kent, kan je hieruit de input-data niet afleiden. We gebruiken dit om een paswoord bij te houden in een Account-object. We willen absoluut vermijden dat paswoorden in een leesbaar formaat in onze objecten worden bijgehouden.

Om een message digest te berekenen in Java moet je eerst de static methode getInstance() aanroepen met als parameter het door jouw gekozen algoritme. In dit voorbeeld wordt er gekozen voor MD5. Je ziet dat de lijn code, ondanks het feit dat alles correct is geschreven, toch rood wordt onderlijnd. Dat komt omdat de methode getInstance() een exception kan opgooien die we verplicht moeten afhandelen.

Oefening 4.3. Open de documentatie van de klasse MessageDigest en bekijk de uitleg voor de static methode getInstance(). Kan je hier zien welke exceptions er kunnen voorkomen?



Figuur 4.6: Een checked exception: NoSuchAlgorithmException

Deze compileerfout wordt veroorzaakt omdat de exception-klasse NoSuchAlgorithmException een checked exception is. Dit betekent dat het aanroepen van de methode die de exception gooit een compileerfout zal geven, omdat er geen code is toegevoegd om correct met de exception om te gaan. Er zijn 2 mogelijke oplossingen om deze compileerfout aan te pakken. Ofwel vang de exception op en handel je ze af in de methode calculateHash(), ofwel voeg je in de signatuur van de methode toe dat je de methode toelaat om een NoSuchAlgorithmException op te werpen.

```
package be.px1.demo;  
  
import org.slf4j.Logger;  
import org.slf4j.LoggerFactory;  
import org.springframework.stereotype.Service;  
  
import java.security.MessageDigest;
```

```

import java.security.NoSuchAlgorithmException;

@Service
public class HashGeneratorService {

    private static final Logger LOGGER =
        ↪ LoggerFactory.getLogger(HashGeneratorService.class);

    public String calculateHash(String algorithm, String original) throws
        ↪ NoSuchAlgorithmException {
        MessageDigest md;
        md = MessageDigest.getInstance(algorithm);
        md.update(original.getBytes());
        byte[] digest = md.digest();
        StringBuilder result = new StringBuilder();
        for (byte b : digest) {
            result.append(String.format("%02x", b));
        }
        return result.toString();
    }
}

```

```

package be.px1.demo.api;

import be.px1.demo.HashGeneratorService;
import be.px1.demo.exception.InvalidMd5Exception;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

import java.security.NoSuchAlgorithmException;

@RestController
public class MessageController {

    private final HashGeneratorService hashGeneratorService;

    public MessageController(HashGeneratorService hashGeneratorService) {
        this.hashGeneratorService = hashGeneratorService;
    }

    @GetMapping(value = "/hash/{algorithm}")
    public ResponseEntity<String> getHash(@PathVariable String algorithm,
        ↪ @RequestParam String text) {

```

```

        try {
            return
                ↳ ResponseEntity.ok(hashGeneratorService.calculateHash(algorithm,
                ↳ text));
        } catch (NoSuchAlgorithmException e) {
            return ResponseEntity.badRequest().build();
        }
    }
}

```

`ResponseEntity` is de representatie van het HTTP response: de statuscode, headers en body. Je kan deze klasse gebruiken als je een andere statuscode wilt geven dan 200 (ok).

Hier is een overzicht van de meest voorkomende HTTP-statuscodes.

- **200 OK:** Dit is de standaard statuscode voor een succesvolle HTTP-aanvraag. Gebruik deze code om aan te geven dat de aanvraag met succes is verwerkt en dat de responsebody een antwoord bevat. Deze statuscode wordt vaak gebruikt bij een succesvolle GET-request.
- **201 Created:** Deze code geeft aan dat de aanvraag met succes is verwerkt en dat er een nieuwe resource is aangemaakt. Het wordt vaak gebruikt bij succesvolle POST-requests.
- **204 No Content:** Deze statuscode geeft aan dat de aanvraag met succes is verwerkt, maar er is geen informatie om terug te geven. Dit wordt vaak gebruikt voor succesvolle DELETE-aanvragen.
- **400 Bad Request:** Gebruik deze code wanneer de client een ongeldige aanvraag heeft gedaan, bijvoorbeeld ontbrekende verplichte parameters of gegevens in een fout formaat.
- **401 Unauthorized:** Dit geeft aan dat de client niet is geautoriseerd om toegang te krijgen tot de bron. Meestal wordt dit gebruikt wanneer de client geen geldige authenticatiegegevens heeft verstrekt.
- **403 Forbidden:** Hiermee geef je aan dat de client weliswaar is geauthenticeerd, maar geen toegang heeft tot de gevraagde bron vanwege beperkingen in de rechten of toegangsregels.
- **404 Not Found:** Deze statuscode wordt gebruikt wanneer de gevraagde resources niet kan worden gevonden.
- **500 Internal Server Error:** Dit geeft aan dat er een onverwachte fout is opgetreden aan de kant van de server. Het wordt vaak gebruikt voor fouten die niet direct kunnen worden toegeschreven aan de client.

4.4 Multi-catch blok en finally

```
import java.util.Scanner;
```

```

public class MultiCatchBlockDemo {

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.println("Kies een positie: ");
        int positie = scanner.nextInt();
        System.out.println("Kies een deler: ");
        int deler = scanner.nextInt();
        try {
            int getallen[] = new int[10];
            getallen[positie] = 30 / deler;
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Je moet een positie kiezen tussen 0 en 9.");
        } catch (Exception e) {
            System.out.println(e.getMessage());
        } finally {
            System.out.println("Je koos positie " + positie);
        }
        System.out.println("Start je het programma nog een keer.");
    }
}

```

Een try-blok kan gevolgd worden door één of meerder catch-blokken. Wanneer een exception optreedt zal bij het eerste catch-blok gestart worden. Indien onze exception een instantie is van de opgevangen exception (instanceof) dan zal dat catch-blok uitgevoerd worden en worden de volgende catch-blokken niet meer bekeken. Indien de exception geen instantie is van de opgevangen exception dan wordt er verder gekeken naar de volgende catch-blokken tot een overeenkomstig catch-blok worden gevonden. Indien er geen catch-blok wordt gevonden zal de runtime-exception doorstromen naar de aanroepende methode.

De volgorde van de catch-blokken is van belang. `ArrayIndexOutOfBoundsException` is een subklasse van `Exception`. Als we eerst een catch-blok aanmaken voor de superklasse en pas daarna een catch-blok voor de subklasse zou het tweede catch-blok “onbereikbaar” zijn. Het eerste catch-blok gaat de exception reeds kunnen afhandelen. Code die onbereikbaar is (unreachable code) wordt opgemerkt door de compiler wat resulteert in een compileerfout van je code.

Oefening 4.4. Wissel beide catch-blokken eens van plaats in bovenstaande code.

Het finally-blok tenslotte is een codeblok dat altijd wordt uitgevoerd: of er nu een exception optreedt of niet. Zelfs als er een exception optreedt en die niet kan worden afgehandeld door een catch-blok, zal toch het finally-blok uitgevoerd worden.

Oefening 4.5. Test de werking van het finally-blok eens uit met bovenstaand programma `MultiCatchBlockDemo`. Verwijder het tweede catch-blok eens en veroorzaak een deling door 0. Wat gebeurt er?

Indien je dezelfde code hebt om verschillende exceptions af te handelen, mag je exceptions combineren in een catch-block. Zo zal het catch-blok in onderstaand voorbeeld zowel `ArrayIndexOutOfBoundsException` als `ArithmeticException` afhandelen.

```
public class MultipleCatches {

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.println("Kies een positie: ");
        int positie = scanner.nextInt();
        System.out.println("Kies een deler: ");
        int deler = scanner.nextInt();
        try {
            int getallen[] = new int[10];
            getallen[positie] = 30 / deler;
        } catch (ArrayIndexOutOfBoundsException | ArithmeticException e) {
            System.out.println(e.getMessage());
        }
        System.out.println("Start je het programma nog een keer.");
    }
}
```

4.5 Eigen unchecked exceptions

We willen een endpoint toevoegen om te controleren of een MD5 hashcode correct is. Wanneer de berekende hashcode en de opgegeven hashcode niet overeenkomen gooien we een `InvalidMd5Exception` op. Dit is een eigen unchecked exception.

```
package be.px1.demo.exception;

public class InvalidMd5Exception extends RuntimeException {
    public InvalidMd5Exception() {
    }

    public InvalidMd5Exception(String message) {
        super(message);
    }

    public InvalidMd5Exception(String message, Throwable cause) {
        super(message, cause);
    }
}
```

```
package be.px1.demo;

import be.px1.demo.exception.InvalidMd5Exception;
import org.slf4j.Logger;
```

```

import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Service;

import java.security.MessageDigest;
import java.security.NoSuchAlgorithmException;

@Service
public class HashGeneratorService {

    private static final Logger LOGGER =
        ↪ LoggerFactory.getLogger(HashGeneratorService.class);

    public String calculateHash(String algorithm, String original) throws
        ↪ NoSuchAlgorithmException {
        MessageDigest md;
        md = MessageDigest.getInstance(algorithm);
        md.update(original.getBytes());
        byte[] digest = md.digest();
        StringBuilder result = new StringBuilder();
        for (byte b : digest) {
            result.append(String.format("%02x", b));
        }
        return result.toString();
    }

    public void verifyMd5(String original, String hash) {
        try {
            if (!calculateHash("MD5", original).equals(hash)) {
                throw new InvalidMd5Exception();
            }
        } catch (NoSuchAlgorithmException e) {
            LOGGER.error("Unexpected exception", e);
            throw new InvalidMd5Exception("Invalid algorithm MD5", e);
        }
    }
}

```

```

package be.px1.demo.api;

import be.px1.demo.HashGeneratorService;
import be.px1.demo.exception.InvalidMd5Exception;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

```



```

import java.security.NoSuchAlgorithmException;

@RestController
public class MessageController {

    private final HashGeneratorService hashGeneratorService;

    @GetMapping(value = "/hash/{algorithm}")
    public ResponseEntity<String> hash(@PathVariable String algorithm,
        ↪ @RequestParam String text) {

        try {
            return
                ↪ ResponseEntity.ok(hashGeneratorService.calculateHash(algorithm,
                ↪ text));
        } catch (NoSuchAlgorithmException e) {
            return ResponseEntity.badRequest().build();
        }
    }

    @PostMapping(value = "/verify-md5")
    public ResponseEntity<Void> getMD5(@RequestBody RequestData data) {
        try {
            hashGeneratorService.verifyMd5(data.original(), data.md5());
            return ResponseEntity.noContent().build();
        } catch (InvalidMd5Exception e) {
            return ResponseEntity.status(418).build();
        }
    }
}

```

Hoofdstuk 5

Spring Validation

Learning goals

De junior-collega

1. kan gebruikmaken van Spring Validation
2. kan de correcte validatieregels gebruiken

Wanneer Spring Boot een HTTP-verzoek ontvangt, is het belangrijk om de gegevens die met dat verzoek zijn meegestuurd te valideren. Spring Boot heeft voorgedefinieerde validatoren waardoor het controleren van veelvoorkomende regels eenvoudig kan gebeuren. Zo kan Spring Boot bijvoorbeeld heel eenvoudig controleren of een e-mailadres het juiste formaat heeft of een getal binnen een bepaald bereik valt,. In Spring Boot is het uiteraard ook mogelijk om aangepaste of eigen validatieregels te programmeren.

Om Spring Validation te gebruiken voeg je een dependency toe aan het project.

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

Hier is een lijst van de meest gebruikte annotaties.

@NotNull	om aan te duiden dat een referentie niet null mag zijn.
@NotEmpty	om aan te duiden dat een list elementen moet bevatten.
@NotBlank	om aan te duiden dat een string niet leeg mag zijn.
@Min and @Max	om aan te duiden dat een numerieke waarde enkel geldig is als de waarde groter of kleiner is dan een gegeven waarde.
@Size	om de lengte van een string te valideren.
@Email	om te controleren of een string een geldig e-mailadres bevat.

```
package be.px1.demo.domain;

import com.fasterxml.jackson.annotation.JsonProperty;
import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
```

```

public class Song {
    @NotBlank
    private String title;
    @NotBlank
    private String artist;
    @JsonProperty("duration_seconds")
    @Min(value = 1, message = "Must be greater than zero")
    private int durationSeconds;
    @NotNull
    private Genre genre;

    ...
}

```

Wanneer Spring Boot een argument vindt dat is geannoteerd met @Valid, wordt het automatisch gevalideerd.

```

@RestController
@RequestMapping("/playlist/songs")
public class MusicPlaylistController {

    private static final Logger LOGGER =
        ↪ LoggerFactory.getLogger(MusicPlaylistController.class);
    private final MusicPlaylistService musicPlaylistService;

    @Autowired
    public MusicPlaylistController(MusicPlaylistService
        ↪ musicPlaylistService) {
        this.musicPlaylistService = musicPlaylistService;
    }

    @PostMapping
    public void addSong(@RequestBody @Valid Song song) {
        if (LOGGER.isInfoEnabled()) {
            LOGGER.info("Adding song [" + song.getTitle() + "]");
        }
        musicPlaylistService.addSong(song);
    }

    ...
}

```

Wanneer het argument niet voldoet aan de regels dan gooit Spring Boot een exception van de klasse MethodArgumentNotValidException.

Sinds Spring Boot versie 2.3 moet je de eigenschap server.error.include-message de waarde always geven om de foutboodschappen te kunnen zien in het response. Spring Boot verbergt namelijk het foutmelding in de respons om het lekken van gevoelige informatie te voorkomen.

De foutmelding blijft ondanks in de instelling eerder cryptisch. Stel dat ik de REST API gebruik om een lied toe te voegen met de volgende gegevens:

```
{
  "title": "",
  "artist": "Bruno Mars",
  "duration_seconds": -2,
  "genre": "OTHER"
}
```

Dan krijg je het volgende respons.

```
{
  "timestamp": "2023-09-23T13:48:50.490+00:00",
  "status": 400,
  "error": "Bad Request",
  "message": "Validation failed for object='song'. Error count: 2",
  "path": "/playlist/songs"
}
```

Meer gegevens krijg ik te zien als ik de eigenschap `server.error.include-binding-errors=always` toevoeg in `application.properties`. Voor bovenstaande `RequestBody` krijg je dan de volgende respons. Er wordt op dat ogenblik al veel interne informatie gelekt.

```
1 {
2   "timestamp": "2023-09-23T13:52:38.028+00:00",
3   "status": 400,
4   "error": "Bad Request",
5   "message": "Validation failed for object='song'. Error count: 2",
6   "errors": [
7     {
8       "codes": [
9         "Min.song.durationSeconds",
10        "Min.durationSeconds",
11        "Min.int",
12        "Min"
13      ],
14      "arguments": [
15        {
16          "codes": [
17            "song.durationSeconds",
18            "durationSeconds"
19          ],
20          "arguments": null,
21          "defaultMessage": "durationSeconds",
22          "code": "durationSeconds"
23        },
24        10
25      ],
26      "defaultMessage": "Invalid duration_seconds: must equal or
```

```

27         ↪ exceed 10.",
28         "objectName": "song",
29         "field": "durationSeconds",
30         "rejectedValue": -2,
31         "bindingFailure": false,
32         "code": "Min"
33     },
34     {
35         "codes": [
36             "NotBlank.song.title",
37             "NotBlank.title",
38             "NotBlank.java.lang.String",
39             "NotBlank"
40         ],
41         "arguments": [
42             {
43                 "codes": [
44                     "song.title",
45                     "title"
46                 ],
47                 "arguments": null,
48                 "defaultMessage": "title",
49                 "code": "title"
50             }
51         ],
52         "defaultMessage": "must not be blank",
53         "objectName": "song",
54         "field": "title",
55         "rejectedValue": "",
56         "bindingFailure": false,
57         "code": "NotBlank"
58     }
59 ],
60 "path": "/playlist/songs"
}

```

De best practice om de `MethodArgumentNotValidException` af te handelen is door gebruik te maken van een exception handler. Hierbij gebruiken we de annotaties `@RestControllerAdvice` en `@ExceptionHandler`.

```

package be.px1.demo.config;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.validation.ObjectError;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

```

```

import java.util.ArrayList;
import java.util.List;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<List<String>>
        ↪ handleValidationErrors(MethodArgumentNotValidException ex) {
        List<String> errors = new ArrayList<>();
        for (ObjectError error: ex.getBindingResult().getAllErrors()) {
            errors.add(error.getDefaultMessage());
        }
        return new ResponseEntity<>(errors, HttpStatus.BAD_REQUEST);
    }
}

```

De annotatie `@RestControllerAdvice` geeft aan dat deze klasse verantwoordelijk is voor het afhandelen van exceptions. Alle exception worden onderschept en indien er voor het type exception een handler is gedefinieerd, zal de bijhorende code worden uitgevoerd.

In deze klasse is er enkel een methode voorzien voor het afhandelen van `MethodArgumentNotValidExceptions`. De methode herkennen we aan de annotatie `@ExceptionHandler`. Binnen de methode wordt een lijst met de foutmeldingen (errors) gegenereerd.

We passen de foutboodschap bij iedere validatie ook best aan. Gebruik een duidelijke foutboodschap om de gebruiker duidelijk te maken welke gegevens hij moet doorgeven. Als je een meertalige applicatie hebt, kan je best een key (zoals `title.not.blank`) doorgeven aan de frontend zodat de eigenlijk foutboodschap in de juiste taal getoond kan worden.

```

@NotBlank(message = "title.not.blank")
private String title;
@Min(value = 10, message = "Invalid duration_seconds: must be equal to
    ↪ or exceed 10.")
private int durationSeconds;

```

Er wordt in het geval van een `MethodArgumentNotValidException` een HTTP status 400 met de volgende response body gegeven:

```

[
  "Invalid duration_seconds: must be equal to or exceed 10.",
  "title.not.blank",
  "Please provide artist name."
]

```

Oefening 5.1. Huizenjacht

We voeren een aantal aanpassingen uit aan de klasse `House`.

- Voeg de volgende eigenschappen toe:
 - area: oppervlakte van de woning in m²

- epcScore: waarde uit de opsomming EPCScore.
- Verwijder de eigenschap price. De prijs is nu een berekende waarde. Vermenigvuldig de oppervlakte met de basisprijs per m² en het percentage behorende bij de epc score. De huidige basisprijs per m² is 2356.75. Voor huizen in Hasselt en Genk is er een meerprijs van 25%.
- Voeg de methode markAsSold() toe. Deze methode verandert de status van het huis van FOR_SALE naar SOLD. Wanneer het huis al verkocht is gooi je een IllegalStateException op.
- Pas vervolgens de POST en PUT requests aan. Bij het toevoegen van een huis zijn de volgende velden verplicht: code, name, area (minstens 50m²), epc-score en city. Bij het updaten kan je enkel volgende velden aanpassen: name, area (minstens 50m²), city en epc-score.
- Voorzie een aangepast endpoint /houses/{code}/sold om een huis als verkocht te registreren.

Test alle endpoints grondig uit.

```
public enum EPCScore {
    A_PLUS(1.5),
    A(1.2),
    B(1),
    C(0.98),
    D(0.90),
    E(0.80),
    F(0.75);

    private double percentage;

    EPCScore(double percentage) {
        this.percentage = percentage;
    }

    public double getPercentage() {
        return percentage;
    }
}
```

Hoofdstuk 6

Unit testen met JUnit

De software die we ontwikkelen moet kwaliteitsvol zijn. Maar hoe kunnen we er nu voor zorgen dat we het aantal bugs in onze code zo laag mogelijk houden? Eén van de strategieën om de kwaliteit van software te waarborgen is testen. Testen is een heel belangrijk aspect van softwareontwikkeling waar je ongetwijfeld nog heel veel over gaat leren.

Als ontwikkelaar zorg je er altijd voor dat wanneer je de gevraagde functionaliteit implementeert, je ook de nodige unit testen schrijft om de kwaliteit van je code te garanderen. Wanneer je unit testen schrijft, ga je alle methoden van je klasse testen. Een methode van een klasse is dus de “unit” die we gaan testen. We gaan automatische testen schrijven. Dit betekent dat iedere keer dat de code wordt aangepast of uitgebreid de testen eenvoudig opnieuw kunnen worden uitgevoerd. Op die manier gaan de fouten die ontstaan tijdens de verdere ontwikkeling van de software snel ontdekt worden. Bedenk ook dat een fout, onduidelijkheid of bug die vroeg in het ontwikkelproces wordt aangepakt, maar een kleine impact heeft (ook financieel) ten opzicht van problemen die later pas opduiken.

Als je unit testen schrijft hoef je geen schrik te hebben om achteraf code aan te passen en te verbeteren. Omdat je beschikt over unit testen kan je rustig aan je code sleutelen. Het proces om code te verbeteren en hierdoor de leesbaarheid en onderhoudbaarheid van de code te verhogen noemen we **refactoren**.

6.1 JUnit

In Java gebruiken we het framework JUnit5 om onze unit testen te schrijven. JUnit5 is opgedeeld in 3 sub-projecten: Jupiter, Vintage en Platform. JUnit Jupiter is het sub-project dat wij gebruiken om onze testen te schrijven en voorziet de engine om deze testen uit te voeren.

De dependency spring-boot-starter-test wordt altijd toegevoegd aan een Spring Boot project.

```
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-test</artifactId>
<scope>test</scope>
```


</dependency>

De scope test is belangrijk en duidt erop dat de dependency niet nodig is voor het normale gebruik van de toepassing. De dependency is enkel nodig voor het compileren en uitvoeren van de testen.

6.2 De klasse House

We gaan nu unit testen schrijven voor onderstaande klasse House.

```
package be.px1.domain;

public class House {
    private static final double BASE_PRICE = 2356.75;
    private final String code;
    private String description;
    private int area;
    private String city;
    private Status status;
    private EPCCScore epcScore;

    public House(String code, String description, int area, EPCCScore
        ↪ epcScore) {
        this.code = code;
        this.description = description;
        this.area = area;
        this.epcScore = epcScore;
        this.status = Status.FOR_SALE;
    }

    public String getDescription() {
        return description;
    }

    public void setDescription(String description) {
        this.description = description;
    }

    public void setEpcScore(EPCCScore epcScore) {
        this.epcScore = epcScore;
    }

    public String getCode() {
        return code;
    }

    public int getArea() {
        return area;
    }
}
```

```

    }

    public void setArea(int area) {
        this.area = area;
    }

    public String getCity() {
        return city;
    }

    public Status getStatus() {
        return status;
    }

    public EPSCore getEpcScore() {
        return epcScore;
    }

    public void setCity(String city) {
        this.city = city;
    }

    public void markAsSold() {
        if (Status.FOR_SALE == status) {
            status = Status.SOLD;
        } else {
            throw new IllegalStateException("House is already marked as
            ↪ sold.");
        }
    }

    public double getPrice() {
        double price = area * BASE_PRICE * epcScore.getPercentage();
        if ("Hasselt".equals(city) || "Genk".equals(city)) {
            price *= 1.25;
        }
        return price;
    }
}

```

```

package be.px1.domain;

public enum EPSCore {
    A_PLUS(1.5),
    A(1.2),
    B(1),
    C(0.98),
    D(0.90),

```

```

    E(0.80),
    F(0.75);

    private double percentage;

    EPCTScore(double percentage) {
        this.percentage = percentage;
    }

    public double getPercentage() {
        return percentage;
    }
}

```

6.3 Klasse met de unit testen

In een Spring Boot-project met Maven als build tool worden de klassen met unit testen in een specifieke mapstructuur geplaatst.

De conventie is om testklassen in de map **src/test/java** te plaatsen. De testklassen moeten zich in deze map bevinden, georganiseerd in dezelfde packages als de bronklassen.

Bijvoorbeeld, als de bronklassen zich in het package `be.pxl.myapp` bevinden, moeten de testklassen zich bevinden in de folder `src/test/java/be/pxl/myapp`.

Het is handig om de testklassen namen te geven die eindigen op **Test** om duidelijk aan te geven dat het om testen gaat. De testen voor de klasse `MyService` noem je dus `MyServiceTest`.

Maak nu de klasse `HouseTest` in het package `be.pxl.domain` in de folder `src/test/java`.

6.4 Unit test voor de constructor

We gaan starten met een test te schrijven om de constructor te testen. We controleren of alle velden correct worden ingevuld als we de constructor hebben aangeroepen. Verder controleren we ook dat de `city` geen waarde krijgt en de initiële waarde voor de status van het huis `FOR_SALE` is.

```

package be.pxl.domain;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class HouseTest {

    @Test
    public void testConstructor() {

```

```

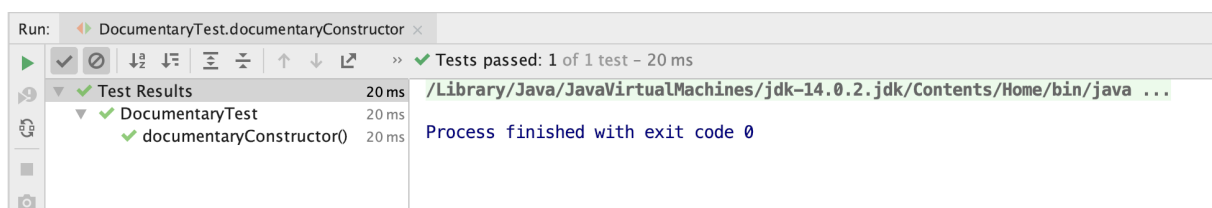
House house = new House("XYZ789", "2-bedroom apartment", 100,
    ↪ EPCScore.B);

assertEquals("XYZ789", house.getCode());
assertEquals("2-bedroom apartment", house.getDescription());
assertEquals(100, house.getArea());
assertEquals(EPCScore.B, house.getEpcScore());
assertEquals(Status.FOR_SALE, house.getStatus());
assertNull(house.getCity()); // City is not set initially
}
}

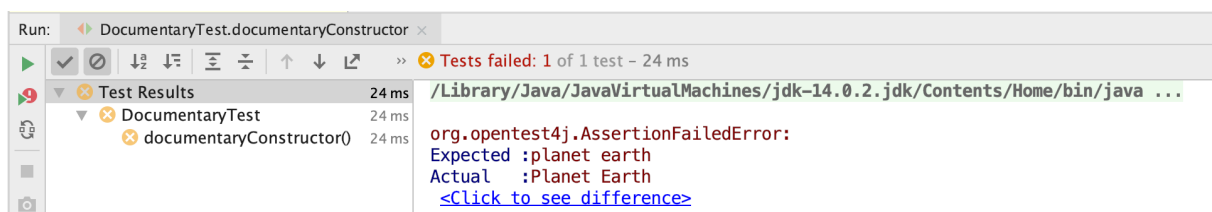
```

We voegen bij een methode in de testklasse de annotatie `@Test` toe zodat de test herkend wordt en uitgevoerd kan worden.

Wanneer je de testklasse `HouseTest` uitvoert kunnen er 3 mogelijke scenario's plaatsvinden. Ofwel slaagt de test, ofwel faalt één van de beweringen (asserts) ofwel loopt er iets onverwachts fout. In het eerste geval krijgt de test een groene kleurcode, in het tweede scenario een oranje en in het laatste scenario een rode.



Figuur 6.1: Geslaagde JUnit5 test



Figuur 6.2: Gefaalde JUnit5 test

Hier is een overzicht van enkele handige static methoden uit de klasse `org.junit.jupiter.api.Assertions`.

6.5 Unit test voor een setter

Een unit test wordt steeds opgebouwd volgens het 3A patroon: Arrange, Act en Assert. In iedere test herken je steeds deze 3 bouwstenen.

Het “arrange”-gedeelte is waar het object dat getest moet worden en alle andere objecten - die nodig zijn om de test goed uit te voeren - worden aangemaakt. Het “act”-gedeelte is waar we de methode die we willen testen aanroepen. Als de methode een returnwaarde heeft ken je dit toe aan een variabele.

Methode	Betekenis
assertEquals()	Evalueert de gelijkheid van 2 waarden. De test slaagt als beide waarden gelijk (equal) zijn.
assertFalse()	Evaluatie van een booleaanse uitdrukking. De test slaagt indien de uitdrukking false is.
assertTrue()	Evaluatie van een booleaanse uitdrukking. De test slaagt indien de uitdrukking true is.
assertNotNull()	Vergelijkt een object referentie met null. De test slaagt indien de object referentie niet null is.
assertSame()	Vergelijkt het geheugenadres van twee object referenties (gebruik maken van == operator). De test slaagt indien beide object referenties naar hetzelfde object verwijzen.
fail()	Zorgt ervoor dat de test zal falen.

Tabel 6.1: Static methoden uit de klasse org.junit.jupiter.api.Assertions

Het “assert”-gedeelte geeft je de mogelijkheid om beweringen over het resultaat te verifiëren.

Laten we de methode `setCity()` testen. We controleren dat een object, na het aanroepen van de methode `setCity` met een gekozen parameter, correct werd aangepast.

```
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class HouseTest {

    @Test
    public void testSetCity() {
        House house = new House("ABC123", "3-bedroom house", 150,
            ↪ EPCScore.GOOD);
        house.setCity("Dilsen-Stokkem");
        assertEquals("Dilsen-Stokkem", house.getCity());
    }
}
```

6.6 @BeforeEach

De annotatie @BeforeEach wordt gebruikt om een methode aan te duiden die vóór elke testmethode in de testklasse wordt uitgevoerd. Deze methode wordt gebruikt voor herhaalbare initialisatiecode die nodig is voor elke (of bijna elke) testmethode. Het vermindert duplicatie van code door gedeelde initialisatie te centraliseren. Deze methode heeft geen parameters en geen returnwaarde.

Oefening 6.1. Voeg een test toe voor iedere setter in de klasse House.

6.7 Unit test voor een berekende waarde

In de klasse House vind je de methode getPrice() terug. Deze methode bevat een berekening die we uiteraard willen testen.

Bedenk welke scenario's er kunnen voorkomen en zorg dat je een test schrijft voor elk scenario.

```
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class HouseTest {
    private House house;

    @BeforeEach
    public void setUp() {
        house = new House("ABC123", "3-bedroom house", 250, EPSScore.B);
    }

    @Test
    public void testGetPrice() {
        house.setArea(150);
        double expectedPrice = 150 * House.BASE_PRICE *
            ↳ EPSScore.B.getPercentage();
        assertEquals(expectedPrice, house.getPrice(), 0.001);
    }

    @Test
    public void testGetPriceWithCity() {
        house.setArea(150);
        house.setCity("Hasselt");

        double expectedPrice = 150 * House.BASE_PRICE *
            ↳ EPSScore.B.getPercentage() * 1.25;
        assertEquals(expectedPrice, house.getPrice(), 0.001);
    }
}
```

```
}
```

6.8 Testen van exceptions die gegooid worden

Wanneer we een verkocht huis (met status SOLD) nog een keer als verkocht willen registreren, dan wordt een `IllegalStateException` opgegooid. We kunnen het opgooien van de exception testen met de `assertThrows()` methode.

```
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class HouseTest {
    private House house;

    @BeforeEach
    public void setUp() {
        house = new House("ABC123", "3-bedroom house", 250, EPSScore.B);
    }

    @Test
    public void testMarkAsSold() {
        house.markAsSold();
        assertEquals(Status.SOLD, house.getStatus());
    }

    @Test
    public void testMarkAsSoldWhenAlreadySold() {
        house.markAsSold();
        assertThrows(IllegalStateException.class, () -> house.markAsSold());
    }
}
```

6.9 Overzicht van alle testen voor de klasse House

```
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class HouseTest {
    private House house;

    @BeforeEach
    public void setUp() {
        house = new House("ABC123", "3-bedroom house", 150, EPSScore.GOOD);
    }
}
```

```

}

@Test
public void testConstructor() {
    House house = new House("XYZ789", "2-bedroom apartment", 100,
        ↪ EPCScore.B);

    assertEquals("XYZ789", house.getCode());
    assertEquals("2-bedroom apartment", house.getDescription());
    assertEquals(100, house.getArea());
    assertEquals(EPCScore.B, house.getEpcScore());
    assertEquals(Status.FOR_SALE, house.getStatus());
    assertNull(house.getCity()); // City is not set initially
}

@Test
public void testSetDescription() {
    house.setDescription("4-bedroom house");
    assertEquals("4-bedroom house", house.getDescription());
}

@Test
public void testSetEpcScore() {
    house.setEpcScore(EPCScore.D);
    assertEquals(EPCScore.D, house.getEpcScore());
}

@Test
public void testSetArea() {
    house.setArea(200);
    assertEquals(200, house.getArea());
}

@Test
public void testGetCity() {
    house.setCity("Dilsen-Stokkem");
    assertEquals("Dilsen-Stokkem", house.getCity());
}

@Test
public void testMarkAsSold() {
    house.markAsSold();
    assertEquals(Status.SOLD, house.getStatus());
}

@Test
public void testMarkAsSoldWhenAlreadySold() {
    house.markAsSold();
}

```



```

        assertThrows(IllegalStateException.class, () -> house.markAsSold());
    }

    @Test
    public void testGetPrice() {
        house.setArea(150);
        double expectedPrice = 150 * House.BASE_PRICE *
            ↳ EPCScore.B.getPercentage();
        assertEquals(expectedPrice, house.getPrice(), 0.001);
    }

    @Test
    public void testGetPriceWithCity() {
        house.setCity("Hasselt");

        double expectedPrice = 150 * House.BASE_PRICE *
            ↳ EPCScore.B.getPercentage() * 1.25;
        assertEquals(expectedPrice, house.getPrice(), 0.001);
    }
}

```

Hoofdstuk 7

Lambda expressies en Streams

De Stream API van Java biedt een elegante manier om collections te manipuleren. De belangrijkste interface uit de API is `Stream<T>`. Als Java ontwikkelaar gebruik je voornamelijk deze interface die alle implementatie-details verbergt. Bij het ontwerpen van de Stream API is, naast het aanbieden van een elegante en eenvoudige API, ook veel aandacht besteed aan performantie.

7.1 Functionele interfaces

7.1.1 De interface `StringConverter`

```
@FunctionalInterface
public interface StringConverter {
    String convert(String original);
}
```

De interface `StringConverter` is een functionele interface. Een functionele interface is een interface met precies één abstracte methode. Het markeren van een functionele interface met de annotatie `@FunctionalInterface` is optioneel, maar het wordt wel aanbevolen om duidelijk te maken dat het de bedoeling is dat er maar één abstracte methode in de interface staat. De compiler waarschuwt dan als de interface niet langer een functionele interface is.

Een functionele interface mag standaardmethoden (default methoden) en statische methoden hebben. Er mag echter maar één abstracte methode zijn.

We maken nu de klasse `UpperCaseConverter` die de interface `StringConverter` implementeert. De `convert` methode zorgt ervoor dat de uppercase-versie van de originele string wordt aangemaakt en teruggegeven.

```
public class UpperCaseConverter implements StringConverter{

    @Override
    public String convert(String original) {
        return original.toUpperCase();
    }
}
```

```
}  
}
```

```
public class Demo {  
  
    public static void main(String[] args) {  
        StringConverter upperCaseConverter = new UpperCaseConverter();  
        System.out.println(upperCaseConverter.convert("LuchtHavenPerSOneeL"));  
    }  
  
}
```

Stel dat we nu ook een LowerCaseConverter, een ReverseConverter, ... willen aanmaken. Voor iedere functionaliteit die je nodig hebt, moet je een nieuwe klasse maken die de StringConverter-interface implementeert. Best wel wat werk, zeker als je de klassen niet gaat hergebruiken. Gelukkig kan het eenvoudiger. Je kan namelijk anonieme innerklassen maken.

Je definieert en implementeert direct een instantie van een klasse zonder de klasse expliciet te benoemen.

Hier gaan we:

```
public class Demo {  
  
    public static void main(String[] args) {  
        StringConverter upperCaseConverter = new StringConverter() {  
            @Override  
            public String convert(String original) {  
                return original.toUpperCase();  
            }  
        };  
        StringConverter reverseConverter = new StringConverter() {  
            @Override  
            public String convert(String original) {  
                StringBuilder temporary = new StringBuilder(original);  
                return temporary.reverse().toString();  
            }  
        };  
        System.out.println(upperCaseConverter.convert("LuchtHavenPerSOneeL"));  
        System.out.println(reverseConverter.convert("LuchtHavenPerSOneeL"));  
    }  
  
}
```

Je kan voor iedere interface of abstracte klasse een anonieme innerklasse gebruiken. Omdat we in dit geval te maken hebben met een functionele interface, kunnen we nog een stap verder gaan en een **lambda expressie** schrijven.

Het proces om de anonieme innerklasse om te vormen naar een lambda expressie omvat

het verwijderen van overhead en het vereenvoudigen van de syntax.

```
public class Demo {  
  
    public static void main(String[] args) {  
        StringConverter upperCaseConverter = (original) ->  
            ↪ original.toUpperCase();  
        StringConverter reverseConverter = (original) -> {  
            StringBuilder temporary = new StringBuilder(original);  
            return temporary.reverse().toString();  
        };  
        System.out.println(upperCaseConverter.convert("LuchtHavenPerSOneel"));  
        System.out.println(reverseConverter.convert("LuchtHavenPerSOneel"));  
    }  
}
```

In de bovenstaande code hebben we de anonieme innerklassen vervangen door lambda-expressies. De lambda-expressies hebben de volgende onderdelen:

- de lijst met parameters - in dit geval (original), maar kan ook leeg zijn () of meerdere parameters bevatten.
- -> scheidt de parameters van de expressie
- expressie

We bereiken hetzelfde eindresultaat met aanzienlijk minder boilerplate code en een eenvoudigere leesbare syntax.

Oefening 7.1. Gegeven de volgende functionele interface:

```
interface Calculator {  
    int calculate(int n);  
}
```

Maak een programma waarmee je 2 lambda expressies maakt met deze functionele interface. De eerste lambda expressie kan je gebruiken om de 2de-macht van het getal te bereken. De twee lambda expressie gebruik je om de 3de-macht van het getal te berekenen.

7.1.2 Function<T, R>

De generieke functionele interface Function voorziet een functie apply die een argument aanneemt en een resultaat produceert. Er zijn dus 2 verschillende generieke datatypes:

- T: het type van het argument
- R het type van het resultaat

```
public class FunctionExample {
```

```

public static void main(String[] args) {
    // Define a Function that converts a String to its length (an
    //   ↪ Integer)
    Function<String, Integer> stringLengthFunction = str ->
        ↪ str.length();

    // Apply the function to a String
    String inputString = "Hello, Function!";
    int length = stringLengthFunction.apply(inputString);

    System.out.println("Length of the string: " + length);
}
}

```

7.1.3 Consumer<T>

De generieke functionele interface Consumer<T> bevat een functie accept die één argument aanneemt. De functie geeft geen resultaat terug.

```

public class ConsumerExample {
    public static void main(String[] args) {
        List<String> names = new ArrayList<>();
        names.add("Alice");
        names.add("Bob");
        names.add("Charlie");

        // Define a Consumer to print names
        Consumer<String> printName = name -> System.out.println("Name: " +
            ↪ name);

        // Iterate through the list and apply the Consumer using forEach
        names.forEach(printName);
    }
}

```

```

public class ConsumerExample {
    public static void main(String[] args) {
        List<String> data = new ArrayList<>();
        data.add("Item 1");
        data.add("Item 2");
        data.add("Item 3");

        // Define a Consumer to process and modify elements
        Consumer<String> processItem = item -> {
            // Append " - Processed" to each item
            data.set(data.indexOf(item), item + " - Processed");
        };
    }
}

```

```

// Process and modify elements using the Consumer
data.forEach(processItem);

System.out.println("Processed Data: " + data);
}
}

```

7.1.4 Predicate<T>

Predicate<T> is een veelgebruikte functionele interface die een functie **test** voorziet met één parameter met datatype T die een boolean-waarde (true of false) teruggeeft. Naast de abstracte methode test zijn er ook nog enkele default functies voorzien om Predicates te combineren met and, or en negate (not).

Methode	Betekenis
boolean test(T t)	It evaluates this predicate on the given argument.
default Predicate<T> and(Predicate<? super T> other)	It returns a composed predicate that represents a short-circuiting logical AND of this predicate and another. When evaluating the composed predicate, if this predicate is false, then the other predicate is not evaluated.
default Predicate<T> or(Predicate<? super T> other)	It returns a composed predicate that represents a short-circuiting logical OR of this predicate and another. When evaluating the composed predicate, if this predicate is true, then the other predicate is not evaluated.
default Predicate<T> negate()	It returns a predicate that represents the logical negation of this predicate.

```

public class PredicateExample {
    public static void main(String[] args) {
        List<String> names = new ArrayList<>();
        names.add("Alice");
        names.add("Bob");
        names.add("Charlie");
        names.add("Anna");
    }
}

```

```

    Predicate<String> startsWithA = name -> name.startsWith("A");
    Predicate<String> endsWithe = name -> name.endsWith("e");

    Predicate<String> startsWithAAndEndsWithe =
        ↪ startsWithA.and(endsWithe);

    System.out.println(startsWithA.test("Alice"));
    System.out.println(startsWithAAndEndsWithe.test("Alice"));

    printElements(names, startsWithA);
}

public static void printElements(List<String> list, Predicate<String>
    ↪ predicate) {
    for (String item : list) {
        if (predicate.test(item)) {
            System.out.println(item);
        }
    }
}
}

```

7.2 External en internal iterators

```

package be.px1.music.service;

import be.px1.music.api.Genre;
import be.px1.music.api.Song;
import org.springframework.stereotype.Service;

import java.util.ArrayList;
import java.util.List;

@Service
public class MusicPlaylistService {
    private List<Song> playlist = new ArrayList<>();

    ...

    public List<Song> getSongsByGenre(Genre genre) {
        List<Song> result = new ArrayList<>();
        for (Song song: playlist) {
            if (song.getGenre() == genre) {
                result.add(song);
            }
        }
    }
}

```

```

        return result;
    }
}

```

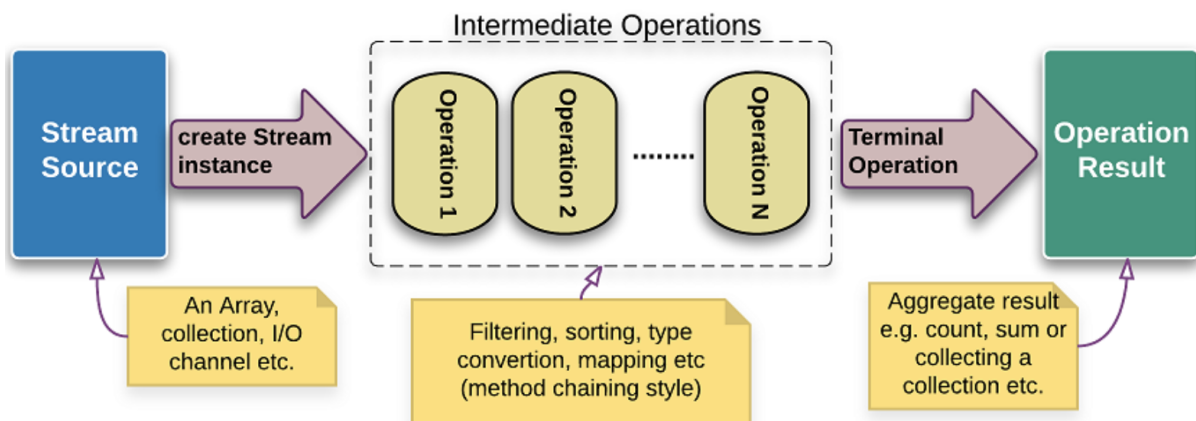
Een stream maakt het mogelijk om op een functionele manier complexe bewerkingen uit te voeren op een verzameling. We kunnen bovenstaande code schrijven met behulp van een stream. Onze external iterator verdwijnt en we krijgen een internal iterator in de plaats.

```

public List<Song> getSongsByGenre(Genre genre) {
    return playlist.stream().filter(song -> song.getGenre() ==
        ↳ genre).toList();
}

```

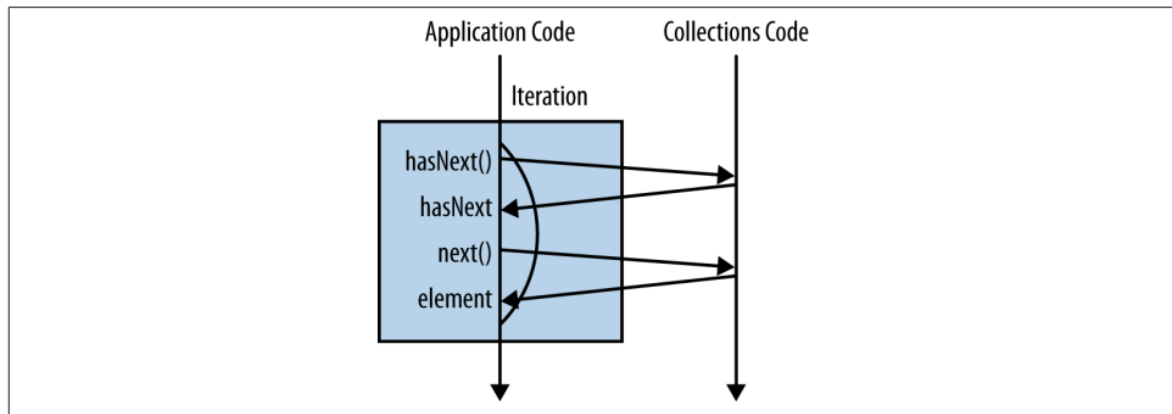
Een stream bestaat uit 3 delen: een (data) bron, intermediate operations (tussentijdse bewerkingen) en een terminal operation (eindbewerking). In ons voorbeeld is de playlist de bron, vervolgens hebben we een intermediate operation *filter* en tenslotte een terminal operation *toList()*.



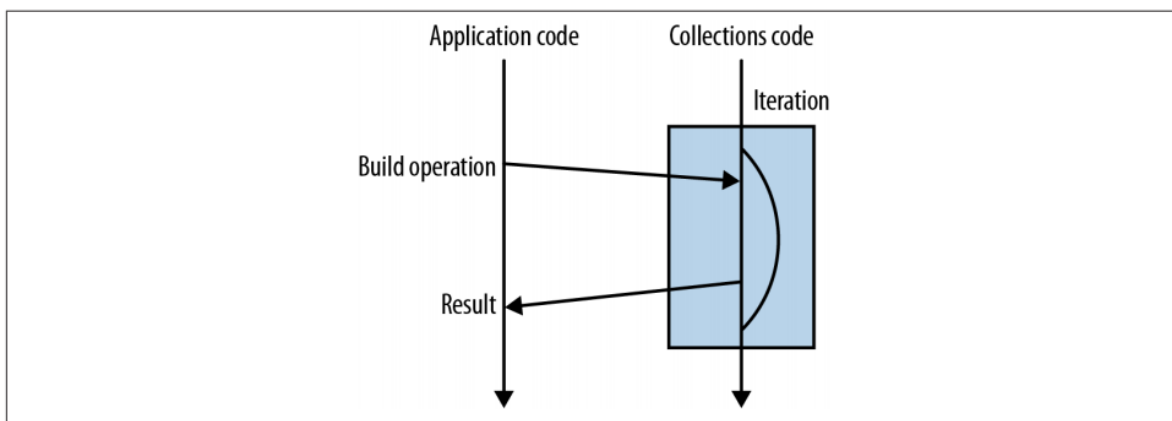
Figuur 7.1: Stream pipeline (logicbig.com)

De intermediate operators verwerken de elementen van de stream één voor één. Alle intermediate operations zijn lui (lazy), ze worden enkel uitgevoerd als de stream wordt afgesloten door een terminal operation.

De internal iterator geniet meestal de voorkeur boven de external iterator. Internal iterations kunnen korter geschreven worden en zijn daardoor ook makkelijker lees- en onderhoudbaar. Toch zijn er situaties, zoals wanneer je 2 verzamelingen gelijktijdig manipuleert, dat je kiest voor een external iterator.



Figuur 7.2: External iteration



Figuur 7.3: Internal iteration

7.3 Intermediate en terminal operations

7.3.1 Terminal operation `.toList()`

```
import java.util.List;
import java.util.stream.Collectors;
import java.util.stream.Stream;

public class DemoCollect {

    public static void main(String[] args) {

        List<String> theBeatles =
            Stream.of("John Lennon", "Paul McCartney", "George
                ↪ Harrison", "Ringo Starr")
                .toList();
        System.out.println(theBeatles);
    }
}
```

```

    }
}

```

Een stream is geen datastructuur of verzameling. Je moet een stream zien als een reeks objecten. Eén van de manieren om zo'n reeks of stream te bouwen is met de static methode *of* van de interface *Stream*.

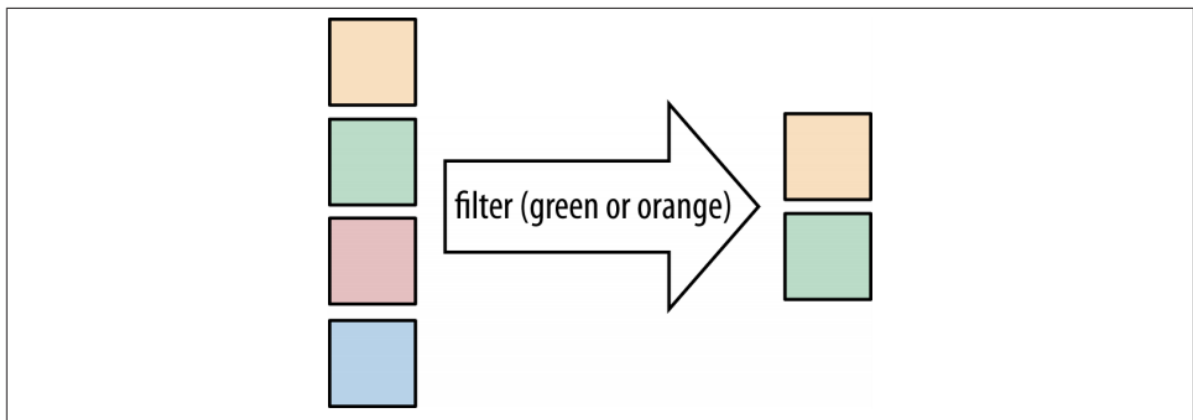
static <T> Stream <T>	of (T... values) Returns a sequential ordered stream whose elements are the specified values.
------------------------------	---

Figuur 7.4: Static methode *of()* in de interface *Stream*

Door gebruik te maken van de operation *.toList()* kunnen we de objecten uit de stream verzamelen in een *List*.

7.3.2 Intermediate operation *.filter()*

Wanneer je bepaalde elementen uit een stream wil selecteren, gebruik je een filter.



Figuur 7.5: Filter operation

Stream <T>	filter (Predicate<? super T> predicate) Returns a stream consisting of the elements of this stream that match the given predicate.
-------------------	--

Figuur 7.6: Methode *filter()* in de interface *Stream*

Aan de hand van een *Predicate* wordt beslist welke elementen geselecteerd worden.

In het onderstaande voorbeeld selecteren we de even getallen uit de verzameling *numbers*.

```

List<Integer> numbers = Arrays.asList(1,2,3,4,5);

```

```
List<Integer> evenNumbers = numbers.stream()
    .filter(n -> n%2 == 0)
    .toList();

assertEquals(Arrays.asList(2,4), evenNumbers);
```

Nog een voorbeeld:

```
List<String> animals = Stream.of("zebra", "dog", "dolphine")
    .filter(a -> a.contains("o"))
    .toList();

assertEquals(Arrays.asList("dog", "dolphine"), animals);
```

Alle String-objecten die een “o” bevatten mogen in de stream aanwezig blijven.

Merk op dat de functie filter() een intermediate operation is. De bewerking heeft als return-type Stream. We bouwen als het ware een pipeline. De filter()-operation is ook lazy en zal pas effectief uitgevoerd worden als er een terminal-operation aanwezig is.

Hier volgt nog een voorbeeld met een verzameling met objecten van een zelfgeschreven klasse Participant.

```
public class Participant {
    private String name;
    private int points;

    public Participant(String name, int points) {
        this.name = name;
        this.points = points;
    }

    public int getPoints() {
        return points;
    }

    public String getName() {
        return name;
    }
}
```

```
Participant john = new Participant("John P.", 15);
Participant sarah = new Participant("Sarah M.", 200);
Participant charles = new Participant("Charles B.", 150);
Participant mary = new Participant("Mary T.", 1);

List<Participant> participants = Arrays.asList(john, sarah, charles, mary);
```

```
List<Participant> over100Points = participants.stream()
    .filter(p -> p.getPoints() > 100)
    .toList();

assertEquals(Arrays.asList(sarah, charles), over100Points);
```

Je kan ook meerdere criteria gebruiken in een filter. Je kan in het Predicate de verschillende criteria samenvoegen via boolean operatoren.

```
List<Participant> selection = participants.stream()
    .filter(p -> p.getPoints() > 100 && p.getName().startsWith("S"))
    .toList();

assertEquals(Collections.singletonList(sarah), selection);
```

Daarnaast kan je ook gebruikmaken van methodes als “or”, “and” en “negate” uit de interface Predicate.

```
Predicate<Participant> over100Points = p -> p.getPoints() > 100;
Predicate<Participant> startingWithS = p -> p.getName().startsWith("S");

List<Participant> selection = participants.stream()
    .filter(over100Points.and(startingWithS))
    .toList();

assertEquals(Collections.singletonList(sarah), selection);
```

7.3.3 Terminal operation .forEach()

De methode .forEach() is een terminal operation en aanvaardt een implementatie van de functionele interface Consumer als parameter. Deze Consumer beschrijft een actie die met ieder element van de verzameling uitgevoerd zal worden.

void	forEach (Consumer<? super T> action)	Performs an action for each element of this stream.
------	--	---

Figuur 7.7: Methode forEach() in de interface Stream

```
public class DemoForEach {

    public static void main(String[] args) {
        Participant john = new Participant("John P.", 15);
        Participant sarah = new Participant("Sarah M.", 200);
        Participant charles = new Participant("Charles B.", 150);
        Participant mary = new Participant("Mary T.", 1);
    }
}
```

```

List<Participant> participants = Arrays.asList(john, sarah,
    ↪ charles, mary);

participants.stream()
    .filter(p -> p.getPoints() >= 200)
    .forEach(System.out::println);

System.out.println("* All participants *");

participants.forEach(System.out::println);
}
}

```

Iedere Collection biedt via de interface Iterable ook een `forEach` methode aan. Met beide `forEach` functies kan je hetzelfde resultaat bereiken.

```

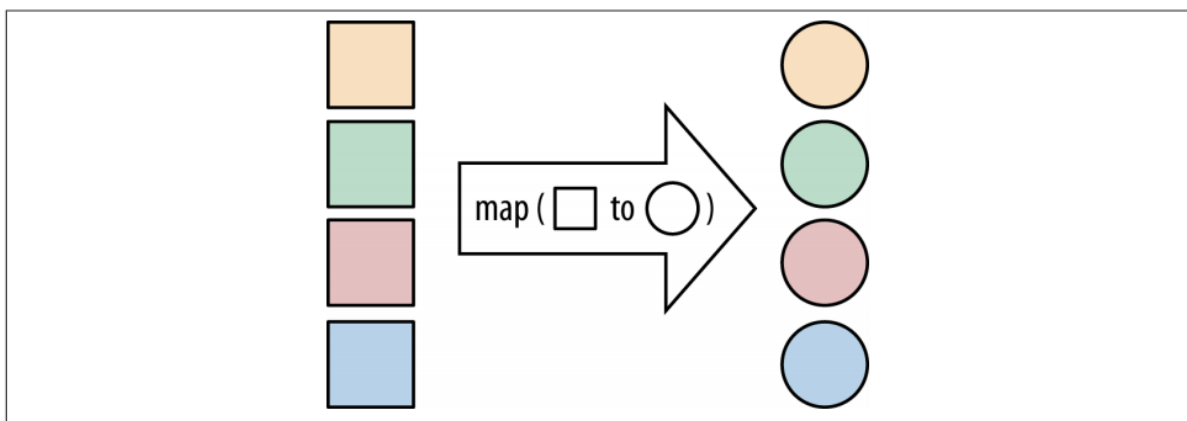
participants.forEach(System.out::println);
participants.stream().forEach(System.out::println);

```

Toch gaat in dit geval onze voorkeur uit naar de eerste optie. Omdat we hier itereren over alle elementen is de stream overbodig.

7.3.4 Intermediate operation `.map()`

Als je een functie hebt om objecten van één datatype te transformeren naar een ander datatype, dan kan je met de bewerking `.map()`, deze functie loslaten op alle objecten van een stream.



Figuur 7.8: Map operation

```

<R> Stream<R> map(Function<? super T,? extends R> mapper)

```

Returns a stream consisting of the results of applying the given function to the elements of this stream.

Figuur 7.9: Methode `map()` in de interface `Stream`

Map is een lazy operator. Zonder terminal operator zal de meegegeven functie niet uitgevoerd worden, en zal je dus ook geen resultaat krijgen. De functie die je meegeeft aan map is een implementatie van de generieke functionele interface `Function<T,R>`.

Hier volgen een twee voorbeelden. In het eerste voorbeeld wijzigt het datatype van de elementen van de stream niet. In het tweede voorbeeld wordt ieder String-object in de stream vervangen door een Integer-waarde.

```
List<String> animals = Stream.of("zebra", "dog", "dolphine")
    .map(String::toUpperCase)
    .toList();

assertEquals(Arrays.asList("ZEBRA", "DOG", "DOLPHINE"), animals);
```

```
List<Integer> lengths = Stream.of("zebra", "dog", "dolphine")
    .map(String::length)
    .toList();

assertEquals(Arrays.asList(5, 3, 8), lengths);
```

7.3.5 Intermediate operation `.sorted()`

Wanneer je de intermediate operation `sorted()` toevoegt aan een stream, worden de elementen in de stream gesorteerd. Zonder parameter zal `sorted()` de natuurlijke sortering gebruiken. Voor objecten van een zelfgeschreven klasse zorg je er dus voor dat de interface `Comparable` geïmplementeerd wordt.

Het is ook mogelijk dat je aan de functie `sorted()` een parameter meegeeft waarmee je een andere volgorde kan afdwingen. In het tweede voorbeeld gebruiken we niet de natuurlijke (alfabetische) volgorde, maar worden de woorden van lang naar kort gesorteerd.

```
List<String> sortedList = Stream.of("zebra", "dog", "dolphine")
    .sorted()
    .toList();

assertEquals(Arrays.asList("dog", "dolphine", "zebra"), sortedList);
```

```
List<String> sortedList = Stream.of("zebra", "dog", "dolphine")
    .sorted((x, y) -> y.length() - x.length())
    .toList();

assertEquals(Arrays.asList("dolphine", "zebra", "dog"), sortedList);
```

7.3.6 Intermediate operation `.distinct()`

De operation `.distinct()` zorgt ervoor dat de elementen in de stream uniek zijn. Elementen die meermaals voorkomen worden verwijderd. Het is de implementatie van de `equals()`

methode (en dus ook hashCode()) van een klasse die beslist of de elementen uniek zijn of niet.

```
List<String> withoutDoubles = Stream.of("zebra", "dog", "zebra",  
    ↪ "dolphine")  
    .distinct()  
    .toList();  
  
assertEquals(3, withoutDoubles.size());
```

7.3.7 Intermediate operation .limit()

De operation .limit() heeft 1 parameter die het maximaal toegelaten elementen in de stream geeft. De stream wordt dus als het ware afgekapt.

```
List<String> animals = Stream.of("zebra", "dog", "elephant", "camel",  
    ↪ "cat", "fish", "dolphine")  
    .limit(4)  
    .toList();  
assertEquals(4, animals.size());
```

Met de methode iterate kunnen we een oneindige stream maken. Dankzij de limit(5) worden slechts de eerste 5 nummers gegenereerd.

```
List<Integer> numbers = Stream.iterate(1, n -> n + n)  
    .limit(5)  
    .toList();  
  
assertEquals(Arrays.asList(1,2,4,8,16), numbers);
```

7.3.8 Intermediate operation peek()

De intermediate operation peek kan gebruikt worden om je pipeline te debuggen. Als je wilt controleren welke elementen er op een gegeven moment in de pipeline zitten, dan voeg je peek toe. Zolang je geen terminal operation hebt toegevoegd zal ook peek geen resultaat laten zien.

```
Stream.of("one", "two", "three", "four")  
    .filter(e -> e.length() > 3)  
    .peek(e -> System.out.println("Filtered value: " + e))  
    .map(String::toUpperCase)  
    .peek(e -> System.out.println("Mapped value: " + e))  
    .toList();
```

Je kan de operation peek() verder ook handig gebruiken om de elementen van je stream aan te passen.

```
Stream<User> userStream = Stream.of(new User("Alice"), new User("Bob"),  
    ↪ new User("Chuck"));  
userStream.peek(u -> u.setName(u.getName().toLowerCase()))  
    .forEach(System.out::println);
```

7.3.9 Terminal operation `.count()`

```
long over100Points = participants.stream().filter(p -> p.getPoints() >  
    ↪ 100).count();  
  
assertEquals(2, over100Points);
```

De terminal operation `count` gebruik je om het aantal elementen in de stream te tellen. Indien je geen gebruik maakt van intermediate operations gebruik je de methode `size()` van je collection om het aantal elementen te achterhalen.

```
System.out.println("Number of participants: " + participants.size());
```

7.4 IntStream, LongStream en DoubleStream

Er zijn ook een aantal afgeleide interfaces van de interface `Stream`. Deze bieden extra functionaliteit aan. Zo hebben we bijvoorbeeld de interface `IntStream`, die speciaal ontworpen is voor streams met gehele getallen. Deze stream bevat elementen met datatype *int*. Naast `IntStream` hebben we ook de interfaces `DoubleStream` en `LongStream`. Al deze interfaces bevatten de methoden `sum()`, `min()`, `max()` en `average()`.

7.4.1 `sum()`

Met de operation `sum` kan je eenvoudig de som berekenen van alle elementen in je stream.

```
long totalPoints =  
    ↪ participants.stream().mapToInt(Participant::getPoints).sum();  
  
assertEquals(366, totalPoints);
```

7.4.2 `range()` en `rangeClosed()`

De static methoden `range()` en `rangeClosed()` zijn beschikbaar in de interfaces `java.util.stream.IntStream` en `java.util.stream.LongStream`. Je kan ze gebruiken om een stream te creëren met gehele getallen vanaf een initiële waarde tot een stop waarde.

static IntStream	range (int startInclusive, int endExclusive)	Returns a sequential ordered IntStream from startInclusive (inclusive) to endExclusive (exclusive) by an incremental step of 1.
static IntStream	rangeClosed (int startInclusive, int endInclusive)	Returns a sequential ordered IntStream from startInclusive (inclusive) to endInclusive (inclusive) by an incremental step of 1.

Figuur 7.10: Aanmaken van IntStream met range() of rangeClosed()

```
long count = IntStream.rangeClosed(10, 20).count();
assertEquals(11, count);
```

7.4.3 min(), max() en average()

OptionalInt	max()	Returns an OptionalInt describing the maximum element of this stream, or an empty optional if this stream is empty.
OptionalInt	min()	Returns an OptionalInt describing the minimum element of this stream, or an empty optional if this stream is empty.

Figuur 7.11: min() en max() uit de interface IntStream

OptionalDouble	average()	Returns an OptionalDouble describing the arithmetic mean of elements of this stream, or an empty optional if this stream is empty.
-----------------------	------------------	--

Figuur 7.12: average() uit de interface IntStream

Merk op dat de methoden max() en min() een object van de klasse OptionalInt als return-type hebben. De methode average() heeft OptionalDouble als return-type.

Oefening 7.2. Zoek de klasse OptionalInt en OptionalDouble op in de Java documentatie.

In de documentatie vind je terug dat dit een container-object is dat een int- of double-waarde kan bevatten. Indien we namelijk een lege stream hebben, dan bestaat er immers geen minimum, maximum of gemiddelde. In plaats van bijvoorbeeld een exception op te gooien wanneer we max() oproepen voor een lege IntStream, is er gekozen om steeds een OptionalInt als resultaat te geven. Bij een lege IntStream bevat het OptionalInt-object geen waarde en *isPresent()* geeft false als resultaat. Indien er wel een maximum berekend kan worden geeft *isPresent()* true. De berekende waarde kan je bekomen via de methode *getAsInt()*.

```
Random random = new Random();
List<Integer> randomNumbers = random.ints(15, 0, 100).boxed().toList();
int max = randomNumbers.stream().mapToInt(x -> x).max().getAsInt();
int min = randomNumbers.stream().mapToInt(x -> x).min().getAsInt();
double average = randomNumbers.stream().mapToInt(x ->
    ↪ x).average().getAsDouble();
```

```
assertTrue(min <= average);
assertTrue(max >= average);

IntSummaryStatistics intSummaryStatistics = random.ints(15, 0,
    ↪ 100).summaryStatistics();
assertTrue(intSummaryStatistics.getMax() >= intSummaryStatistics.getMin());
```

7.5 Method reference

Method references zijn een speciaal type lambda expressies. Ze worden gebruikt om lambda expressies te vereenvoudigen door te verwijzen naar een bestaande methode, zonder dat je expliciet de parameters moet invullen.

Er zijn 4 soorten methodes in Java:

- static methods
- constructoren
- instance methods

7.5.1 Static methods

Syntax:

`ClassName::staticMethod`

```
IntBinaryOperator min = (x, y) -> Math.min(x, y);
IntBinaryOperator max = Math::max;

System.out.println(min.applyAsInt(-3, 17));
System.out.println(max.applyAsInt(-3, 17));
```

7.5.2 Instance method (unbounded)

Syntax:

`ClassName::instanceMethod`

```
Function<String, String> toUpperCase = s -> s.toUpperCase();
Function<String, String> toLowerCase = String::toLowerCase;
System.out.println(toUpperCase.apply("abcdef"));
System.out.println(toLowerCase.apply("ABCDEF"));
```

7.5.3 Instance method (bounded)

Syntax:

`instance::instanceMethod`

De klasse `Random` voorziet 2 versies van de methode `nextInt()`: een eerste versie zonder parameter en een tweede versie met één parameter, de bovengrens. Dit principe noemen we methode overloading. Aan de hand van de functionele interface die je implementeert, kan je nu de gewenste methode als een lambda functie schrijven. Hier illustreren we hoe je deze lambda functie ook nog eens verkort kan schrijven door gebruik te maken van method reference.

```
Random random = new Random();
IntSupplier randomInt = random::nextInt;
IntUnaryOperator randomIntWithBound = random::nextInt;
System.out.println(randomInt.getAsInt());
System.out.println(randomIntWithBound.applyAsInt(12));
```

7.5.4 Constructor

Je kan ook voor een constructor een method reference maken. De syntax van de method reference voor een constructor ziet er als volgt uit.

`ClassName::new`

```
System.out.println("Constructor");
Supplier<Random> randomCreator = Random::new;
Random random = randomCreator.get();
```

```
public class Person {
    private String name;

    public Person() {
        this.name = "Unknown";
    }

    public Person(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }
}
```

```
public class ConstructorMethodReferenceExample {
    public static void main(String[] args) {
        // Using a constructor method reference to create a new Person
        //    ↪ object
        Supplier<Person> personSupplier = Person::new;
        Person person = personSupplier.get();
        System.out.println("Name: " + person.getName()); // Outputs: Name:
```

→ Unknown

```
// Using a constructor method reference to create a new Person
```

→ object with a name

```
Function<String, Person> personFunction = Person::new;
```

```
Person person2 = personFunction.apply("Alice");
```

```
System.out.println("Name: " + person2.getName()); // Outputs: Name:
```

→ Alice

```
}
```

```
}
```

Oefening 7.3. In de klasse `be.pxl.ja.oefening1.StudentList` vind je een static methode `createStudents()`. Voorzie nu een hoofdprogramma waarin je start met deze lijst en de volgende vragen beantwoordt met behulp van een stream.

- Welke studenten zijn er vandaag jarig. Toon hun naam. Je verandert best een geboortedata van de studenten om je stream uit te testen.
- Toon alle gegevens van de student met de naam Carol.
- Hoeveel studenten studeerden af in 2017?
- Wat is de hoogste score ooit behaald? Wie behaalde deze hoogste score ooit?
- Wie is de oudste persoon in de lijst?
- Geef de namen van alle studenten gescheiden door komma (,) in één String.
- Wie is de jongste student van afstudeerjaar 2018?
- Maak een map met per afstudeerjaar de gemiddelde score.
- Sorteert de studenten eerst op afstudeerjaar (recentste jaar eerst) en per afstudeerjaar volgens hun score (van hoog naar laag). Toon naam, afstudeerjaar en score.

Hoofdstuk 8

3-lagen architectuur: introductie van Spring Data JPA

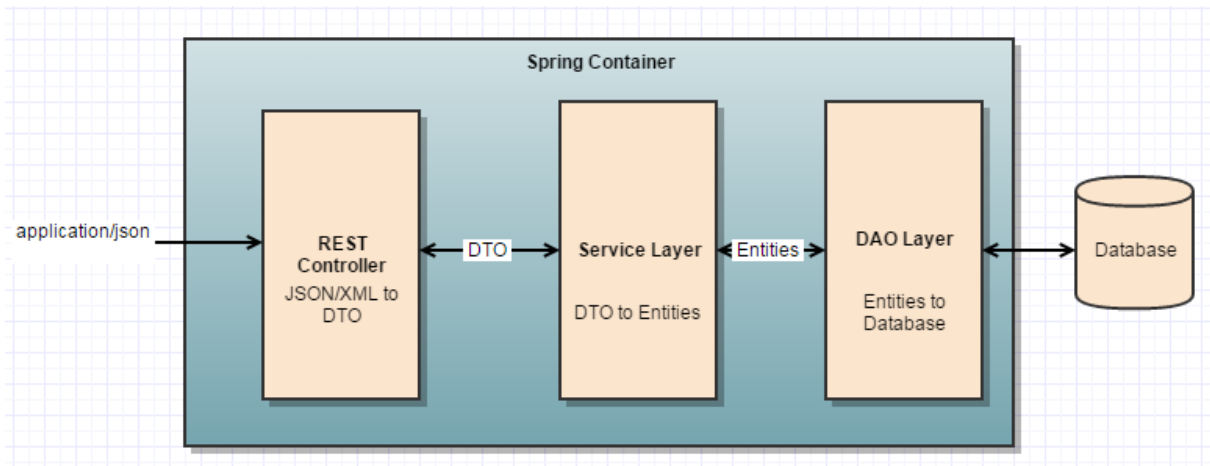
De Spring Boot applicaties die we tot nu toe hebben gebouwd bestaan uit 2 lagen. De RestController uit de router-laag gebruikt de functionaliteit of business-logica uit de service-laag. Nu voegen we een derde laag toe aan de Spring Boot applicaties: de persistence-laag. Deze laag is verantwoordelijk om gegevens op te halen uit en weg te schrijven naar de databank.

Onze RESTful webapplicaties bestaan vanaf nu dus uit 3 lagen: API-laag (of router-laag), service- of business-logica-laag en persistence-laag.

- **Router- of API-laag:** Het afhandelen en verwerken van HTTP-verzoeken, het vertalen van JSON-parameters naar objecten, authenticatie van gebruikers en het beschermen van gegevens tegen kwaadwillige gebruikers,
- **Servicelaag:** Deze laag bevat de business-logica, de kernfunctionaliteit van je applicatie. Alle berekeningen, beslissingen, evaluaties, gegevensverwerking, ... worden door deze laag afgehandeld.
- **Data- of persistence-laag:** Deze laag is verantwoordelijk voor de interactie met de database. Deze laag slaat de gegevens op en haalt deze op uit de database.

Elke laag heeft zijn eigen annotatie voor Spring Beans:

- @Component: generieke annotatie voor alle componenten beheerd door Spring
- @RestController: voor de componenten in de API-laag
- @Service: voor de componenten in de service-laag
- @Repository: voor de componenten in de persistence-laag



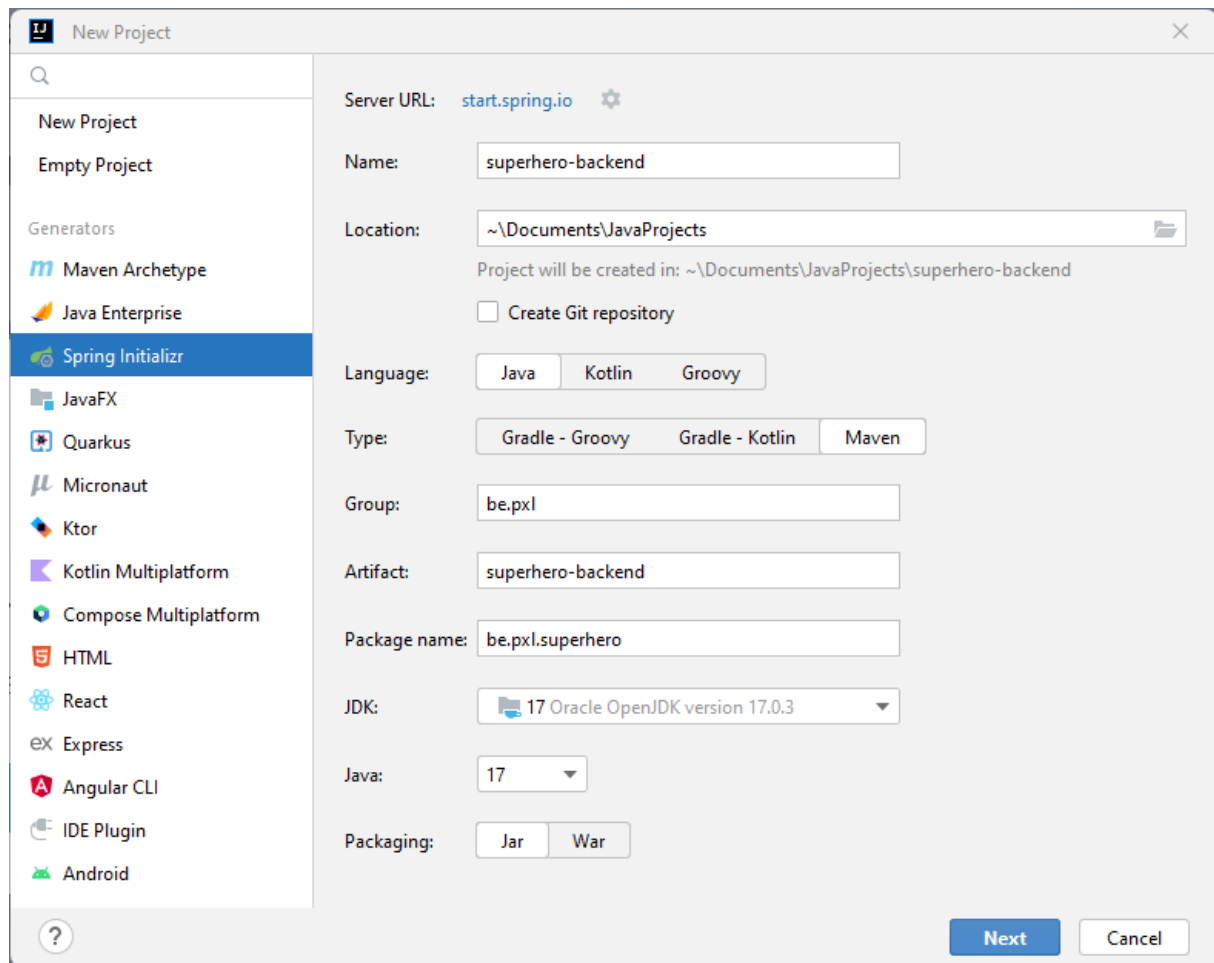
Bovenstaande afbeelding toont alle lagen die we ontwikkelen in een RESTful web applicatie. De RestController in de API-laag biedt REST-endpoints aan. De API-laag communiceert via DTO's of Data Transfer Objects met de servicelaag. De servicelaag implementeert alle business-logica. DTO's worden omgevormd naar entiteiten. Entiteiten zijn objecten uit het domein die we persisteren in de databank. Deze entiteiten worden doorgegeven aan de persistence-laag. De persistence-laag is verantwoordelijk voor het opslaan van de entiteiten in de database.

De Spring-container staat centraal in het Spring Framework. De container is verantwoordelijk voor het maken en beheren van objecten voor klassen die zijn geannoteerd met `@Service`, `@Repository`, ... De container zal deze objecten (Spring beans) beschikbaar stellen wanneer ze nodig zijn.

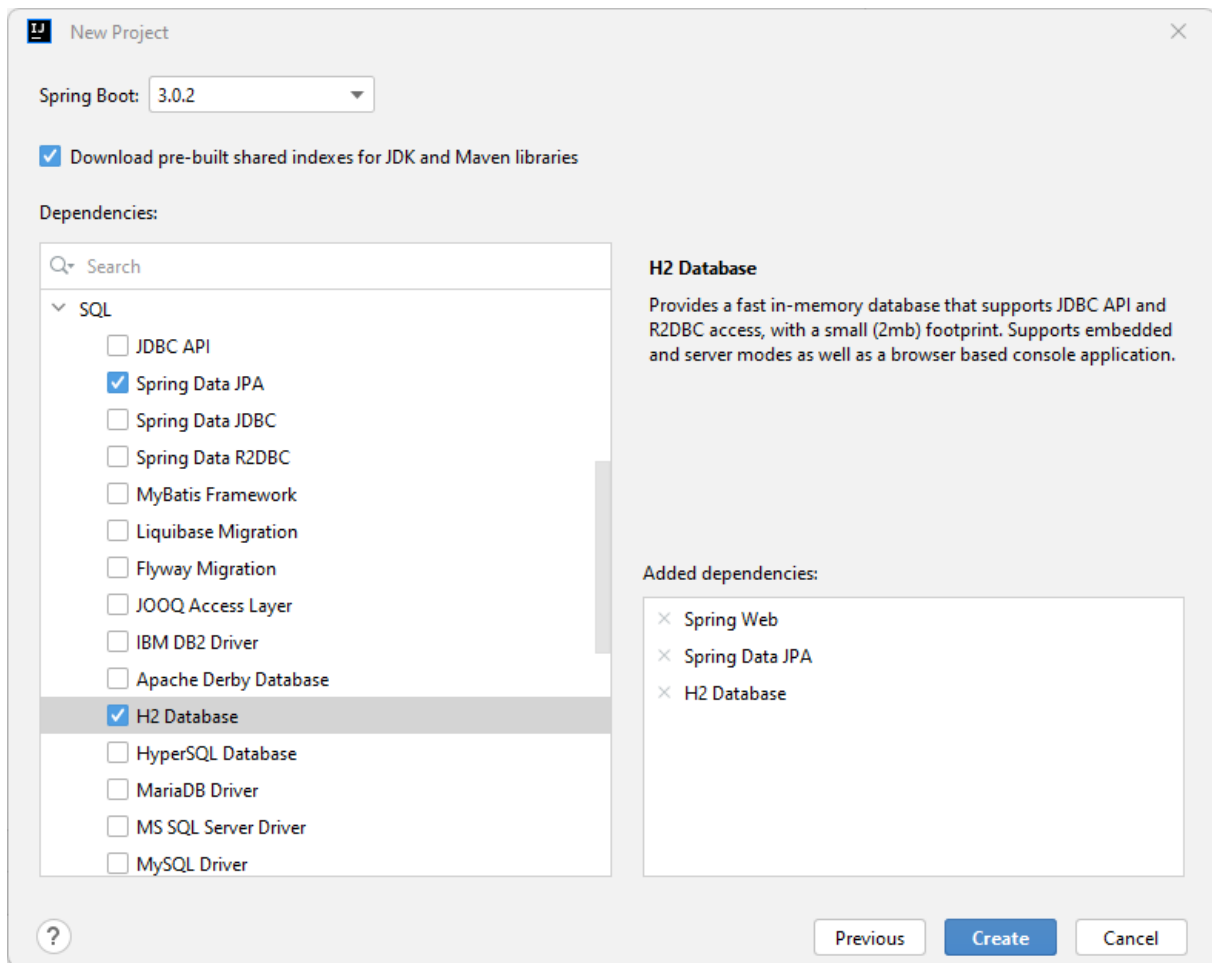
8.1 Een voorbeeld toepassing

Dit hoofdstuk begeleidt je bij het ontwikkelen van een Spring Boot-toepassing waarbij we gegevens gaan wegschrijven naar en ophalen uit een databank. Je implementeert een RESTful web applicatie om de gegevens van de 'Superhero Company' te beheren. We zullen REST-endpoints implementeren voor het creëren, aanpassen en verwijderen van superhelden. Later voeg je functionaliteit toe om missies voor de superhelden te creëren, te updaten en te verwijderen.

We gebruiken Spring Initializr om deze Spring Boot-toepassing op te zetten.



Vul de metadata voor het project in. Gebruik Maven als build tool.



Oefening 8.1. Maak het Spring Boot project met de naam ‘superhero-backend’ aan. Naast Spring Web en Spring Validation voeg je ook Spring Data JPA toe aan het project. Tenslotte voeg je nog de H2 Database dependency toe. Dit is een in-memory database en je hebt amper configuratie nodig om deze database te gebruiken. Dit is handig voor snelle prototyping, maar al je gegevens gaan verloren zodra je de toepassing opnieuw start.

8.2 Gegevens opslaan en ophalen

8.2.1 Entiteitsklasse Superhero

Allereerst hebben we een klasse nodig om de objecten te representeren die worden opgeslagen en opgehaald uit de database. Elk object van deze klasse zal overeenkomen met één rij van een tabel in de relationele database. We implementeren de klasse Superheld. Om Superheld-objecten in de database op te slaan en deze later uit de database op te halen, annoteren we deze klasse als een entiteit. Een entiteitsklasse vertegenwoordigt een entiteit of object in het domeinmodel en wordt gemapt naar een tabel in de database. Elke instantie van deze klasse komt overeen met een rij in deze tabel, waardoor de eigenschappen van het object worden opgeslagen als kolommen in de database.

Hier is de entiteitsklasse Superhero:


```

package be.px1.superhero.domain;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

@Entity
@Table(name="superheroes")
public class Superhero {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String firstName;
    private String lastName;
    private String superheroName;

    public Superhero() {
        // JPA only
    }

    public Superhero(String firstName, String lastName, String
        ↪ superheroName) {
        this.firstName = firstName;
        this.lastName = lastName;
        this.superheroName = superheroName;
    }

    public Long getId() {
        return id;
    }

    public String getFirstName() {
        return firstName;
    }

    public void setFirstName(String firstName) {
        this.firstName = firstName;
    }

    public String getLastName() {
        return lastName;
    }
}

```

```

    public void setLastName(String lastName) {
        this.lastName = lastName;
    }

    public String getSuperheroName() {
        return superheroName;
    }

    public void setSuperheroName(String superheroName) {
        this.superheroName = superheroName;
    }

    @Override
    public String toString() {
        return superheroName;
    }
}

```

De annotatie **@Entity** geeft aan dat de klasse Superhero een entiteitsklasse is. Spring is in staat om automatisch de databasetabel ('superheroes') te genereren met de nodige kolommen (velden) in de H2-database. De primaire sleutel van de tabel is gemarkeerd met de annotatie **@Id**. Met de annotatie **@GeneratedValue(strategy = GenerationType.IDENTITY)**, hoeven we de primaire sleutels niet zelf toe te wijzen aan de objecten. De database is verantwoordelijk voor het genereren en toewijzen van de primaire sleutels.

Oefening 8.2. Voeg de entiteitsklasse Superhero toe aan het project. Je voegt deze klasse toe in het package *be.pxl.superhero.domain*.

8.2.2 Repository

Om queries in de database uit te voeren, hebben we een extra interface nodig die een **repository** wordt genoemd. Spring is in staat om automatisch databasequeries te genereren. Wanneer je de interface JpaRepository uitbreidt, zijn eenvoudige queries (zoals save, findById,...) al beschikbaar zonder dat je een regel code hoeft te schrijven. De generieke interface JpaRepository hoeft alleen maar te weten welke gegevens het moet opslaan en ophalen. Daarom moet het de naam van de entiteitsklasse en het gegevenstype van de primaire sleutel van de entiteitsklasse weten.

```

package be.pxl.superhero.repository;

import be.pxl.superhero.domain.Superhero;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface SuperheroRepository extends JpaRepository<Superhero,
    ↳ Long> {

```

```
}
```

Oefening 8.3. Bestudeer de documentatie van de generieke interface `JpaRepository` en zijn subinterfaces. Voeg de repository `SuperheroRepository` toe in het project. Maak hiervoor het package `be.px1.superhero.repository` aan.

8.2.3 Service-layer

Zoals je al weet is de service-laag verantwoordelijk voor de implementatie van de businesslogica. De klassen in de service-laag zullen gebruik maken van repositories en andere service-klassen.

Het is een goed idee om voor elke klasse in de servicelaag een interface te voorzien. De serviceklassen mogen nooit entiteitsobjecten retourneren. We hebben DTO's (Data Transfer Objects) nodig om gegevens van de servicelaag naar de API-laag te sturen.

```
package be.px1.superhero.service;

import be.px1.superhero.api.SuperheroDTO;
import be.px1.superhero.api.SuperheroRequest;

import java.util.List;

public interface SuperheroService {

    List<SuperheroDTO> findAllSuperheroes();

    SuperheroDTO findSuperheroById(Long superheroId);

    Long createSuperhero(SuperheroRequest superheroRequest);

    SuperheroDTO updateSuperhero(Long superheroId, SuperheroRequest
        ↪ superheroRequest);

    boolean deleteSuperhero(Long superheroId);
}
```

```
package be.px1.superhero.api;

import be.px1.superhero.domain.Superhero;

public class SuperheroDTO {

    private final Long id;
    private final String firstName;
    private final String lastName;
    private final String superheroName;
```

```

public SuperheroDTO(Superhero superhero) {
    this.id = superhero.getId();
    this.firstName = superhero.getFirstName();
    this.lastName = superhero.getLastName();
    this.superheroName = superhero.getSuperheroName();
}

public Long getId() {
    return id;
}

public String getFirstName() {
    return firstName;
}

public String getLastName() {
    return lastName;
}

public String getSuperheroName() {
    return superheroName;
}
}

```

```

package be.px1.superhero.api;

public class SuperheroRequest {

    private String firstName;
    private String lastName;
    private String superheroName;

    public SuperheroRequest(String firstName, String lastName, String
        ↪ superheroName) {
        this.firstName = firstName;
        this.lastName = lastName;
        this.superheroName = superheroName;
    }

    public String getFirstName() {
        return firstName;
    }

    public void setFirstName(String firstName) {
        this.firstName = firstName;
    }
}

```

```

    public String getLastName() {
        return lastName;
    }

    public void setLastName(String lastName) {
        this.lastName = lastName;
    }

    public String getSuperheroName() {
        return superheroName;
    }

    public void setSuperheroName(String superheroName) {
        this.superheroName = superheroName;
    }
}

```

De klasse `SuperheroServiceImpl`, geannoteerd met `@Service`, biedt de implementatie voor de interface `SuperheroService`. Alle CRUD-operaties (create-read-update-delete) voor Superhero-objecten worden hier aangeboden. In de klasse `SuperheroServiceImpl` wordt alle bedrijfslogica geïmplementeerd. Als we bijvoorbeeld moeten controleren of de superheldennaam van een superheld uniek is, is deze klasse de plek om deze controle uit te voeren.

De `SuperheroRepository` wordt door Spring Boot ter beschikking gesteld in de `SuperheroServiceImpl`. Daarom kan de serviceklasse gegevens opslaan en ophalen uit de database met behulp van deze repository.

```

package be.px1.superhero.service.impl;

import be.px1.superhero.api.SuperheroDTO;
import be.px1.superhero.api.SuperheroRequest;
import be.px1.superhero.domain.Superhero;
import be.px1.superhero.exception.ResourceNotFoundException;
import be.px1.superhero.repository.SuperheroRepository;
import be.px1.superhero.service.SuperheroService;
import org.springframework.stereotype.Service;

import java.util.List;
import java.util.stream.Collectors;

@Service
public class SuperheroServiceImpl implements SuperheroService {

    private final SuperheroRepository superheroRepository;

    @Autowired

```

```

public SuperheroServiceImpl(SuperheroRepository superheroRepository) {
    this.superheroRepository = superheroRepository;
}

public List<SuperheroDTO> findAllSuperheroes() {
    return superheroRepository.findAll()
        .stream().map(SuperheroDTO::new)
        .toList();
}

public SuperheroDTO findSuperheroById(Long superheroId) {
    return superheroRepository.findById(superheroId)
        .map(SuperheroDTO::new)
        .orElseThrow(() -> new
            ↪ ResourceNotFoundException("Superhero", "ID",
            ↪ superheroId));
}

public Long createSuperhero(SuperheroRequest superheroRequest) {
    Superhero superhero = new Superhero();
    superhero.setFirstName(superheroRequest.getFirstName());
    superhero.setLastName(superheroRequest.getLastName());
    superhero.setSuperheroName(superheroRequest.getSuperheroName());
    Superhero newSuperhero = superheroRepository.save(superhero);
    return newSuperhero.getId();
}

public SuperheroDTO updateSuperhero(Long superheroId, SuperheroRequest
    ↪ superheroRequest) {
    return superheroRepository.findById(superheroId).map(superhero -> {
        superhero.setFirstName(superheroRequest.getFirstName());
        superhero.setLastName(superheroRequest.getLastName());
        superhero.setSuperheroName(superheroRequest.getSuperheroName());
        return new SuperheroDTO(superheroRepository.save(superhero));
    }).orElseThrow(() -> new ResourceNotFoundException("Superhero",
        ↪ "id", superheroId));
}

public boolean deleteSuperhero(Long superheroId) {
    return superheroRepository.findById(superheroId)
        .map(superhero -> {
            superheroRepository.delete(superhero);
            return true;
        }).orElseThrow(() -> new
        ↪ ResourceNotFoundException("Superhero", "id",
        ↪ superheroId));
}

```

```
}
```

```
package be.pxl.superhero.exception;

@ResponseStatus(HttpStatus.NOT_FOUND)
public class ResourceNotFoundException extends RuntimeException {
    public ResourceNotFoundException(String resource, String field, String
        ↪ value) {
        super("Not found: " + resource + " with " + field + "=" + value);
    }

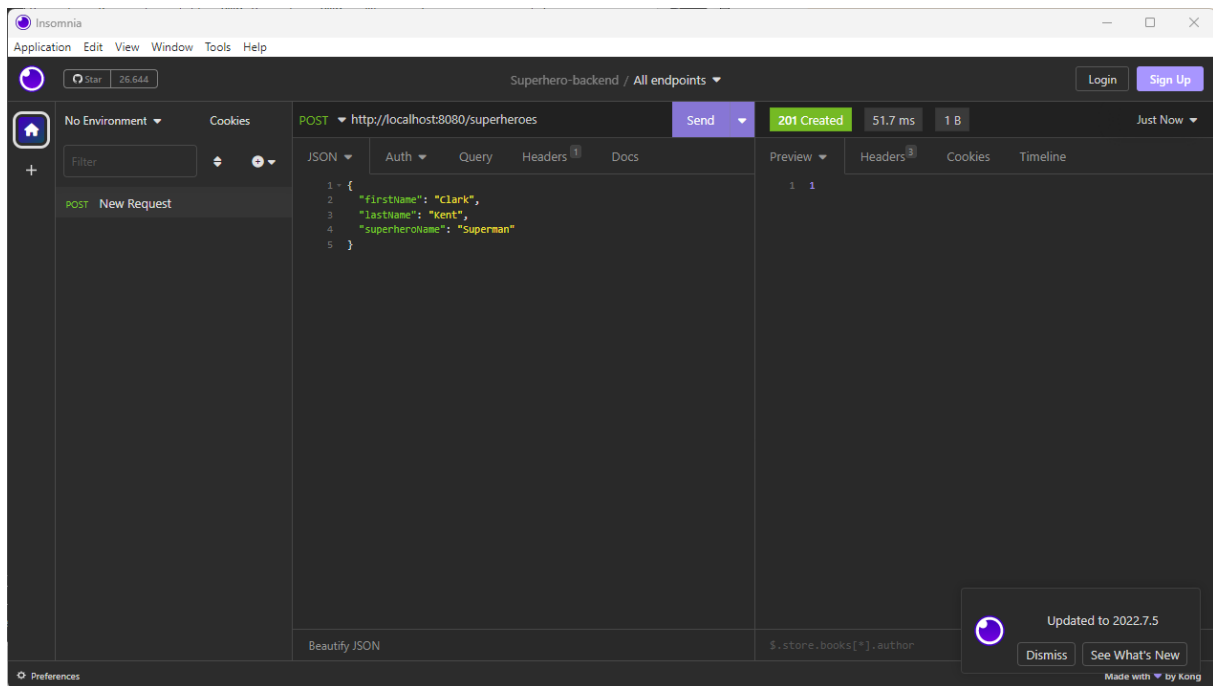
    public ResourceNotFoundException(String resource, String field, long
        ↪ value) {
        this(resource, field, Long.toString(value));
    }
}
```

Oefening 8.4. Maak het pakket *be.pxl.superhero.api* aan en voeg het request-object *SuperheroRequest* en de DTO *SuperheroDTO* toe. Deze klassen worden gebruikt voor communicatie met de API-laag. Voeg de interface *SuperheroService* en de implementatie *SuperheroServiceImpl* toe aan je project. De interface bevindt zich in het pakket *be.pxl.superhero.service*. De implementatie staat in het package *be.pxl.superhero.service.impl*. Voeg tot slot de uitzonderingsklasse *ResourceNotFoundException* toe aan het package *be.pxl.superhero.exception*.

8.2.4 RESTController

Nu kunnen we de REST-endpoints toevoegen voor het creëren, bijwerken, verwijderen en ophalen van superhelden.

Voor het creëren van een nieuwe superheld gebruiken we een POST-verzoek met een requestbody in JSON-formaat. Deze requestbody bevat alle informatie over de nieuwe superheld. Je hebt de klasse *SuperheroRequest* al aan het project toegevoegd. Deze klasse wordt gebruikt om de gegevens van de requestbody naar een object te mappen.



```
package be.px1.superhero.api;

import be.px1.superhero.service.SuperheroService;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.List;

@RestController
@RequestMapping("/superheroes")
public class SuperheroController {

    private final SuperheroService superheroService;

    public SuperheroController(SuperheroService superheroService) {
        this.superheroService = superheroService;
    }

    @GetMapping
    public List<SuperheroDTO> getSuperheroes() {
        return superheroService.findAllSuperheroes();
    }
}
```



```

    }

    @GetMapping("/{superheroId}")
    public SuperheroDTO getSuperheroById(@PathVariable Long superheroId) {
        return superheroService.findSuperheroById(superheroId);
    }

    @PostMapping
    public ResponseEntity<Long> createSuperhero(@RequestBody
        ↳ SuperheroRequest superheroRequest) {
        return new
            ↳ ResponseEntity<>(superheroService.createSuperhero(superheroRequest),
            ↳ HttpStatus.CREATED);
    }

    @PutMapping("/{superheroId}")
    public SuperheroDTO updateSuperhero(@PathVariable Long superheroId,
        ↳ @RequestBody SuperheroRequest superheroRequest) {
        return superheroService.updateSuperhero(superheroId,
            ↳ superheroRequest);
    }

    @DeleteMapping("/{superheroId}")
    public ResponseEntity<Void> deleteSuperhero(@PathVariable Long
        ↳ superheroId) {
        boolean deleted = superheroService.deleteSuperhero(superheroId);
        return deleted? new ResponseEntity<>(HttpStatus.OK) : new
            ↳ ResponseEntity<>(HttpStatus.BAD_REQUEST);
    }
}

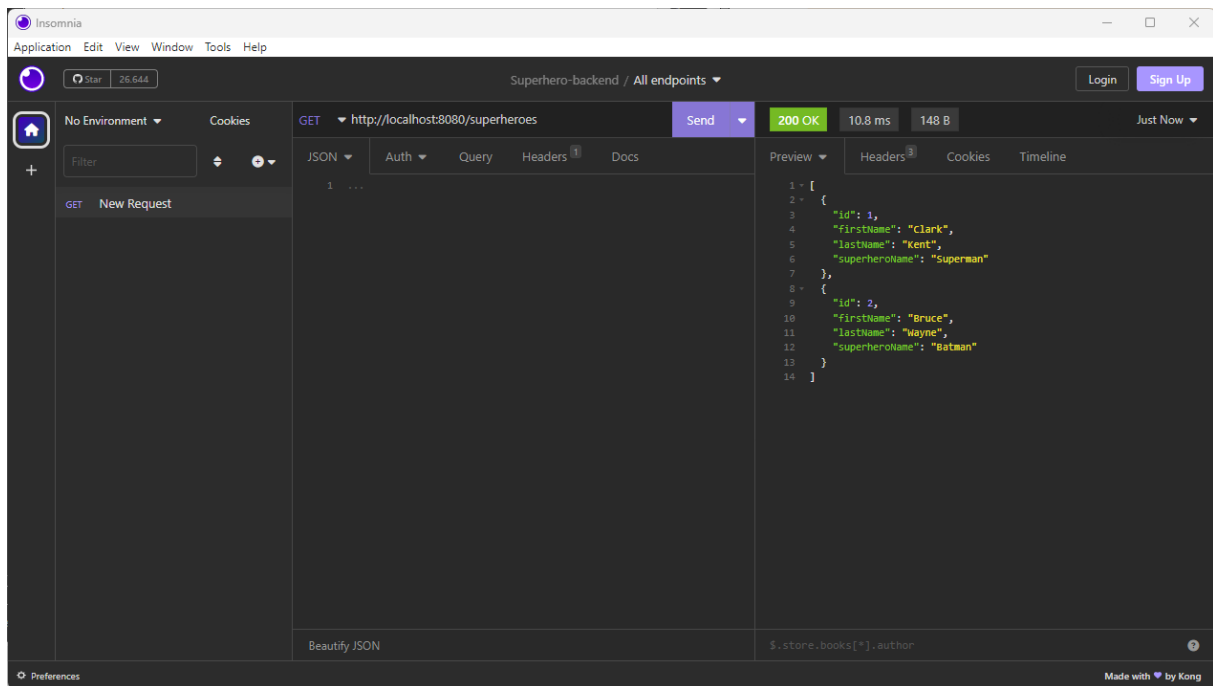
```

Oefening 8.5. Voeg @RestController SuperheroController toe aan je Spring Boot-toepassing. Herstart het project en maak een nieuwe superheld aan met Insomnia of Postman. Vervolgens roep je het REST-eindpunt aan om alle superhelden op te halen of om de superheld op te halen op basis van ID. Hier is het JSON-formaat om een superheld te creëren:

```

{
    "firstName": "Clark",
    "lastName": "Kent",
    "superheroName": "Superman"
}

```



8.3 URL context path

Wanneer je een voorvoegsel (prefix), bijv. `/api`, aan alle URL's van de applicatie wilt toevoegen, kun je het volgende sleutel-waarde-paar toevoegen in het bestand `application.properties`:

```
server.servlet.context-path=/api
```

Oefening 8.6. Verander het context-pad van je Spring Boot-applicatie. Het voorvoegsel `/api` moet worden gebruikt voor de applicatie. Herstart de applicatie en test de endpoints.

8.4 API documentation

```

<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
  <version>2.2.0</version>
</dependency>

```

De Swagger-documentatie in XML-formaat die te vinden is op de URL <http://localhost:8080/api/v3/api-docs> is niet gebruiksvriendelijk. Als je echter de URL <http://localhost:8080/api/swagger-ui.html> opent in je browser, kun je een gebruiksvriendelijke Swagger-pagina zien waar je zelfs je API kunt testen.

Oefening 8.7. Voeg swagger documentatie toe aan het project. Zoek eens op hoe je bijkomende Swagger documentatie in je RESTController kan toevoegen.

H2 database

De H2 in-memory database verdwijnt wanneer je de applicatie afsluit en alle data gaat verloren. Je kunt bestanden gebruiken om de data permanent op te slaan. Als je de data van je in-memory database wilt bekijken, kun je de volgende eigenschap toevoegen aan het bestand `application.properties`:

```
spring.h2.console.enabled=true
```

Als je de Spring Boot applicatie opstart, krijg je de unieke naam van de database.

```
H2 console available at '/h2-console'. Database available at  
↪ 'jdbc:h2:mem:f5f92e54-3aff-4986-9d00-a0028b0eb6ed'
```

Als je de URL <http://localhost:8080/api/h2-console> in een browser open en de unieke naam van de databank invult en username “sa” en blanco paswoord ingeeft, dan krijg je toegang tot de tabellen en gegevens van de in-memory databank.

The screenshot shows the H2 console web interface. At the top, there is a language dropdown menu set to 'English' and navigation links for 'Preferences', 'Tools', and 'Help'. Below this is a 'Login' section with a blue header. The 'Saved Settings' section shows a dropdown menu with 'Generic H2 (Embedded)' selected. Below it, the 'Setting Name' is also 'Generic H2 (Embedded)', with 'Save' and 'Remove' buttons. The 'Driver Class' is set to 'org.h2.Driver'. The 'JDBC URL' is 'jdbc:h2:mem:f5f92e54-3aff-4986-9d00-a0028b0eb6ed'. The 'User Name' is 'sa' and the 'Password' field is empty. At the bottom, there are 'Connect' and 'Test Connection' buttons.

Gebruik H2 Database Niet voor Productiesystemen

De H2 database is ontworpen voor prototyping, testen en hobbyprojecten en wordt niet aanbevolen voor gebruik in productieomgevingen. Hoewel H2 lichtgewicht en eenvoudig te configureren is, mist het belangrijke eigenschappen zoals robuuste beveiliging, schaalbaarheid en geavanceerd transactioneel beheer die essentieel zijn voor productiegebruik. Bovendien kan het gebrek aan brede ondersteuning voor grote datasets en complexe queries leiden tot prestatieproblemen in veeleisende omgevingen. Gebruik H2 uitsluitend voor ontwikkel- en testdoeleinden; kies voor een professionele database-oplossing zoals PostgreSQL, MySQL of SQL Server in productie.

8.5 Frontend

Een frontend voor the superhero applicatie, geschreven in Angular, kan je vinden op github. (Credits to: <https://github.com/shoul10>).

Source code

De frontend code is beschikbaar op: <https://github.com/custersnele/superhero-frontend.git>

Zorg ervoor dat de nodige configuratie voorziet zodat de frontend de backend kan bereiken. Denk eraan dat je ook CORS (Cross-Origin Resource Sharing) moet toestaan in je Spring Boot applicatie.

Oefening 8.8. Voeg CORS-ondersteuning toe aan je project. Voeg de configuratie toe in het package *be.pxl.superhero.config*. Herstart de Spring Boot applicatie. Download de frontend code van github en open de code in een ontwikkelomgeving zoals WebStorm. Start de frontend toepassing op (ng serve) en creëer, update en verwijder je superhelden.

Hoofdstuk 9

Spring Data JPA

Learning goals

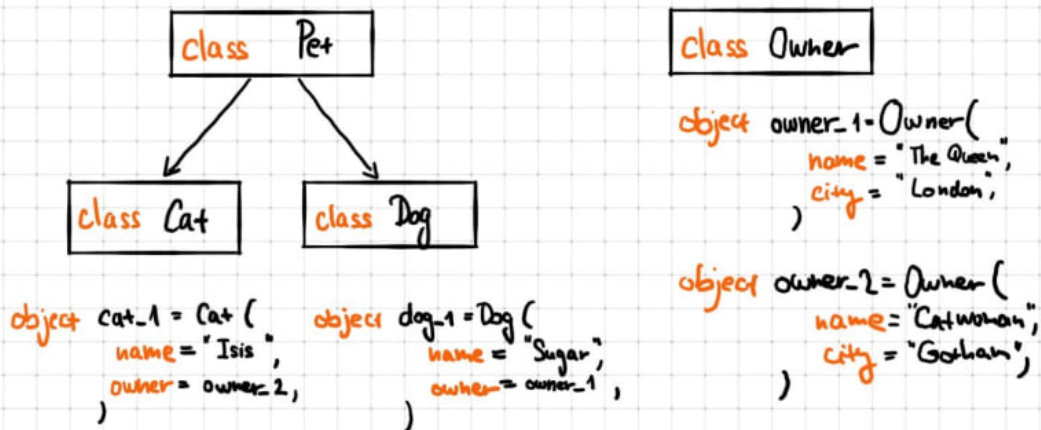
De junior-collega

1. kan het concept ORM beschrijven.
2. kan uitleggen wat JPA is.
3. kan verschillende JPA-implementaties benoemen.
4. kan beschrijven wat een *persistence unit* is.
5. kan de fundamentele interfaces van JPA beschrijven.
6. kan uitleggen wat de *persistence context* is.
7. kan entiteitsklassen implementeren.
8. kan de levenscyclus van entiteitsobjecten beschrijven.
9. kan verschillende soorten relaties tussen entiteitsklassen implementeren.
10. kan CRUD-operaties implementeren.
11. kan query's implementeren.

9.1 Wat is ORM?

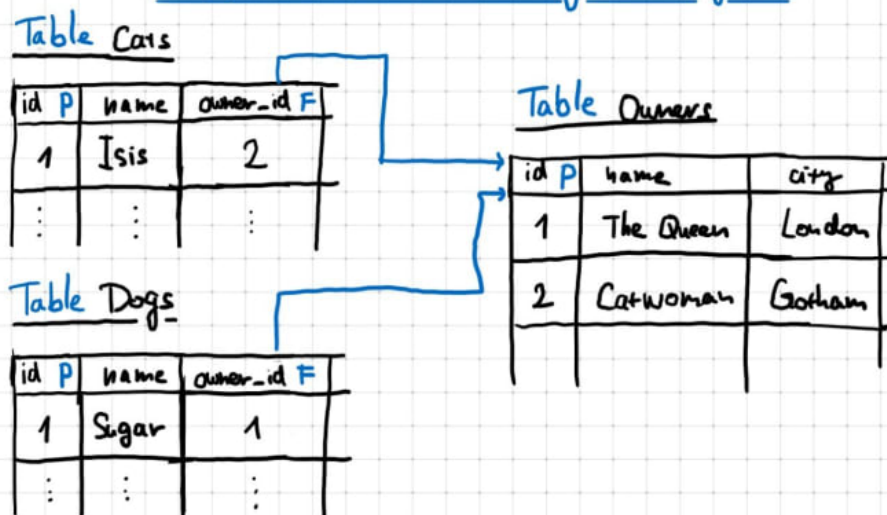
Object-Relational Mapping (ORM) is een techniek waarmee je gegevens uit een relationele database kunt opvragen en manipuleren met behulp van een objectgeoriënteerde programmeertaal.

Object Oriented Programming



ORM LAYER

Relational Database Management System



9.2 Wat is JPA?

De Java Persistence API (officieel hernoemd naar Jakarta Persistence API) is een specificatie die definieert hoe gegevens gepersisteerd kunnen worden in een Java-toepassing.

JPA is een specificatie. Dit betekent dat JPA bestaat uit implementatie richtlijnen voor de Java ORM-laag. De specificatie wordt alleen geleverd met interfaces, zonder eigenlijke implementatie. Er wordt een referentie-implementatie voorzien, maar andere bedrijven kunnen hun eigen implementatie van de specificatie maken en distribueren.

2.1. The Entity Class

The entity class must be annotated with the *Entity* annotation or denoted in the XML descriptor as an entity.

The entity class must have a no-arg constructor. The entity class may have other constructors as well. The no-arg constructor must be public or protected.

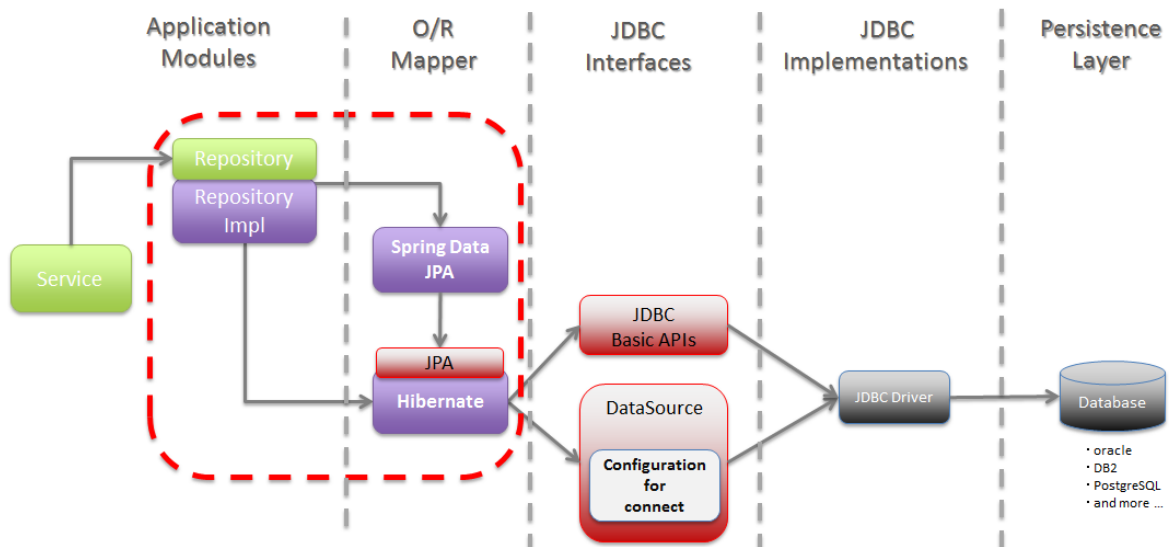
The entity class must be a top-level class. An enum or interface must not be designated as an entity.

In deze cursus gebruiken we Hibernate als implementatie van de JPA specificatie ¹.

Spring Data JPA biedt extra functionaliteit en vereenvoudigt het werken met JPA. Je krijgt een no-code implementatie van het repository design pattern. Database queries kunnen gegenereerd worden door Spring Data JPA aan de hand van een methodenaam.

9.3 Datasource

De datasource of gegevensbron is de configuratie die wordt gebruikt om verbinding te maken met een database of andere externe gegevensopslag. Het bevat informatie zoals de database-URL, gebruikersnaam, wachtwoord en andere instellingen die nodig zijn om gegevens uit een externe bron op te halen of ernaar te schrijven.



De datasource wordt gespecificeerd in het bestand *application.properties*. Volgende basis-eigenschappen zijn configureerbaar:

¹<https://hibernate.org/andhttps://hibernate.org/orm/>

spring.datasource.url	JDBC URL van de database.
spring.datasource.username	Gebruikersnaam voor de database.
spring.datasource.password	Paswoord voor de database.
spring.jpa.show-sql	Logging van de SQL statements. Default: false
spring.jpa.hibernate.ddl-auto	Automatisch genereren van tabellen. Mogelijke waarden: none (production), create, create-drop, validate and update. ²

De H2 in-memory databank die we in het vorige hoofdstuk hebben gebruikt, gaan we nu vervangen door een MySQL databank. We zullen de MySQL-database draaien in een Docker-container. Elke databaseserver heeft zijn eigen specifieke protocol voor de overdracht van gegevens tussen de server en het programma. Vergeet niet om de h2-dependency in de pom.xml te vervangen door de MySQL-driver.

```
<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
  <scope>runtime</scope>
</dependency>
```

De volgende docker-compose.yml kan gebruikt worden om een docker-container met een MySQL databaseserver te starten. Ga naar de folder waar dit bestand opgeslagen is en voer het commando `docker compose up` uit.

```
version: "3.3"

services:
  superhero-db:
    image: mysql:latest
    ports:
      - "3306:3306"
    environment:
      MYSQL_DATABASE: 'superherodb'
      MYSQL_USER: 'user'
      MYSQL_PASSWORD: 'password'
      MYSQL_ROOT_PASSWORD: 'admin'
```

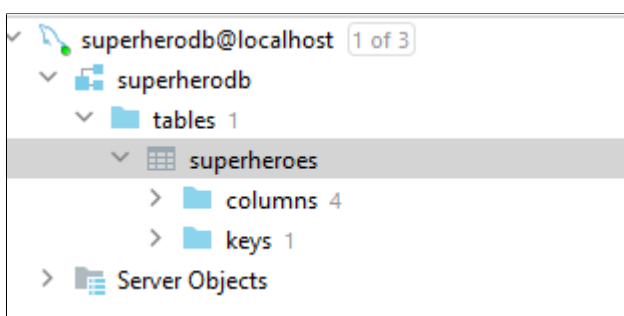
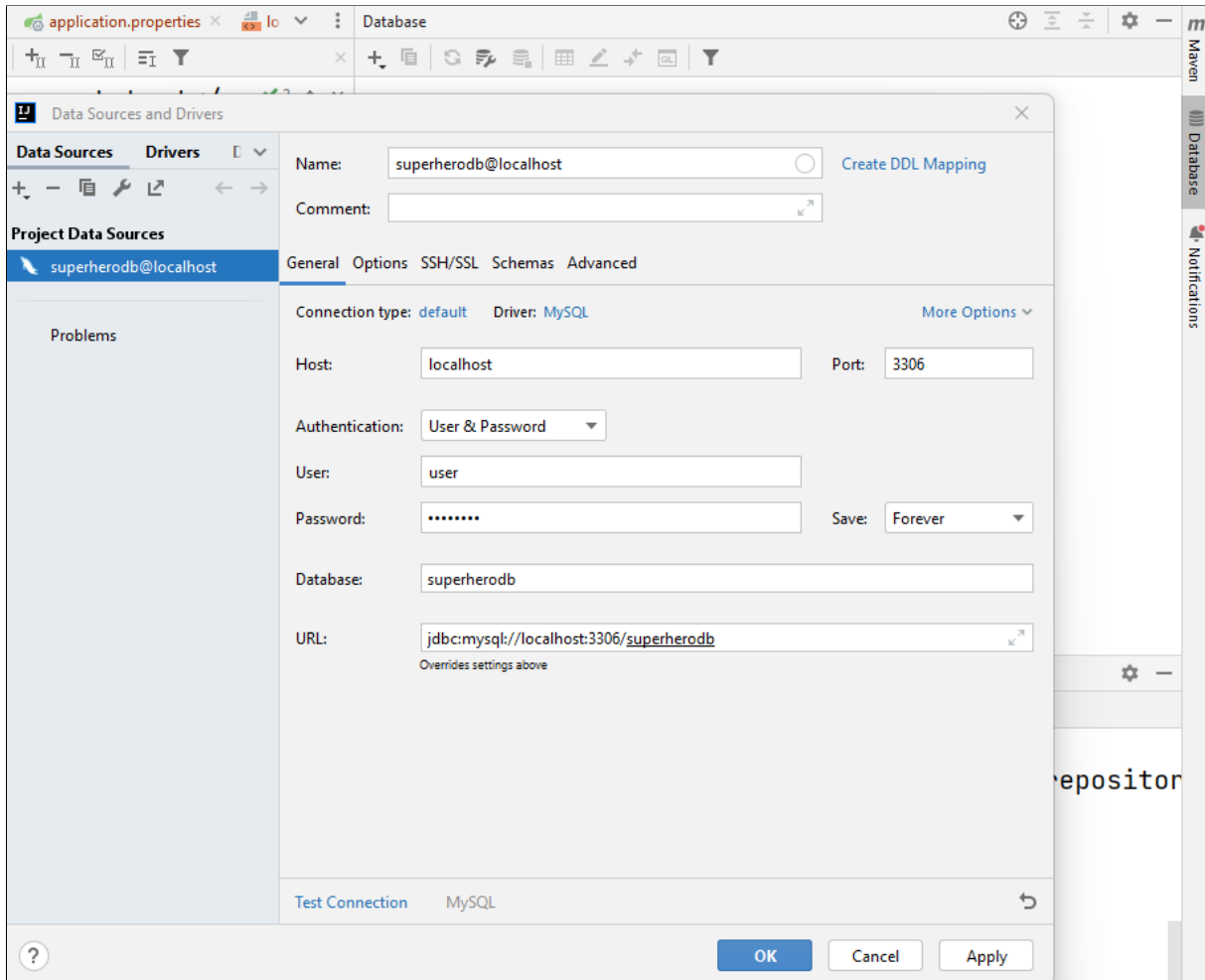
Oefening 9.1. Zorg dat je superhero-backend project gebruikmaakt van een MySQL databank. Pas de nodige dependencies in de pom.xml aan. Maak de folder `src/main/docker` aan en voeg hier het bestand `docker-compose.yml` toe. Start de MySQL database server op. Voeg de volgende configuratie toe in het bestand `application.properties`:

```
spring.datasource.url=jdbc:mysql://localhost:3306/superherodb
spring.datasource.username=user
spring.datasource.password=password
```

```
spring.jpa.hibernate.ddl-auto=update
```

Start de Spring Boot applicatie.

IntelliJ IDEA heeft een database-tool waarmee je de database kan inspecteren.



De eigenschap *spring.jpa.hibernate.ddl-auto* bepaalt hoe Hibernate omgaat met het genereren van het databaseschema. De volgende waarden zijn mogelijk voor deze eigenschap:

none	Hibernate voert geen schemageneratie of validatie uit.
validate	Hibernate valideert het bestaande schema ten opzichte van de entiteitsklassen en meldt eventuele verschillen zonder het schema te wijzigen.
update	Hibernate werkt het bestaande schema bij op basis van de entiteitsklassen. Het voegt tabellen, kolommen of beperkingen toe voor nieuwe entiteiten of wijzigingen aan bestaande entiteiten. Het laat bestaande tabellen of kolommen ongemoeid.
create	Hibernate maakt het schema vanaf nul, waarbij tabellen worden verwijderd en opnieuw gemaakt, wat kan leiden tot gegevensverlies.
create-drop	Vergelijkbaar met create, maar het schema wordt verwijderd wanneer de SessionFactory wordt gesloten (bijvoorbeeld wanneer de toepassing wordt afgesloten).

De eigenschappen create en update kunnen handig zijn tijdens de ontwikkeling, wanneer je wilt dat Hibernate het schema automatisch genereert.

In een productieomgeving wordt aanbevolen om automatische schemageneratie uit te schakelen (none) of, op zijn minst, alleen validatie uit te voeren (validate) zonder het schema te wijzigen. Het is ook belangrijk om versiebeheer voor het databaseschema te hebben. Hiervoor gebruik je migratietools zoals Flyway of Liquibase.

9.4 De Entity klasse

```
package be.px1.superhero.domain;

import jakarta.persistence.*;
import org.apache.logging.log4j.LogManager;
import org.apache.logging.log4j.Logger;

import java.util.ArrayList;
import java.util.List;

@Entity
@Table(name="superheroes")
public class Superhero {

    private static final Logger LOGGER =
        ↪ LogManager.getLogger(Superhero.class);

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```

private Long id;

private String firstName;
private String lastName;
@Column(unique = true)
private String superheroName;
@Column(name="notes")
private String description;

public Superhero() {
    // JPA only
}

public Superhero(String firstName, String lastName, String
    ↪ superheroName) {
    if (LOGGER.isDebugEnabled()) {
        LOGGER.debug("Creating a new superhero...");
    }
    this.firstName = firstName;
    this.lastName = lastName;
    this.superheroName = superheroName;
}

public Long getId() {
    return id;
}

public String getFirstName() {
    if (LOGGER.isTraceEnabled()) {
        LOGGER.trace(String.format("FirstName of %s was revealed.",
            ↪ superheroName));
    }
    return firstName;
}

public void setFirstName(String firstName) {
    this.firstName = firstName;
}

public String getLastName() {
    if (LOGGER.isFatalEnabled()) {
        LOGGER.fatal(String.format("LastName of %s was revealed.",
            ↪ superheroName));
    }
    return lastName;
}

public void setLastName(String lastName) {

```

```

        this.lastName = lastName;
    }

    public String getSuperheroName() {
        return superheroName;
    }

    public void setSuperheroName(String superheroName) {
        this.superheroName = superheroName;
    }

    @Override
    public String toString() {
        return superheroName;
    }
}

```

Een JPA-entityklasse is een POJO (Plain Old Java Object) klasse die gebruikt wordt om objecten in de database te vertegenwoordigen.

De **@Id**-annotatie duidt de primaire sleutel aan.

De **@Column**-annotatie wordt gebruikt om details van de kolom te specificeren. Met de **@Column**-annotatie kan je de volgende attributen aanpassen:

- **name**: specificeer de naam van de kolom.
- **length**: specificeer de grootte van de kolom, met name voor een String-waarde.
- **nullable**: markeer de kolom als NOT NULL wanneer het databaseschema wordt gegenereerd.
- **unique**: de kolom om alleen unieke waarden te bevatten.

Oefening 9.2. □ Maak een entity-klasse Mission.

Een missie heeft de volgende velden:

id	Long	De primaire sleutel.
name	String	Unieke naam van de missie.
completed	boolean	De missie is voltooid.

9.5 Repository

De JpaRepository-interface is een uitbreiding van CrudRepository. JpaRepository breidt PagingAndSortingRepository uit, dat op zijn beurt CrudRepository uitbreidt.

Hun belangrijkste functies zijn:

- CrudRepository biedt voornamelijk CRUD-functies.
- PagingAndSortingRepository biedt methoden voor paginering en het sorteren van records.

- JpaRepository biedt enkele JPA-gerelateerde methoden, zoals het flushen van de persistentiecontext en het verwijderen van records in batches.

Oefening 9.3. Maak de interface MissionRepository aan. Maak daarna de MissionService en MissionController zodat het mogelijk wordt om een missies aan te maken, op te halen (zowel adhv id als een volledige lijst), te updaten en te verwijderen. Voeg de nodige DTOs toe. Hou er rekening mee dat de namen van missies uniek moeten zijn. Voorzie je eigen exceptions en zorg voor de nodige foutafhandeling.

mission-controller

GET /missions/{missionId}

PUT /missions/{missionId}

DELETE /missions/{missionId}

GET /missions

POST /missions

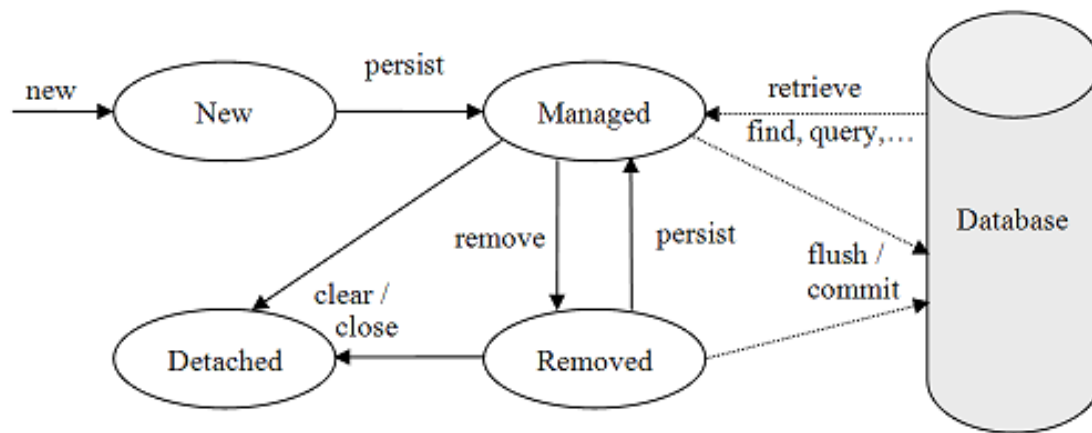
```
MissionRequest {
  missionName      string
  completed        boolean
}
```

```
MissionDTO {
  id                integer($int64)
  missionName      string
  completed        boolean
}
```

9.6 Entity lifecycle

De **PersistenceContext** is één van de belangrijkste concepten in JPA. Het is de verzameling van alle entiteitsobjecten die momenteel gebruikt worden of recentelijk zijn ge-

bruikt. Je kunt de PersistenceContext beschouwen als een soort first-level cache. Elk entiteitsobject in de PersistenceContext vertegenwoordigt een record in de database. De PersistenceContext beheert deze entiteitsobjecten en hun levenscyclus. Elk entiteitsobject heeft één van de 4 statussen: New, Managed, Removed en Detached.



Een nieuw aangemaakt entiteitsobject bevindt zich in status **New**. Het entiteitsobject is nog niet opgeslagen, dus er is nog geen database-record. Het De PersistenceContext is nog niet op de hoogte van het bestaan van het entiteitsobject.

Alle entiteitsobjecten die aan de PersistenceContext zijn gekoppeld, bevinden zich in de status **Managed**. Een entiteitsobject wordt beheerd wanneer het wordt opgeslagen in de database. Entiteitsobjecten die uit de database worden opgehaald, bevinden zich ook in de status **Managed**. Als een beheerd entiteitsobject wordt aangepast binnen een actieve transactie, dan detecteert de PersistenceContext de wijziging en wordt deze doorgegeven aan de database.

Wanneer een beheerd entiteitsobject binnen een actieve transactie wordt verwijderd, verandert de status van beheerd naar verwijderd, en wordt het record fysiek uit de database verwijderd (wanneer de transactie wordt bevestigd).

Een entiteit wordt losgekoppeld (detached) wanneer deze niet langer wordt beheerd door de PersistenceContext maar nog steeds een record in de database vertegenwoordigt. Losgekoppelde entiteitsobjecten hebben beperkte functionaliteit.

Bij het gebruik van Spring Data JPA hoeven we zelf geen interactie met de entitymanager te implementeren. Wanneer een Repository wordt aangemaakt, biedt de door Spring Data JPA geleverde SimpleJpaRepository-klasse de standaardimplementatie. De SimpleJpaRepository-klasse gebruikt intern de JPA-entitymanager.

Oefening 9.4. Bestudeer de klasse SimpleJpaRepository (bijv. save()-methode).

9.7 Queries

Met Spring Data JPA kun je query-methoden creëren om records uit de database te selecteren zonder SQL-queries te schrijven. Achter de schermen zal Spring Data JPA SQL-queries genereren op basis van de query-methode.

Spring Data JPA kan de query afleiden uit de naam van de query-methode.

In een UserRepository kan je bijvoorbeeld een query-methode hebben met de naam findByEmailAddressAndLastname().

```
public interface UserRepository extends JpaRepository<User, Long> {  
    List<User> findByEmailAddressAndLastname(String emailAddress, String  
        ↪ lastname);  
}
```

Spring Data JPA genereert achter de schermen een query die wordt vertaald naar JPQL (Java Persistence Query Language, een dialect van SQL): select u from User u where u.emailAddress = ?1 and u.lastname = ?2.

Oefening 9.5. Voeg in de interface SuperheroRepository een methode toe om een Superhero terug te vinden aan de hand van zijn superheldennaam. De methode retourneert een Optional.

9.8 Relationships

Vanuit een databaseperspectief hebben we de volgende relaties tussen tabellen:

- one-to-many
- one-to-one
- many-to-many

JPA biedt meerdere annotaties aan om deze relaties in de code te weerspiegelen. Voor de superhero-backend gebruiken we een many-to-many relatie tussen superheroen en missies. Een superhero kan deelnemen aan meerdere missies, en meerdere superheroen bundelen hun krachten in één missie. Een koppelingstabel is noodzakelijk om deze gegevens op te slaan.

Relaties kunnen zowel unidirectioneel als bidirectioneel zijn. Slechts één partij in een relatie is de eigenaar van de relatie.

9.8.1 Many-to-many bidirectional relationship

De Superhero-klasse is de eigenaar van de relatie tussen Mission en Superhero. Omdat de relatie bidirectioneel is, onderhouden we altijd een set van de missies van een superhero en in een missie onderhouden we een set van de deelnemende superheroen. Wanneer je een missie aan een superhero toevoegt, moet je ervoor zorgen dat de superhero ook wordt toegevoegd aan de lijst van superheroen van de missie. Bestudeer onderstaande code.

```
package be.px1.superhero.domain;  
  
import jakarta.persistence.*;  
import org.apache.logging.log4j.LogManager;  
import org.apache.logging.log4j.Logger;
```



```

import java.util.ArrayList;
import java.util.List;

@Entity
@Table(name="superheroes")
public class Superhero {

    private static final Logger LOGGER =
        ↪ LogManager.getLogger(Superhero.class);

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String firstName;
    private String lastName;
    @Column(unique = true)
    private String superheroName;
    @Column(name="notes")
    private String description;
    @ManyToMany
    private List<Mission> missions = new ArrayList<>();

    public Superhero() {
        // JPA only
    }

    public Superhero(String firstName, String lastName, String
        ↪ superheroName) {
        if (LOGGER.isDebugEnabled()) {
            LOGGER.debug("Creating a new superhero...");
        }
        this.firstName = firstName;
        this.lastName = lastName;
        this.superheroName = superheroName;
    }

    public Long getId() {
        return id;
    }

    public String getFirstName() {
        if (LOGGER.isTraceEnabled()) {
            LOGGER.trace(String.format("FirstName of %s was revealed.",
                ↪ superheroName));
        }
        return firstName;
    }
}

```

```

public void setFirstName(String firstName) {
    this.firstName = firstName;
}

public String getLastName() {
    if (LOGGER.isFatalEnabled()) {
        LOGGER.fatal(String.format("LastName of %s was revealed.",
            ↪ superheroName));
    }
    return lastName;
}

public void addMission(Mission mission) {
    if (mission.isCompleted()) {
        throw new IllegalArgumentException("This mission was already
            ↪ completed.");
    }
    if (onMission(mission)) {
        throw new IllegalArgumentException("This mission was already
            ↪ assigned.");
    }
    missions.add(mission);
    mission.addSuperhero(this);
}

public boolean onMission(Mission mission) {
    return missions.stream().anyMatch(m ->
        ↪ m.getName().equals(mission.getName()));
}

public List<Mission> getMissions() {
    return missions;
}

public void setLastName(String lastName) {
    this.lastName = lastName;
}

public String getSuperheroName() {
    return superheroName;
}

public void setSuperheroName(String superheroName) {
    this.superheroName = superheroName;
}

@Override

```

```
    public String toString() {  
        return superheroName;  
    }  
}
```

```
package be.px1.superhero.domain;  
  
import jakarta.persistence.*;  
  
import java.util.ArrayList;  
import java.util.List;  
  
@Entity  
public class Mission {  
    @Id  
    @GeneratedValue(strategy = GenerationType.IDENTITY)  
    private Long id;  
  
    @Column(unique = true)  
    private String name;  
  
    private boolean completed;  
  
    @ManyToMany(mappedBy = "missions")  
    private List<Superhero> superheroes = new ArrayList<>();  
  
    public Long getId() {  
        return id;  
    }  
  
    public String getName() {  
        return name;  
    }  
  
    public void setName(String name) {  
        this.name = name;  
    }  
  
    public boolean isCompleted() {  
        return completed;  
    }  
  
    public void setCompleted(boolean completed) {  
        this.completed = completed;  
    }  
  
    public void addSuperhero(Superhero superhero) {  
        superheroes.add(superhero);  
    }  
}
```

```

    }

    public List<Superhero> getSuperheroes() {
        return superheroes;
    }
}

```

Oefening 9.6. Voeg de bi-directionele many-to-many relatie toe tussen de entity-klasse SuperHero en Mission.

9.9 Transactions

Nu kunnen we superhelden laten deelnemen aan een missie.

POST /superheroes/add-superhero-to-mission

Parameters: No parameters

Request body *required*: application/json

Example Value | Schema

```

SuperheroMissionRequest {
  superheroId integer($int64)
  missionId integer($int64)
}

```

Responses

Code	Description	Links
200	OK	No links

```

package be.px1.superhero.service.impl;

import be.px1.superhero.api.SuperheroDTO;
import be.px1.superhero.api.SuperheroRequest;
import be.px1.superhero.domain.Mission;
import be.px1.superhero.domain.Superhero;
import be.px1.superhero.exception.ResourceNotFoundException;
import be.px1.superhero.repository.MissionRepository;
import be.px1.superhero.repository.SuperheroRepository;
import be.px1.superhero.service.SuperheroService;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;

@Service
public class SuperheroServiceImpl implements SuperheroService {

```

```

private final SuperheroRepository superheroRepository;
private final MissionRepository missionRepository;

public SuperheroServiceImpl(SuperheroRepository superheroRepository,
                           MissionRepository missionRepository) {
    this.superheroRepository = superheroRepository;
    this.missionRepository = missionRepository;
}

// other methods not included

@Transactional
public void addSuperheroToMission(Long superheroId, Long missionId) {
    Superhero superhero =
        ↳ superheroRepository.findById(superheroId).orElseThrow(() ->
        ↳ new ResourceNotFoundException("Superhero", "ID",
        ↳ superheroId));
    Mission mission =
        ↳ missionRepository.findById(missionId).orElseThrow(() -> new
        ↳ ResourceNotFoundException("Mission", "ID", missionId));
    superhero.addMission(mission);
}
}

```

In de service-laag halen we onze SuperHero- en Mission-entiteitsobjecten op en werpen een uitzondering op als ze niet bestaan. Vervolgens wordt het Mission-entiteitsobject toegevoegd aan de lijst van missies van de superheld en vice versa. Dankzij de @Transactional-annotatie wordt dit alles uitgevoerd in één transactie. Wanneer de transactie wordt bevestigd, worden de wijzigingen opgeslagen in de database.

Tot slot hebben we extra velden nodig in de DTO's voor gedetailleerde informatie over de superheld en zijn missies.

```

{
  "id": 0,
  "firstName": "string",
  "lastName": "string",
  "superheroName": "string",
  "missions": [
    {
      "id": 0,
      "missionName": "string",
      "completed": true
    }
  ]
}

```

Hoofdstuk 10

Database relaties met Spring Data JPA

Doelstellingen

De junior-collega

1. Kan entiteitsklassen implementeren.
2. Kan verschillende soorten relaties tussen entiteitsklassen implementeren: on-ton, many-to-one and many-to-many op een unidirectionele en bidirectionele manier.
3. Kan eenvoudige queries implementeren.
4. Kan het $N + 1$ queryprobleem uitleggen, identificeren en oplossen. (optioneel)
5. Kan 'lazy loading' gebruiken, zelfs als OSIV niet is ingeschakeld. (optioneel)

10.1 Relationships

Spring Data JPA ondersteunt relationele database associaties:

- one-to-one associaties,
- many-to-one associaties en
- many-to-many associaties.

Je kan de associaties zowel uni- als bidirectional maken.

10.1.1 One-to-One associatie

In een één-op-één-relatie is één record in een tabel geassocieerd met slechts één record in een andere tabel.

```
package be.px1.demo.domain;

import jakarta.persistence.*;

@Entity
@Table(name = "contact_info")
public class ContactInformation {
    @Id
```

```

@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;
private String phone;
private String email;
private String linkedIn;

public Long getId() {
    return id;
}

public String getPhone() {
    return phone;
}

public void setPhone(String phone) {
    this.phone = phone;
}

public String getEmail() {
    return email;
}

public void setEmail(String email) {
    this.email = email;
}

public String getLinkedIn() {
    return linkedIn;
}

public void setLinkedIn(String linkedIn) {
    this.linkedIn = linkedIn;
}

@Override
public String toString() {
    return "ContactInformation{" +
        "id=" + id +
        ", phone='" + phone + '\'' +
        ", email='" + email + '\'' +
        ", linkedIn='" + linkedIn + '\'' +
        '}';
}
}

```

Een one-to-one relatie resulteert in een foreign key in de tabel die eigenaar (owner) is van de associatie. De instantievariabelen met het datatype van de geassocieerde entiteitsklasse wordt geannoteerd met @OneToOne. Je kunt de naam van de kolom met de foreign key

aanpassen met de annotatie @JoinColumn.

```
package be.px1.demo.domain;

import jakarta.persistence.*;

@Entity
public class Researcher {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Column(length = 40, nullable = false)
    private String firstname;
    @Column(length = 40, nullable = false)
    private String lastname;
    @OneToOne(cascade = CascadeType.ALL)
    private ContactInformation contactInformation;

    public Long getId() {
        return id;
    }

    public String getFirstname() {
        return firstname;
    }

    public void setFirstname(String firstname) {
        this.firstname = firstname;
    }

    public String getLastname() {
        return lastname;
    }

    public void setLastname(String lastname) {
        this.lastname = lastname;
    }

    public ContactInformation getContactInformation() {
        return contactInformation;
    }

    public void setContactInformation(ContactInformation
        ↪ contactInformation) {
        this.contactInformation = contactInformation;
    }
}
```



```

@Override
public String toString() {
    return "Researcher{" +
        "id=" + id +
        ", firstname='" + firstname + '\'' +
        ", lastname='" + lastname + '\'' +
        ", contactInformation=" + contactInformation +
        '}';
}
}

```

De betekenis van CascadeType.ALL is dat de persistentiecontext alle EntityManager-operaties (PERSIST, REMOVE, REFRESH, MERGE, DETACH) zal doorgeven (cascaden) naar de gerelateerde entiteiten. Dit betekent dat wanneer een operatie wordt uitgevoerd op de hoofdentiteit, deze operatie automatisch ook wordt toegepast op alle gerelateerde entiteiten. Bijvoorbeeld, als een hoofdentiteit wordt opgeslagen, worden alle gerelateerde entiteiten ook opgeslagen. Hetzelfde geldt voor verwijderen, vernieuwen, samenvoegen en loskoppelen.

10.1.2 Many-to-one relatie

Hier is de entiteitsklasse Project.

```

package be.px1.demo.domain;

import jakarta.persistence.*;

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;

@Entity
public class Project {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    private LocalDate start;
    @Enumerated(value= EnumType.STRING)
    private ProjectPhase projectPhase;

    public Project() {
        // JPA only
    }

    public Project(String name) {
        this.name = name;
        this.start = LocalDate.now();
    }
}

```

```

        this.projectPhase = ProjectPhase.INITIATING;
    }

    public Long getId() {
        return id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public LocalDate getStart() {
        return start;
    }

    public void setStart(LocalDate start) {
        this.start = start;
    }

    public ProjectPhase getProjectPhase() {
        return projectPhase;
    }

    public void setProjectPhase(ProjectPhase projectPhase) {
        this.projectPhase = projectPhase;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) {
            return true;
        }
        if (o == null || getClass() != o.getClass()) {
            return false;
        }

        Project project = (Project) o;

        return id != null ? id.equals(project.id) : project.id == null;
    }

    @Override
    public int hashCode() {
        return id != null ? id.hashCode() : 0;
    }

```

```

    }

    @Override
    public String toString() {
        return name;
    }
}

```

```

package be.px1.paj.domain;

public enum ProjectPhase {
    INITIATING,
    PLANNING,
    EXECUTING,
    CLOSING;
}

```

De meest voorkomende optie om een enum-waarde in JPA te mappen naar en vanuit zijn database-representatie is het gebruik van de `@Enumerated` annotatie. Op deze manier zal de JPA-provider de enum-waarde omzetten naar de ordinale waarde of String-waarde. Wanneer we de String-waarde gebruiken, kunnen we veilig nieuwe enum-waarden toevoegen of de volgorde van onze enum-waarden wijzigen. Het hernoemen van een enum-waarde zal echter nog steeds de databasedata breken.

Stel dat er meerdere onderzoekers werken aan één project, en elke onderzoeker is toegewezen aan exact één project.

Om de relatie te creëren tussen een onderzoeker en het project waaraan hij werkt, voegen we een `@ManyToOne` relatie toe in de entiteitsklasse `Researcher`.

```

package be.px1.demo.domain;

import jakarta.persistence.*;
import org.apache.logging.log4j.LogManager;
import org.apache.logging.log4j.Logger;

@Entity
public class Researcher {

    private static final Logger LOGGER =
        ↪ LogManager.getLogger(Researcher.class);

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Column(length = 40, nullable = false)
    private String firstname;
    @Column(length = 40, nullable = false)
    private String lastname;
}

```

```

@OneToOne(cascade = CascadeType.ALL)
private ContactInformation contactInformation;
@ManyToOne
private Project project;

public Long getId() {
    return id;
}

public String getFirstname() {
    return firstname;
}

public void setFirstname(String firstname) {
    this.firstname = firstname;
}

public String getLastname() {
    return lastname;
}

public void setLastname(String lastname) {
    this.lastname = lastname;
}

public ContactInformation getContactInformation() {
    return contactInformation;
}

public void setContactInformation(ContactInformation
    ↪ contactInformation) {
    this.contactInformation = contactInformation;
}

public void setProject(Project project) {
    if (this.project != null) {
        if (this.project.equals(project)) {
            LOGGER.info("Researcher [" + id + "] already assigned to ["
                ↪ + project.getName() + "]");
            return;
        }
    }
    this.project = project;
}

@Override
public String toString() {
    return String.format("%s %s (%d)", firstname, lastname, id);
}

```

```
}  
}
```

Op dit moment is het niet mogelijk om te zien welke onderzoekers aan een project werken. Om dit mogelijk te maken moeten we de relatie tussen project en onderzoeker bidirectioneel maken. Onderzoeker wordt gedefinieerd als de **eigenaar** of owner van de relatie. Om deze associatie bidirectioneel te maken, moeten we de entiteit waarnaar verwezen wordt definiëren. Bij de inverse relatie mappen we naar de eigenaarskant van de relatie. De waarde van mappedBy is de naam van het attribuut voor associatie-mapping aan de eigenaarskant. De manier om deze gerelateerde data op te halen is standaard **lazy**. Dit betekent dat de onderzoekers voor een project alleen uit de database worden opgehaald wanneer de methode `getResearchers()` wordt aangeroepen.

```
package be.px1.demo.domain;  
  
import jakarta.persistence.*;  
  
import java.time.LocalDate;  
import java.util.ArrayList;  
import java.util.List;  
  
@Entity  
public class Project {  
    @Id  
    @GeneratedValue(strategy = GenerationType.IDENTITY)  
    private Long id;  
    private String name;  
    private LocalDate start;  
    @OneToMany(mappedBy = "project")  
    private List<Researcher> researchers = new ArrayList<>();  
    @Enumerated(value= EnumType.STRING)  
    private ProjectPhase projectPhase;  
  
    public Project() {  
    }  
  
    public Project(String name) {  
        this.name = name;  
        this.start = LocalDate.now();  
        this.projectPhase = ProjectPhase.INITIATING;  
    }  
  
    public Long getId() {  
        return id;  
    }  
  
    public String getName() {  
        return name;  
    }  
}
```

```

    }

    public void setName(String name) {
        this.name = name;
    }

    public LocalDate getStart() {
        return start;
    }

    public void setStart(LocalDate start) {
        this.start = start;
    }

    public List<Researcher> getResearchers() {
        return researchers;
    }

    public ProjectPhase getProjectPhase() {
        return projectPhase;
    }

    public void setProjectPhase(ProjectPhase projectPhase) {
        this.projectPhase = projectPhase;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) {
            return true;
        }
        if (o == null || getClass() != o.getClass()) {
            return false;
        }

        Project project = (Project) o;

        return id != null ? id.equals(project.id) : project.id == null;
    }

    @Override
    public int hashCode() {
        return id != null ? id.hashCode() : 0;
    }

    public void addResearcher(Researcher researcher) {
        researchers.add(researcher);
    }

```

```

    public void removeResearcher(Researcher researcher) {
        researchers.remove(researcher);
    }

    @Override
    public String toString() {
        return name;
    }
}

```

Bij het gebruik van een bidirectionele relatie moeten beide associaties binnen de relatie zorgvuldig worden beheerd. Daarom moeten we de methode *setProject()* in de klasse *Researcher* herschrijven.

```

public void setProject(Project project) {
    if (this.project != null) {
        if (this.project.equals(project)) {
            LOGGER.info("Researcher [" + id + "] already assigned to [" +
                ↪ project.getName() + "]");
            return;
        }
        this.project.removeResearcher(this);
    }
    this.project = project;
    project.addResearcher(this);
}

```

Het is geen goed idee om de klasse *Project* eigenaar te maken van de relatie tussen projecten en onderzoekers. Je zult dan een inefficiënt databaseschema krijgen. Meer informatie is te vinden in deze post:

<https://vladmihalcea.com/the-best-way-to-map-a-onetomany-association-with-jpa-and-hibernate/>.

10.1.3 Many-to-many relaties

Eigenlijk willen we dat onze onderzoekers aan meerdere projecten werken. Daarom moeten we een *@ManyToMany* relatie introduceren tussen de klasse *Researcher* en *Project*.

```

package be.px1.demo.domain;

import org.apache.logging.log4j.LogManager;
import org.apache.logging.log4j.Logger;

import jakarta.persistence.*;
import java.util.ArrayList;
import java.util.List;

```

```

@Entity
public class Researcher {

    // all other properties left out intentionally

    @ManyToMany
    private List<Project> projects = new ArrayList<>();

    public void addProject(Project project) {
        if (this.projects.contains(project)) {
            LOGGER.info("Researcher [" + id + "] already assigned to ["
                ↪ + project.getName() + "]");
            return;
        }
        this.projects.add(project);
        project.addResearcher(this);
    }

    public List<Project> getProjects() {
        return projects;
    }

    // all other methods left out intentionally
}

```

Er zal nu een koppeltabel ontstaan waar wordt bijgehouden welke onderzoekers werken aan welke projecten.

```

package be.px1.demo.domain;

import jakarta.persistence.*;
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;

@Entity
public class Project {

    // all other properties left out intentionally

    @ManyToMany(mappedBy = "projects")
    private List<Researcher> researchers = new ArrayList<>();

    public void addResearcher(Researcher researcher) {
        researchers.add(researcher);
    }
}

```



```
    // all other methods left out intentionally
}
```

10.2 N + 1 query problem (optioneel)

Stel dat we de entiteitsklasse `Post` en de entiteitsklasse `PostComment` hebben. Er is een unidirectionele veel-op-één relatie tussen `PostComment` en `Post`. Een `PostComment` behoort tot precies één `Post`, maar voor één `Post` kunnen er meerdere `PostComments` bestaan.

```
package be.px1.demo.domain;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.Id;

@Entity
public class Post {
    @Id
    @GeneratedValue
    private Long id;
    private String title;

    public Post() {
        // JPA only
    }

    public Post(String title) {
        this.title = title;
    }

    public Long getId() {
        return id;
    }

    public String getTitle() {
        return title;
    }
}
```

```
package be.px1.demo.domain;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
```

```

import jakarta.persistence.Id;
import jakarta.persistence.ManyToOne;

@Entity
public class PostComment {
    @Id
    @GeneratedValue
    private Long id;
    @ManyToOne
    private Post post;
    private String review;

    public PostComment() {
        // JPA only
    }

    public PostComment(Post post, String review) {
        this.post = post;
        this.review = review;
    }

    public Long getId() {
        return id;
    }

    public Post getPost() {
        return post;
    }

    public String getReview() {
        return review;
    }
}

```

We hebben een JpaRepository voor beide entiteitsklassen. De PostService implementeert een methode om een post te maken, een methode voor het maken van reacties en een methode om alle reacties uit de database op te halen.

```

package be.px1.demo.service;

import be.px1.demo.api.dto.PostCommentDTO;
import be.px1.demo.domain.Post;
import be.px1.demo.domain.PostComment;
import be.px1.demo.exception.ResourceNotFoundException;
import be.px1.demo.repository.PostCommentRepository;
import be.px1.demo.repository.PostRepository;
import org.springframework.stereotype.Service;

```

```

import java.util.List;

@Service
public class PostService {

    private final PostCommentRepository postCommentRepository;
    private final PostRepository postRepository;

    public PostService(PostCommentRepository postCommentRepository,
                       PostRepository postRepository) {
        this.postCommentRepository = postCommentRepository;
        this.postRepository = postRepository;
    }

    public long createPost(String title) {
        return postRepository.save(new Post(title)).getId();
    }

    public void createPostComment(long postId, String review) {
        Post post = postRepository.findById(postId).orElseThrow(() -> new
            ↳ ResourceNotFoundException("Post", "id", postId));
        PostComment postComment = new PostComment(post, review);
        postCommentRepository.save(postComment);
    }

    public List<PostCommentDTO> findAll() {
        List<PostComment> allComments = postCommentRepository.findAll();
        return allComments.stream().map(pc -> new
            ↳ PostCommentDTO(pc.getPost().getTitle(),
            ↳ pc.getReview()))).toList();
    }
}

```

Dan hebben we de PostController waar we twee endpoints implementeren. Eén endpoint voor het genereren van testdata en een ander endpoint voor het ophalen van alle reacties op de post.

```

package be.px1.demo.api;

import be.px1.demo.api.dto.PostCommentDTO;
import be.px1.demo.service.PostService;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.List;

```

```

@RestController
@RequestMapping("posts")
public class PostController {

    private final PostService postService;

    public PostController(PostService postService) {
        this.postService = postService;
    }

    @GetMapping("init")
    public void init() {
        long post1Id = postService.createPost("PXL'er Ward Lemmelijn kroont
            ↳ zich tot wereldkampioen indoorroeien.");
        long post2Id = postService.createPost("PXL naar finale in
            ↳ Cybersecurity challenge.");
        long post3Id = postService.createPost("Luister naar PXLRadio!!");

        postService.createPostComment(post1Id, "Schitterend ***");
        postService.createPostComment(post1Id, "Proficiat!");
        postService.createPostComment(post2Id, "Ik ben zeker van de
            ↳ partij!");
        postService.createPostComment(post2Id, "Ik hou van uitdagingen!");
        postService.createPostComment(post3Id, "Leuke muziek!");
        postService.createPostComment(post3Id, "Toffe radiozender!");
        postService.createPostComment(post3Id, "Zet die plaat af.");
    }

    @GetMapping
    public List<PostCommentDTO> getAllComments() {
        return postService.findAll();
    }
}

```

Om alle SQL instructies te kunnen analyseren enablen we de property show-sql in het bestand application.properties.

```

spring.jpa.show-sql=true
# Following property can be used to format the sql shown
# spring.jpa.properties.hibernate.format_sql=true

```

Nadat we het endpoint hebben aangeroepen om testgegevens te genereren, halen we alle reacties op uit de database. In de logbestanden vind je de volgende sql instructies.

```

Hibernate: select p1_0.id,p1_0.post_id,p1_0.review from post_comment p1_0
Hibernate: select p1_0.id,p1_0.title from post p1_0 where p1_0.id=?
Hibernate: select p1_0.id,p1_0.title from post p1_0 where p1_0.id=?
Hibernate: select p1_0.id,p1_0.title from post p1_0 where p1_0.id=?

```

Nadat de reacties uit de database zijn opgehaald, wordt de originele post bij de reactie ook opgehaald. Dus, als je N originele posts hebt, worden er N+1-query's uitgevoerd. Dat is niet efficiënt!

Je zou dit probleem moeten aanpakken door de query voor het ophalen van alle postreacties te herschrijven.

```
package be.px1.demo.repository;

import be.px1.demo.domain.PostComment;
import org.springframework.data.jpa.repository.EntityGraph;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.List;

public interface PostCommentRepository extends JpaRepository<PostComment,
    ↪ Long> {

    @Override
    @EntityGraph(
        type = EntityGraph.EntityGraphType.FETCH,
        attributePaths = {
            "post"
        }
    )
    List<PostComment> findAll();
}
```

Bij het ophalen van reacties met de methode `findAll()`, zorgt de `@EntityGraph` annotatie ervoor dat Spring Data JPA en Hibernate de geassocieerde entiteiten ophalen die zijn opgenomen in `attributePaths`.

Er wordt nu slechts één query uitgevoerd.

```
Hibernate: select p1_0.id,p2_0.id,p2_0.title,p1_0.review from post_comment
    ↪ p1_0 left join post p2_0 on p2_0.id=p1_0.post_id
```

Het is tijdens de development-fase steeds belangrijk om de gegenereerde query's te controleren. Het is een goede gewoonte om SQL-output in te schakelen tijdens het werken aan een Spring Boot-project, gewoon als een sanity check. Je zult af en toe problemen vinden waarvan je je niet bewust was, door het controleren van de SQL-output.

Een uitgebreid voorbeeld van het N+1 query-probleem en de oplossing is te vinden in deze post: <https://tech.asimio.net/2020/11/06/Preventing-N-plus-1-select-problem-using-Spring-Data-JPA-EntityGraph.html>.

10.3 Lazy loading en OSIV (optioneel)

De entiteiten in een @OneToMany en @ManyToMany relatie worden op een 'lazy' manier geladen. Het laden van de data gebeurt enkel wanneer het nodig is. **Lazy loading** verbetert de performantie van de applicatie. Je hebt echter een transactie (of sessie) nodig om de geassocieerde data uit de database op te halen. Spring Boot schakelt echter standaard een mechanisme in dat OSIV (Open Session in View) wordt genoemd. In Spring Boot applicaties die JPA gebruiken, worden transacties gecreëerd zelfs voordat requests bij de restcontroller aankomen, zelfs als het verzoek helemaal geen databasebewerkingen uitvoert. De Spring OpenSessionInViewInterceptor klasse beheert deze sessies. OSIV is een slecht idee vanuit een prestatie- en schaalbaarheidsperspectief.

Zorg er dus voor dat je in het configuratiebestand application.properties de volgende eigenschap toevoegt:

```
spring.jpa.open-in-view=false
```

Wat gebeurt er als je gegevens ophaalt van een 'lazy loaded' associatie zonder een actieve transactie of sessie?

Laten we starten met de associatie tussen Post en PostComment bidirectioneel te maken.

```
package be.px1.demo.domain;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.Id;
import jakarta.persistence.OneToMany;

import java.util.List;

@Entity
public class Post {
    @Id
    @GeneratedValue
    private Long id;
    private String title;
    @OneToMany(mappedBy = "post")
    private List<PostComment> commentList;

    public Post() {
        // JPA only
    }

    public Post(String title) {
        this.title = title;
    }
}
```

```

    public Long getId() {
        return id;
    }

    public String getTitle() {
        return title;
    }

    public List<PostComment> getCommentList() {
        return commentList;
    }
}

```

```

package be.px1.demo.service;

import be.px1.demo.api.dto.PostCommentDTO;
import be.px1.demo.api.dto.PostDTO;
import be.px1.demo.domain.Post;
import be.px1.demo.domain.PostComment;
import be.px1.demo.exception.ResourceNotFoundException;
import be.px1.demo.repository.PostCommentRepository;
import be.px1.demo.repository.PostRepository;
import org.springframework.stereotype.Service;

import java.util.List;

@Service
public class PostService {

    private final PostCommentRepository postCommentRepository;
    private final PostRepository postRepository;

    public PostService(PostCommentRepository postCommentRepository,
                       PostRepository postRepository) {
        this.postCommentRepository = postCommentRepository;
        this.postRepository = postRepository;
    }

    public PostDTO getPost(long postId) {
        Post post = postRepository.findById(postId)
            .orElseThrow(() -> new ResourceNotFoundException("Post",
                ↳ "id", postId));
        return new PostDTO(post.getTitle(),
            post.getCommentList().stream().map(PostComment::getReview).toList());
    }
}

```

```

package be.px1.demo.api.dto;

import java.util.List;

public record PostDTO(String title, List<String> review) {
}

```

```

package be.px1.demo.api;

import be.px1.demo.api.dto.PostCommentDTO;
import be.px1.demo.api.dto.PostDTO;
import be.px1.demo.service.PostService;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.List;

@RestController
@RequestMapping("posts")
public class PostController {

    private final PostService postService;

    public PostController(PostService postService) {
        this.postService = postService;
    }

    @GetMapping(path = "{postId}")
    public PostDTO getPost(@PathVariable long postId) {
        return postService.getPost(postId);
    }
}

```

Als we het endpoint aanroepen om een post op te halen krijgen we de volgende LazyInitializationException:

```

org.hibernate.LazyInitializationException: failed to lazily initialize a collection of role: be.px1.demo.domain.Post.commentList: could not
  ↳ initialize proxy - no Session
  at org.hibernate.collection.spi.AbstractPersistentCollection.throwLazyInitializationException(AbstractPersistentCollection.java:635)
  ↳ ~[hibernate-core-6.1.7.Final.jar:6.1.7.Final]
  at org.hibernate.collection.spi.AbstractPersistentCollection.withTemporarySessionIfNeeded(AbstractPersistentCollection.java:218)
  ↳ ~[hibernate-core-6.1.7.Final.jar:6.1.7.Final]
  at org.hibernate.collection.spi.AbstractPersistentCollection.initialize(AbstractPersistentCollection.java:615)
  ↳ ~[hibernate-core-6.1.7.Final.jar:6.1.7.Final]
  at org.hibernate.collection.spi.AbstractPersistentCollection.read(AbstractPersistentCollection.java:136)
  ↳ ~[hibernate-core-6.1.7.Final.jar:6.1.7.Final]
  at org.hibernate.collection.spi.PersistentBag.iterator(PersistentBag.java:366) ~[hibernate-core-6.1.7.Final.jar:6.1.7.Final]
  at java.base/java.util.Spliterators$IteratorSpliterator.estimateSize(Spliterators.java:1865) ~[na:na]
  at java.base/java.util.Spliterator.getExactSizeIfKnown(Spliterator.java:414) ~[na:na]
  at java.base/java.util.stream.AbstractPipeline.evaluate(AbstractPipeline.java:470) ~[na:na]
  at java.base/java.util.stream.AbstractPipeline.evaluate(AbstractPipeline.java:574) ~[na:na]
  at java.base/java.util.stream.AbstractPipeline.evaluateToArrayNode(AbstractPipeline.java:260) ~[na:na]

```



```
at java.base/java.util.stream.ReferencePipeline.toArray(ReferencePipeline.java:616) ~[na:na]
at java.base/java.util.stream.ReferencePipeline.toArray(ReferencePipeline.java:622) ~[na:na]
at java.base/java.util.stream.ReferencePipeline.toList(ReferencePipeline.java:627) ~[na:na]
at be.pxl.demo.service.PostService.getPost(PostService.java:39) ~[classes/:na]
...
```

In de methode `getPost()` in onze `PostService` worden de reacties op onze post 'lazy loaded', maar op dat moment is er geen sessie (of transactie) actief. De juiste oplossing is om een transactie te definiëren voor de methode `getPost()` in de `PostService`. Het is ook mogelijk om de relatie als `EAGER` in plaats van `LAZY` te definiëren, maar dit is zelden een goede oplossing.

```
package be.pxl.demo.service;

import be.pxl.demo.api.dto.PostCommentDTO;
import be.pxl.demo.api.dto.PostDTO;
import be.pxl.demo.domain.Post;
import be.pxl.demo.domain.PostComment;
import be.pxl.demo.exception.ResourceNotFoundException;
import be.pxl.demo.repository.PostCommentRepository;
import be.pxl.demo.repository.PostRepository;
import org.springframework.stereotype.Service;

import java.util.List;

@Service
public class PostService {

    private final PostCommentRepository postCommentRepository;
    private final PostRepository postRepository;

    public PostService(PostCommentRepository postCommentRepository,
                       PostRepository postRepository) {
        this.postCommentRepository = postCommentRepository;
        this.postRepository = postRepository;
    }

    @Transactional(readOnly = true)
    public PostDTO getPost(long postId) {
        Post post = postRepository.findById(postId)
            .orElseThrow(() -> new ResourceNotFoundException("Post",
                ↪ "id", postId));
        return new PostDTO(post.getTitle(),
            post.getCommentList().stream().map(PostComment::getReview).toList());
    }
}
```

Meer info over dit onderwerp vind je in de volgende artikels:

- <https://www.baeldung.com/spring-open-session-in-view>

- <https://techiemanas.wordpress.com/2020/12/01/spring-open-session-in-vieu-osiv/>